The Guidebook

Market making, backtests and this repo, with the Qminers Algo Trade Arena as the case study

2026-11-04

Table of contents

Preface: how to read this 7

The game 8

What happens in a round? 9
- The story . . . 9
- The idea . . . 10
  - A round is a loop . . . 10
  - Four rehearsals, four different things that went wrong . . . 11
  - What this rehearsal teaches . . . 12

What is an exchange? 13
- The story . . . 13
- The idea . . . 14
  - The order book . . . 14
  - Taking and making . . . 14
  - Matching in full . . . 15
  - One order, worked through . . . 16
  - Position limits, and how exchanges enforce them . . . 16
  - Profit and loss . . . 17
  - What the data looks like . . . 17
  - Reading a claim about the book . . . 18

Why market making pays, and when it doesn't 20

What is the price? 21
- The story . . . 21
- The idea . . . 22
  - The mid is noisy . . . 22
  - The walls know more . . . 22
  - Noise against signal: the signature plot . . . 23
  - The micro-price . . . 23
  - Smoothing, and what a filter would choose . . . 24
  - Robust beats clever . . . 24
  - What the fair is for . . . 25

Why does a market maker earn anything? 27
- The story . . . 27
- The idea . . . 28
  - The spread is a fee for immediacy . . . 28
  - One tick of the template's quotes . . . 28
  - What can go wrong: the price moves after you trade . . . 30
  - Measuring the margin before writing a bot . . . 30

So where does the income come from? . . . 31
Competition is the spread's other side . . . 31

Who is on the other side? 34
The story . . . 34
The idea . . . 34
The fills you get are the ones you didn't want . . . 34
Reading a markout curve . . . 36
Why there is a spread at all . . . 36
Two kinds of market . . . 37
Being picked off . . . 37
When someone is informed, follow them . . . 37

What does holding a position cost? 41
The story . . . 41
The idea . . . 41
A position is a bet nobody chose . . . 41
Skew: leaning the quotes . . . 43
Avellaneda–Stoikov in words . . . 43
Is the position a signal or the flow? . . . 44
Why the skew lost here . . . 44
The skew a famous model prescribes . . . 44
The other levers, a line each . . . 45

What do the prices do? 48
The story . . . 48
The idea . . . 49
Every pattern has a timescale . . . 49
The shapes a price takes, and the notes that find them . . . 50
When the past stops describing the day . . . 51
What a regime change costs: the stress test . . . 52

Beyond plain market making 55

What else can a bot trade? 56
The story . . . 56
The idea . . . 56
A target instead of a quote . . . 56
Levels . . . 57
Baskets and their spread . . . 57
Options in a page . . . 59
What paid on Prosperity's options . . . 60
Copying the informed, and fading jumps . . . 61
Edges a replay can't value, and edges that aren't allowed . . . 61

Who else is in the market? 63
The story . . . 63
The idea . . . 63
Alone, or in one market . . . 63
What a second bot takes . . . 64
Which setting is best depends on who else is there . . . 64

The herd: what an LLM writes . . . 65
Bots built to hurt . . . 66
Gray areas, and the gate . . . 66
The field is noisy too . . . 66

How we know **69**

Why is my backtest lying? **70**
The story . . . 70
The idea . . . 70
The question a replay can't answer . . . 70
What the market offers at each distance . . . 71
Why the error isn't noise . . . 72
Where we stand in the queue . . . 72
The other things a replay assumes . . . 73
What to do about it . . . 74

How do I know a result is real? **76**
The story . . . 76
The idea . . . 76
A grid is a hyperparameter search, with very few folds . . . 76
Hold out days, not ticks . . . 77
One grid, read in full . . . 77
PBO: does the in-sample best stay good out of sample? . . . 77
The DSR: is the best Sharpe ratio better than the best of that many with no edge? 79
SPA and StepM: does any cell, and which, beat the bot as is? . . . 79
Why we read them at 4% . . . 80
The pick . . . 80
The permutation test: does the choice need the order of events? . . . 80
When the profit is on one day . . . 81

What does the portal tell me? **83**
The story . . . 83
The idea . . . 83
The twelve figures . . . 83
The score is an identity, if you know what is summed . . . 84
The standard error says how much the score is worth . . . 84
The position quantiles say how hard the bot leans on its limits . . . 85
Calibrate: fitting the simulator to the real thing . . . 85
Refusing a bad fit . . . 86

How should I size risk? **88**
The story . . . 88
The idea . . . 88
What the score rewards . . . 88
Rank is not expected profit . . . 89
Kelly: size by edge over odds . . . 89
Value at risk: size the bet to the lead . . . 91
The tail you can't backtest . . . 91
Round weights and carry-over . . . 92

The repo **94**

How is the code laid out? **95**
The story . . . 95
The idea . . . 95
The repo on one page . . . 95
The engine: one loop over ticks . . . 96
Formats are plug-ins . . . 97
The bot: PARAMS, then three sections . . . 97
One tick, followed through . . . 98
The tools, the twins and the notes . . . 99

What does each parameter do? **101**
The story . . . 101
The idea . . . 101
How to read an IDEAS row . . . 101
The settings, group by group . . . 102
Changing one on the day . . . 108

What did we do on the day? **110**
The story . . . 110
The idea . . . 110
How the plan was made . . . 110
The weekend . . . 111
The morning: settle what changes the code . . . 111
Why each round runs as it does . . . 112
The rules that hold all day . . . 112
Each round, choose again . . . 113
Risk, once there is a score to protect . . . 113
The rounds as they went . . . 113

Appendices **115**

Appendix A: Glossary **115**

Appendix B: The datasets **119**
Prosperity 2, 3 and 4: the stand-in . . . 119
`arena.sim`: a market that reacts . . . 120
ABIDES: a market we didn't write . . . 120
LOBSTER: a real book, still to come . . . 121
The mock rounds: rehearsals in disguise . . . 121
The Arena . . . 121
The zoo is bots, not data . . . 121

Appendix C: The commands on one page **123**

Appendix D: The reading route **125**
1. The map: the wiki's page on *Trades, Quotes and Prices* . . . 125
2. Harris, *Trading and Exchanges*, chapters 13 and 14 . . . 125
3. Bouchaud and colleagues, chapter 16's first two sections and chapter 17 . . . 125
4. Avellaneda and Stoikov, *High-frequency trading in a limit order book* . . . 126
5. Signals: Stoikov's micro-price, and the first three sections of Bouchaud's chapter 7 . . . 126

6. Adverse selection measured: the first three sections of Bouchaud's chapter 11, then DeLise 126
7. Bouchaud's chapter 21, its third and fourth sections 126
8. Validation: *Pseudo-mathematics*, the wiki, then PBO 126
9. Then, as needed 127

Appendix E: The Arena, round by round **128**
What each round's section will hold 128
Round 1, Saturday 13:00–18:00 128
Round 2, Saturday 19:00–24:00 128
Round 3, Sunday 01:00–06:00 128
Round 4, Sunday 07:00–12:00 129

Preface: how to read this

This is the long version of the field guide: the same questions, each answered with its evidence, six more chapters on what the field guide left out, and the Qminers Algo Trade Arena of 7 and 8 November 2026 as the case study that runs through all of them. It assumes what an engineer with machine learning behind them already knows (probability, overfitting, hyperparameter search, simulators) and builds the rest from zero.

This is a working draft, begun on 4 November 2026, before the event. A box headed "On the Arena" marks what is written only after the event, from each round's handover note; until then it says so.

The book has five parts:

- **The game:** a round from start to finish, and what an exchange is.
- **Why market making pays, and when it doesn't:** the price, the spread, who is on the other side, what holding a position costs, and what prices do.
- **Beyond plain market making:** what else a bot can trade, and who else is in the market.
- **How we know:** why backtests lie, how to tell a real result from luck, what the portal says, and how to size risk.
- **The repo:** how the code is laid out, what each parameter does, and what we did on the day.

Every chapter answers one question and opens on something that happened: in a Prosperity competition, in a past Qminers hackathon, in one of our rehearsals, in the literature, or at the Arena. Then comes the idea in plain words, at most one or two formulas with a sentence that says each aloud, and the boxes: **If you know ML** (the analogy, and where it breaks), **What would differ on another market** (in Parts II and IV), **In the repo**, **On the Arena**, and **Go deeper** (the literature, with pages).

Five markets recur, and they are kept apart. *Prosperity* is IMC's competition, whose data the prep used as a stand-in; its prices are in credits. `arena.sim` is our simulator. *ABIDES* is an independent simulator with informed traders; its prices are in cents. *The mock rounds* are disguised rehearsals with their own portals; the Varde Exchange scores in euros. *The Arena* is the real thing. Nothing measured on one is stated as a fact about another.

Every number can be checked. A figure comes from `scripts/make_data.py`, which runs a committed command and writes the CSV in `data/` that the figure is drawn from; a number in the text names its `IDEAS.md` row or its source. A figure from a team's own write-up that our engine did not reproduce is marked *their claim*.

The audio edition reads the same text. Each formula is read as its sentence, a table as its summary, and a figure as a short description of it.

The game

What happens in a round?

You'll need nothing yet. This chapter is a tour: it uses, loosely, most of the terms the rest of the book defines, so that you have the map before the detail. Don't stop on a word you don't know; its chapter is named next to it.

The story

Just after midnight on 3 November we rehearsed round 1 against an exchange none of our tools had seen, the Varde Exchange. It was built to be unlike anything in the repo: its own data layout, its own way of calling the bot, prices in cents on a five-cent step, results in euros, and a portal that matched and scored orders in its own code, not in our engine. A fresh agent session got the prompt it will get on the day, a deadline an hour away and no help. Its log gives every step a clock time (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`).

- **0:00.** It read the rules in full. Three products, LUMEN, QUILL and SORREL, each with a *position limit* of 40, 60 and 30 lots: how much of a product the bot may hold, long or short (chapter 2). Orders *rest* on the book until they are filled or cancelled. A fee is charged per lot traded and per order placed. Three public sessions of 5,000 steps to practise on; the score comes from two sessions nobody sees.
- **0:02.** It wrote down what the rules settle, what they leave open (the price at which open positions are valued at the end, among others) and where they contradict how our engine assumes a market works. That list is the first thing in the round's handover note (`python/experiments/mock_round/round4/results/HANDOVER.md`).
- **0:03.** It wrote a *format plug-in*, the piece of code that reads this exchange's files into the engine's own shape (chapters 2 and 14), and ran `arena.diagnose`, which describes each product from the data alone: how wide its *spread* is (chapter 2), whether its price wanders or comes back to a level (chapter 7), whether its trades move the price (chapter 5). A minute later a second tool found that LUMEN and QUILL open at the same level every session and return to it.
- **0:05.** It wrote the bot: the repo's *template*, with a block of settings, `PARAMS`, for each product (chapter 15), and an *adapter* that speaks this exchange's language (chapter 14). Its first *backtest*, a replay of the public sessions with the bot trading in them (chapter 10), made 73,038 cents, about 730 euros. SORREL alone lost 567 euros by quoting.
- **0:06.** A *grid*: a few settings in every combination, each backtested, under two *fill models*, two answers to what would have happened to orders the recording never saw (chapters 10 and 11). SORREL set to take only, which in practice meant it never traded, lifted the backtest to 133,240 cents under the optimistic model and 124,872 under the strict one, and the grid's own pick agreed. `arena.validate` then checked the file against the rules: its size, 60 kilobytes; its slowest tick, a tenth of a millisecond; no outside libraries.
- **0:07.** The upload. The portal's script crashed on the laptop's old Python; it ran under the repo's own. The portal scored the bot **€367.61** on the three public sessions and returned twelve figures and a file of every fill.

- **0:09.** *calibrate* searched 144 of our engine's settings for one that reproduces the portal's twelve figures (chapter 12). It **refused**: no setting came within 5%, and the least bad was 98% off on the score. The session did what the refusal says. It pasted nothing into the engine, rebuilt the score by hand from the fills file, and found two things. A fee of half a cent per order placed had taken €143.24, 28% of the €510.85 the bot made before the order fee. And the portal had filled far fewer lots than any of our settings: 8,466, against 11,345 to 18,772.
- **0:13.** It wrote what it had learned into the repo's notes, and stopped, as the prompt tells it to, after one validated upload and an honest reading of it. The whole run took fourteen minutes.

On the two hidden sessions the bot scored **€208.77**; the example bot the exchange handed out scored **€80.10** (`python/experiments/mock_round/round4/README.md`, re-run with its `portal.py hidden`). The cause of the fill gap was in the rehearsal's answer key, not in anything the session could read: only half of each trade reached resting orders. The session didn't pin that down in fourteen minutes, but it knew not to trust its engine until someone did.

The idea

A round is a loop

The Arena gives five hours a round and a limited number of uploads. Inside those, the work is a loop (Figure 1), run as often as time and submissions allow.

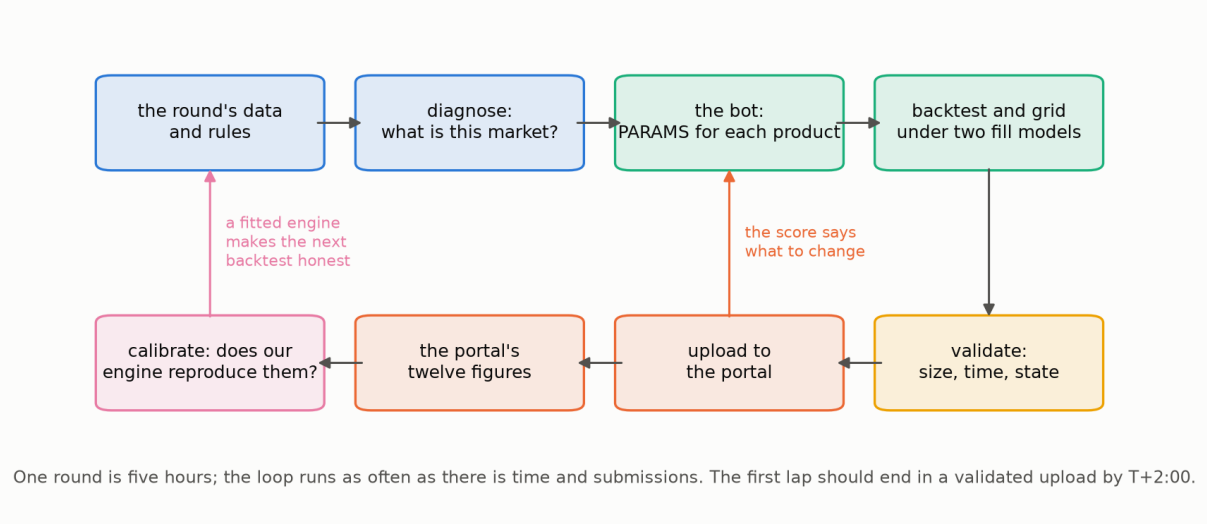


Figure 1: The round's loop. The first lap ends in a validated upload by two hours in (`RUNBOOK.md`, "Each round").

1. **Read the market.** The data is a recording: the *order book* at every *tick* (who is offering to buy and sell, at what prices, how much) and the trades that happened (chapter 2). The first question is what kind of market each product is: where its price is (chapter 3), whether it comes back to a level or wanders (chapter 7), who trades it (chapter 5).
2. **Write the bot.** A bot is called once a tick, sees the book and its own position, and returns orders. Ours is mostly a *market maker*: it offers to buy a little below its estimate of the fair price and to sell a little above it, and earns the gap when others trade with it (chapter

4). It loses when the people trading with it know more than it does (chapter 5), and it pays for whatever it is left holding (chapter 6).
3. **Backtest.** Replay the recorded days with the bot in them. The replay has to guess what would have happened to orders that were never really there, and that guess can be worth more than the strategy (chapter 10).
4. **Choose settings without fooling yourself.** A grid tries many settings, and the best of many is flattered by luck; a round's few days of data are very few (chapter 11).
5. **Upload, and read what comes back.** The portal is the only judge that matters. Its figures say how far our replay is from the real exchange, and calibrate turns that gap into engine settings, or refuses to (chapter 12).

Two habits keep the loop honest. **Upload early:** a validated bot by two hours in, so that there is always something scored and the engine's error is measured before it is relied on. **Change only what the evidence supports:** each change has to beat the bot before it on days the choice didn't see (the kickoff prompt's step 5), because an upload is one noisy measurement and submissions are few.

Four rehearsals, four different things that went wrong

Varde was the fourth disguised round. Each was built from our simulator's products, renamed and repriced, with a layout, a bot interface and rules unlike the round before. Their scores are in each round's own units, so they compare only within a round (`IDEAS.md`, "The kickoff prompts on a disguised round").

- **The first** (26 October) gave a function for an API, orders that rest, and one product whose price jumped and snapped back. Both sessions, one with the repo and one in an empty folder writing its own backtester, found the jump in minutes, and about 89% of each one's profit came from fading it: 42,446 on the hidden days for the repo's bot, 42,944 for the clean-room one. It also showed that whether the volume a bot buys stays gone from the replayed book is worth a quarter of the score: 31,172 when it does (chapter 10).
- **The second**, the Halden Exchange (28 October), tested whether reorganising the repo made a session faster. It didn't: both runs spent most of their thought mapping the rules onto the engine. The first run's bot made 9,777 on the hidden days, between the round's own market maker at 8,632 and its reference bot at 12,195.
- **The third**, the Kestrel Bourse (30 October), hid four rules, among them that orders reach the exchange a step after the bot sends them. Both uploads lost: −3,553 and −489 credits on the hidden sessions, where the round's plain market maker made +325. The first loss was the adapter's: it didn't count the orders still on their way, so it re-sent its quotes every step and overfilled its limits, and 2,191.75 credits of the loss were charges for the breaches (chapter 14). Counting those orders moved the second bot only from −489 to −389; the rest came from quoting one product whose quotes the round's strict matching filled only when the price ran through them, a choice of settings, not of code.
- **The fourth**, Varde, put the matching and the scoring outside our engine altogether, and is the story above.

In none of the four did a clever strategy decide the result. The first rewarded reading each product right; the second and the third's worst loss were decided by the adapter, the thin layer between our code and theirs; the fourth by rules our engine couldn't express, one of them unstated. That is the pattern this book keeps returning to.

What this rehearsal teaches

The most useful moment of the Varde run was the refusal. A tool that always gives an answer would have handed over the least bad setting, 98% off, and every backtest of the next five hours would have been tuned to an engine that wasn't the exchange's. A tool that prints "NO SETTING REPRODUCES THE PORTAL within 5%: don't paste an ENGINE.", and says where to look (`python/experiments/day/calibrate.py:298`), sent the session to the one place the truth was, the portal's own fills. It found the fee there in three minutes.

Most of this book is about that distinction: what our tools can measure, what they can only assume, and how to tell the two apart under time pressure. The rest is the market itself, which is where Part II begins, after one more chapter on what an exchange is.

If you know ML

The loop is model development against a black-box evaluator: a local validation set (the backtest), a hidden test set (the scored sessions) and a limited number of test-set queries (the uploads). Two things break the analogy. The local validator is a simulator with a known bias, not a held-out sample of the same data, so a model that wins locally can lose on the portal for reasons no amount of local data reveals (chapter 10). And every query costs one of a few submissions and blocks the next until it is scored, so each upload is planned like an experiment.

In the repo

- The rehearsal: `python/experiments/mock_round/round4/` (its README rebuilds and scores it in seconds); the run's own record, `results/DRYRUN_LOG.md` and `results/HANDOVER.md`. The earlier three: `python/experiments/mock_round/`, `round2/` and `round3/`.
- The loop's tools: `arena.diagnose`, `arena.backtest`, `arena.grid`, `arena.validate`, `experiments.day.calibrate`.
- The prompt the session got: `KICKOFF_PROMPT.md`, round 1. The results: `IDEAS.md`, "The kickoff prompts on a disguised round".

On the Arena

Written after the event, from the round handovers: the Arena's round 1 as it went, step by step against the same clock, from `data/arena/round1/HANDOVER.md` and its upload log, with a figure of the round's timeline, and what it did differently from the Varde rehearsal.

Go deeper

- Harris, *Trading and Exchanges*, ch. 13 "Dealers" (from PDF p. 291): what a market maker is, in plain prose, before any of the models.

What is an exchange?

You'll need nothing before this chapter. It defines the order book, orders, matching, fills, the position limit, profit and the data an exchange publishes, which every later chapter uses.

The story

The Varde rehearsal's first upload (chapter 1) scored €110.21 on its first public session. The portal showed totals and a file of fills, and the session rebuilt the rest by hand. The portal's own scoring code, run on the same bot, shows where the number comes from. It is two large numbers that nearly cancel (`data/sessions.csv`, from the rehearsal portal's own scoring):

- **cash: −€2,814.63.** Over the session the bot spent more on buying than it received from selling, fees included.
- **the position, valued: +€2,924.84.** At the end it still held what it had bought and not sold, 17 lots net across its products, and the portal valued them at its *settlement price*, the price it chooses for what is still held.
- **the score: €110.21,** their sum.

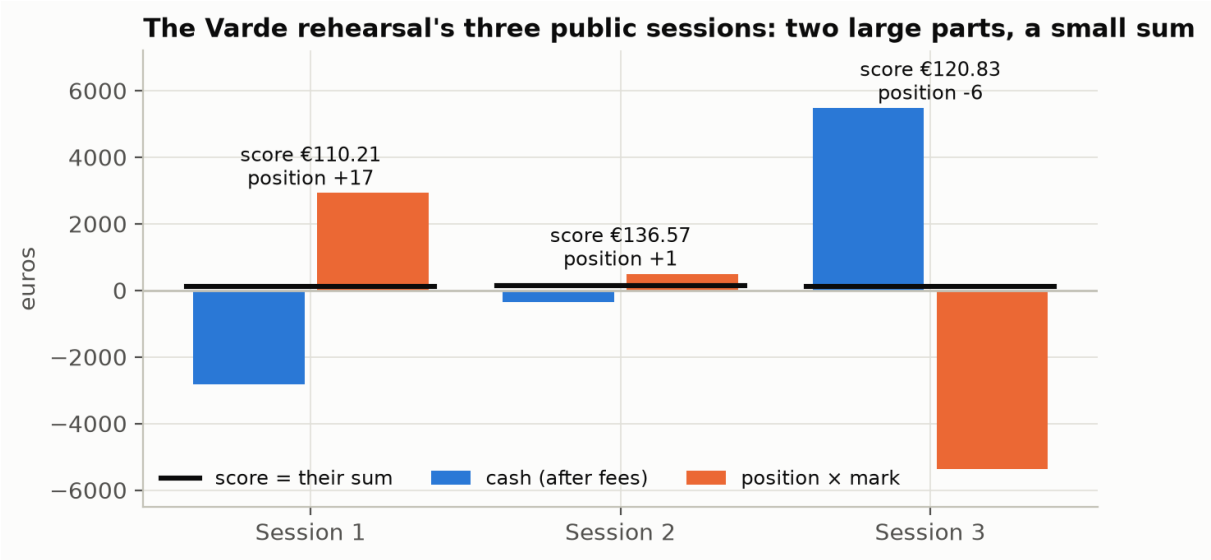


Figure 2: The score's two parts in each of the Varde rehearsal's three public sessions. Data: `data/sessions.csv`, from `python/experiments/mock_round/round4/private/portal.py`'s scoring of the uploaded bot.

All three sessions are in Figure 2. In the third the signs flip: €5,486.92 of cash against a short position worth −€5,366.09, for a score of €120.83. In every session the score is a small difference between two numbers, and one of them depends on a price the portal chose. The session didn't know which: valuing the positions at each session's

last mid gave €369.67 against the portal's €367.61, close enough to use and not exact (`python/experiments/mock_round/round4/results/HANDOVER.md`, "Portal against local"). The answer key says the portal used the average price of each product's last 100 trades, weighted by size. Hold that thought: it is why the end of a session matters more than any single tick, and why chapter 10 counts the price used to value a position among a backtest's assumptions.

The idea

The order book

An exchange is a place where offers to trade wait until someone takes them. Each offer is a *limit order*: buy up to this many *lots* (units of the product, the smallest quantity it trades in) at this price or lower, or sell up to this many at this price or higher. The offers waiting at a moment make up the *order book*: the *bids* (offers to buy) on one side, the *asks* (offers to sell) on the other, each side grouped into *levels*, one per price (Figure 3).

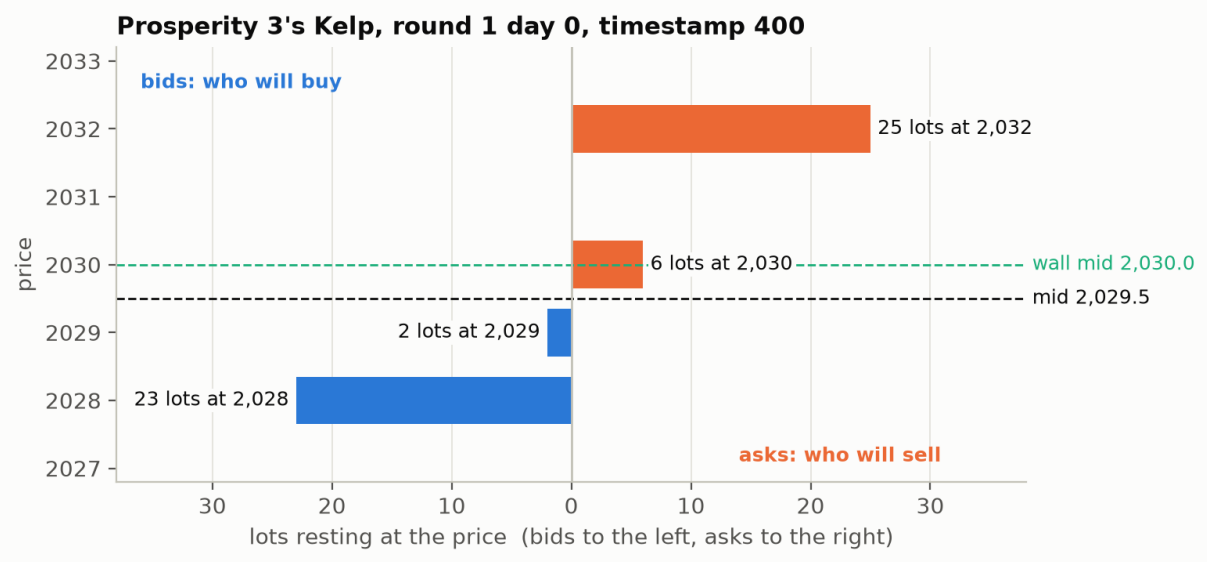


Figure 3: One tick of Prosperity 3's Kelp, round 1, day 0. Data: `data/order_book.csv`, from the engine's loader.

The highest bid is the *best bid* (2,029 here); the lowest ask is the *best ask* (2,030). Together they are the *touch*, and the gap between them is the *spread*, here 1 credit. Prices move in fixed steps, the *tick size*: 1 credit on Prosperity, 5 cents on the Varde Exchange. A spread of one tick is as narrow as a book can be. The word *tick* has a second sense, which the book also uses: one step of the market's clock, one snapshot of the book and one call of the bot; the context says which. Halfway across the touch is the *mid*, 2,029.5. Chapter 3 is about why the mid is often the wrong number to trust, and what the large orders at 2,028 and 2,032, the *walls*, tell you instead.

Taking and making

A new order either crosses the book or it doesn't.

- A **buy at or above the best ask** (or a sell at or below the best bid) *crosses*. It trades at once with the waiting orders, at their prices, best first, until it is filled or the next level is

beyond its own price. This is *taking*, and the order is *aggressive*: you pay the spread for trading now.
- An order that **doesn't cross** joins the book and waits. This is *making*, or quoting, and the order is *passive*: you offer liquidity and hope someone takes it. When they do, your order is *filled*, at your price.

A trade between two other participants is a *print*, and the side that crossed is its *aggressor*. Chapters 4 and 5 are about why the passive side earns the spread on average, and when it doesn't.

Matching in full

The rule that decides who trades with whom is the exchange's *matching*. Nearly every exchange uses *price-time priority*: an incoming order trades first with the best price on the other side, and among orders at the same price, with the one that arrived first. So a waiting order has a place in a *queue*. Joining a busy level means waiting behind everyone already there, and you move up only as the orders ahead of you trade or are cancelled. Prosperity's own glossary says its exchange enforces exactly this (`references/prosperity-wikis/prosperity4/trading-glossary-d14f71.md:69`).

On Prosperity, though, the queue hardly exists, because nothing waits long enough to be in one. Our engine follows its rules, checked fill for fill against the open-source backtester (`python/arena/engine.py`, the module's docstring; `ARCHITECTURE.md`, "Matching rules (Prosperity semantics)"):

1. Every tick the bot sees the book and returns orders.
2. If a product's orders could take its position past the limit, every order for that product is rejected (below).
3. Each order trades against the visible book at the book's prices, best price first.
4. What's left can fill against that tick's prints, at the order's own price.
5. Anything still unfilled is cancelled. No order rests into the next tick.

Rule 4 is the one a replay has to guess at, because the prints happened between other bots: whether a print at our price would have come to us is the *fill model*, and chapter 10 is about how much rides on it. Rule 5 is the likely reason one Prosperity 4 team's probe found "no queueing": orders at the best price were served regardless of when they were placed (*their claim*) (`references/online-snapshots/Leo-Hawking_IMC-Prosperity-4-Review.README.md:204`).

Most other exchanges, and all four of our rehearsals, let orders **rest** until they are filled or cancelled. The engine's resting mode replaces rule 5 with an exchange where they do (`python/arena/engine.py`):

- What doesn't trade at once rests at its price, at the back of its level's queue: the volume the book already shows there is ahead of it. An order marked *immediate-or-cancel* doesn't rest.
- Prints at its price first use up the volume ahead of it, then fill it. A print through its price fills it at once, since it would have traded first.
- If a later book shows the other side at or through its price, those orders would have hit it, and it fills at its own price.
- The queue moves up when the level shrinks by more than the prints explain, because orders ahead were cancelled; how much of that happens ahead of us is a guess (chapter 10).
- The bot's own orders never trade with each other.

Each exchange then adds its own details. The Varde Exchange applied a step's cancels before its new orders, took at most 10 actions a step, had no immediate-or-cancel orders, and filled resting orders only from prints, best price then oldest first (`python/experiments/mock_round/round4/public/rules.md`, "Your bot" and "Matching"). The Kestrel Bourse allowed 6 actions a step, cancels included, and 3 working orders a product. Each of those is a line in the format's engine settings, and a line the bot's adapter has to respect.

One order, worked through

Take the book of Figure 3, at timestamp 400, and send one order: buy 10 Kelp at 2,031 (`data/worked_order.csv`, from `arena.engine.run_day`).

- The order crosses: 2,031 is above the best ask of 2,030. It takes the 6 lots there, at 2,030, the book's price, not its own. The next ask, 2,032, is above its price, so the taking stops.
- Four lots are left at 2,031, the highest bid in the book now and alone at their price.
- Under Prosperity's rules, no print that tick reaches 2,031, so the four lots are cancelled. The bot bought 6.
- With orders resting, the four lots wait. At the next tick, 500, the book's best ask has dropped to 2,031, at our price: those sellers would have traded with our bid, so it fills, 4 lots at 2,031.

The same order, the same data, two exchanges: 6 lots or 10. Nothing in the recording says which happened, only the rules do.

Position limits, and how exchanges enforce them

Your *position* in a product is what you hold: positive after buying more than you sold (*long*), negative after selling more than you bought (*short*; the exchanges in these competitions let you sell what you don't hold). Each product has a *position limit*, and how it is enforced differs more than you would guess.

- **Prosperity** rejects the whole tick for a product. If all of a tick's buy orders for Kelp, filled, could take the position past 50, or all its sells past -50, every Kelp order that tick is rejected, the sells with the buys (`ARCHITECTURE.md`, "Matching rules (Prosperity semantics)", rule 2). A bot that quotes its whole room on both sides can lose all its orders for a tick by miscounting one lot.
- **The Varde Exchange** did the same, counting what already rests: if a step's new orders could pass the limit, all that step's new orders for the product were rejected, while the resting ones stayed (`python/experiments/mock_round/round4/public/rules.md`, "Limits, fees and errors").
- **The Halden Exchange** checked each order on its own, in the order the bot sent them, and rejected only the ones that didn't fit (`python/experiments/mock_round/round2/public/rules.md`, "How each step is processed").
- **The Kestrel Bourse** rejected nothing. After every step it charged 0.25 credits for each lot of a position beyond its limit. The first bot to trade there didn't count the orders still on their way, overfilled its limits, and paid 2,191.75 credits in charges on the two hidden sessions, toward a score of $-3{,}553$ credits (`data/kestrel_breach.csv`, from the round's `portal.py hidden`).

- **AlgoTrade 2026**, a live competition in Zagreb whose rules, one team's code and its recorded data are in `references/`, set different limits for each side: 2,000 units long, 200 short (`references/algotrade2026-pitfa19/.firecrawl/algotrade-api.md:687-688`).

A rule that rejects is a cliff, and a rule that charges is a slope. Under the first the bot must never send what could breach; under the second a breach is a cost to weigh. Chapter 10 shows the cliff's worst case, orders that arrive a tick late and are checked again against a position their own earlier fills have moved.

Profit and loss

PnL is what a bot would have if it closed everything now:

$$\text{PnL} = \text{cash} + \text{position} \times \text{price}$$

Cash goes down by the price of everything bought and up by everything sold; fees come out of it. The price that values the position is the *mark*: our engine's default is the mid, Prosperity's own data carries one, and the Varde portal used its 100-trade average (`python/arena/engine.py`, `--mark`). The story's identity is this formula, session by session.

What the data looks like

Every exchange records the same market differently, and the repo meets three shapes (Figure 4).

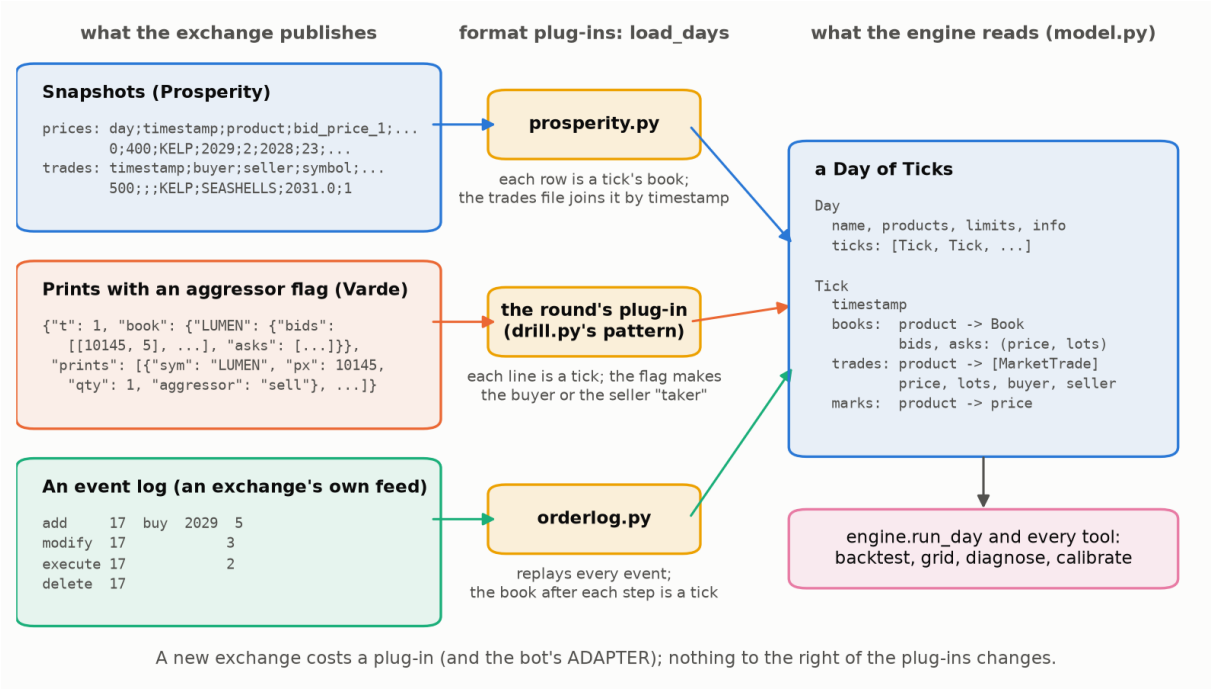


Figure 4: The three shapes of exchange data, each read by its format plug-in into the engine's one form, a `Day` of `Tick`s. The samples are real rows: Prosperity 3's Kelp at timestamps 400 and 500, the Varde rehearsal's first steps; the event log is illustrative. Diagram; the types from `python/arena/model.py`.

- **Snapshots** (Prosperity): one row per tick and product with three levels a side, 10,000 ticks a day, and a separate file of the trades between ticks, with buyer and seller names when the round reveals them. The plug-in reads each row as a tick's book and joins the trades to it by timestamp.
- **Prints with an aggressor flag** (the Varde Exchange): one line per step holding the book and the step's prints, each marked with the side that crossed. The plug-in keeps the flag by naming the crossing side `taker` in the trade's buyer or seller field, so a tool can ask who was in a hurry.
- **An event log** (LOBSTER, and many real exchanges' own feeds): every order added, changed, cancelled or executed, by id. The plug-in replays the events and takes the book as it stands after each step as a tick, so any snapshot can be rebuilt from it. (Our LOBSTER and ABIDES days are converted into Prosperity's layout before they are loaded.)

Whatever the shape, the plug-in's `load_days` returns the same thing: a `Day`, with its products, limits and per-product facts such as the tick size, and its `Ticks`, each with a timestamp, a `Book` of bids and asks per product, the prints, and the prices to value positions at (`python/arena/model.py`; `python/arena/formats/`). The engine and every tool work on that, never on the exchange's own files. That is why a new exchange on the day costs a plug-in and an adapter, not a new set of tools, and why the Varde session had its plug-in written three minutes in.

Reading a claim about the book

A book shows a few levels. Is that the market? A team in AlgoTrade 2026 looked at where trades printed on one of its exchanges, ZSE, and found "53.9% of all ZSE trades print outside the visible top-3", which they read as the exchange's own market maker quoting at levels 4 and 5 (`references/algotrade2026-pitfa19/analysis/mm_analysis.md:153`). Two things in their own files change what that number means. It came from the 3,115 prints they had when they wrote it (*their claim*) (`mm_analysis.md:43`). And "visible" meant visible in their recording, which kept three levels a side, while the exchange sent every team five (`mm_analysis.md:48-49`; `.firecrawl/algotrade-api.md:518-546`). So what they measured was mostly what their recorder dropped, not depth that nobody could see.

We re-ran their test on the 1.4 million prints committed with their analysis. Each print against the last recorded snapshot before it gives 34.0%, from 12 to 21% on the exchange-traded funds to 48% on one stock. But a book moves between snapshots: counting only prints outside both the snapshot before and the one after gives 5.7% (`data/hidden_depth.csv`, from the committed data). Neither number says how deep the real book was, because the recording can't. On Prosperity 3's round 1, whose data is what the bots were shown, a similar check, against the books before and at each print, finds no print outside the levels shown, on any of its three products; 3.4 to 3.8% of its trades hit a price with no shown level, about half of them inside the spread (`data/hidden_depth.csv`; `experiments.day.structure --data p3 --days 1 --sections fingerprints`). The lesson is about reading, not about depth. Before trusting a figure, ask what sample it came from and what the data it was measured on left out. And a number from someone's write-up is a claim until it is re-run.

> **i** If you know ML
>
> The order book is the state, a tick is a time step, and the bot is a policy from state to actions; the format plug-in is a data loader that maps every source to one schema. Where the analogy breaks: the state is partly made of other agents' intentions, which change when you act, and a replay freezes them (chapter 10). The environment's transition rules,

matching and limits, differ between exchanges in ways that matter more than most model choices, and they are given in prose, not code. And the reward, PnL, is defined only after a choice of price for what you hold, like a metric that needs a reference you have to pick.

In the repo

- The internal form: `python/arena/model.py` (`Book`, `Tick`, `Day`, `Order`, `Cancel`, `Fill`, `MarketTrade`).
- The matching rules: `ARCHITECTURE.md`, "Matching rules (Prosperity semantics)" and "Resting mode"; the engine's docstring, `python/arena/engine.py`. Flags: `--resting`, `--queue`, `--mark`, `--limit-penalty`, `--max-actions`, `--max-resting`.
- The plug-ins: `python/arena/formats/` (`prosperity.py`, `drill.py`, `snapshots.py`, `orderlog.py`, and the day's `qminers.py`); `ARCHITECTURE.md`'s paragraph "Formats are plug-ins".
- Prints at prices the book didn't show, on any data: `uv run python -m experiments.day.structure --data D --days DAYS --sections fingerprints`.

On the Arena

Written after the event, from the round handovers: the Arena's own format, its data shape and bot interface, and its matching: whether orders rest, where a new order joins the queue, how the position limit is enforced, what its position-limit penalty charges, and how many levels its data shows, from the answers written into the "First hour" section of ON_THE_DAY and the first upload.

Go deeper

- Harris, *Trading and Exchanges*, ch. 6 "Order-Driven Markets", §6.2 (PDF pp. 129–133): order precedence rules, the matching procedure and trade pricing, with worked examples.
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 3 §3.2.1–3.2.3 (PDF pp. 73–75): priority rules, order types and auctions, in a few pages.

Why market making pays, and when it doesn't

What is the price?

You'll need the order book, the touch, the spread, the mid and the walls (chapter 2), and the market maker and the backtest as chapter 1 introduced them. Options appear once and are chapter 8's.

The story

In a backtest of Prosperity 4's round 1, our template traded Osmium in one market with three other teams' published bots, all with resting orders. It lost 371 thousand credits. The cause, traced on one day against one of those bots, was one number.

The template priced Osmium at its *wall mid*, the midpoint of the largest order on each side of the book. At tick 2,400 of day 0 the real wall on the bid side was 23 lots at 9,991. But a rival bot had a 40-lot bid resting at 9,906, far below the touch, and 40 is more than 23. So the template took the rival's order for the wall, put its fair value at 9,957 against the recorded wall mid of 10,001.5, 44.5 below it, and spent the session selling into bids that were worth more than it thought. Its fair was 5 or more off on 36% of ticks (`IDEAS.md`, "The template against the teams' P4 traders in one market").

The fix was a different definition of "the wall": the first level from the top with at least 15 lots, which a large order far from the touch can't capture. With it, the same market made +34 thousand credits instead of −371 thousand (`data/rival_fair.csv`; `IDEAS.md`, "Wall-mid definitions"). That pair comes from the template as it then stood for Osmium, with no smoothing and no stop loss. Today the 15-lot rule is the template's default and Kelp's, while Osmium still uses the largest level, but its fair is slowed (`smoothing` 0.9) and a stop loss ends a bad day early (`python/bots/template.py`, `PARAMS`). With both, the largest-level rule loses only 3,168 credits in the same market and the 15-lot rule makes 70,348 (`data/rival_fair.csv`). Each layer of caution caught part of the same error.

A Prosperity 4 team, Ryan Challman's, met the opposite problem. Their fair value for the round's vouchers, a kind of option, was a running mean that lagged behind the live price. They took the lag for a defect, and early in round 3 tested a textbook option price that tracked the market closely instead. On their training days it lost about 64,000 credits per strike (*their claim*). Every fair that followed the live price more tightly, including a micro-price and a moving average on its own, "collapsed the deep-ITM fills", those of the vouchers deep in the money, whose prices move almost one for one with the underlying's: the lag was the edge (`references/prosperity4-ryan-challman/README.md:266`). In round 4 they found what that kind of edge depends on. While the price rises, a lagging running mean sits below it, so resting sells fill on the rise. On a day that drifted down 65 ticks, the same lag left resting buys above the market: the deepest voucher, the one that tracks the underlying most mechanically, turned to a loss, and the others that track it lost smaller amounts (*their claim*) (`references/prosperity4-ryan-challman/README.md:79-87`).

One fair value was wrong because a single order could move it; the other was right because it was slow. Neither is about finding the true price. Both are about what the number is for.

The idea

A market maker quotes around a *fair value*, its estimate of what the product is worth right now. It buys below that number and sells above it, and everything it earns is measured from it. So the first question on any market is how to estimate it from the book, and the second is how much history to let into the estimate.

The mid is noisy

The mid, halfway between the best bid and the best ask, is the obvious estimate, and on many books it is a poor one. The touch is often made of small orders that come and go: a 2-lot bid appears one tick above the rest, and the mid jumps half a tick without anything having changed. Prosperity's Kelp shows it over 300 ticks (Figure 5).

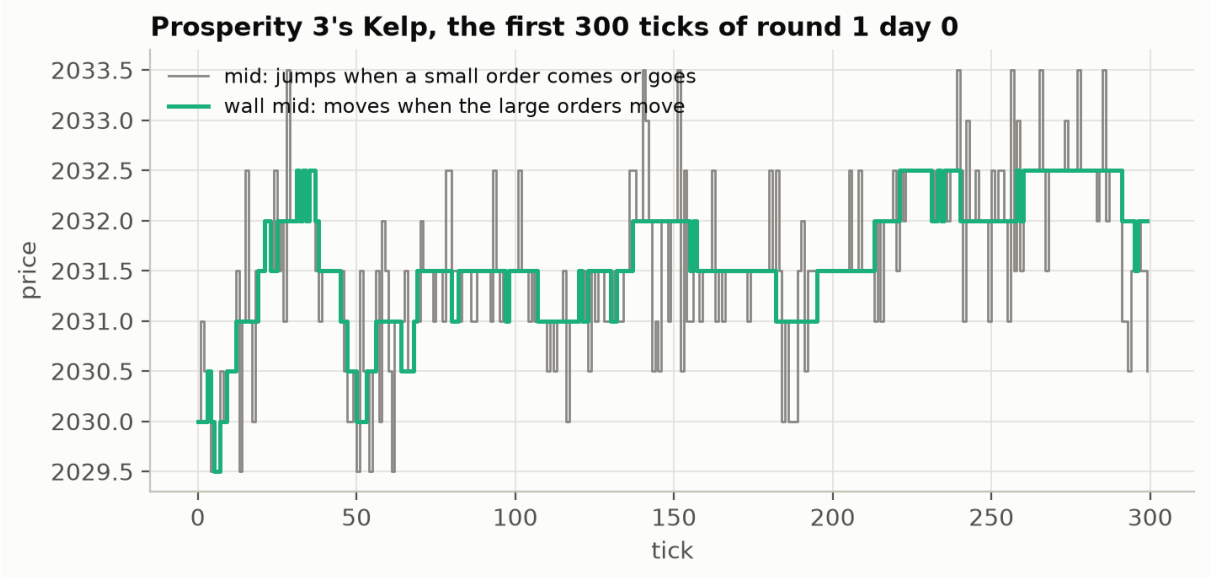


Figure 5: The mid and the wall mid over 300 ticks of Prosperity 3's Kelp. Data: `data/mid_and_wall.csv`, from the engine's loader.

The walls know more

On Prosperity, Frankfurt Hedgehogs found that each product's large orders, the walls, came from market-making bots that "appeared to know the true price and simply quoted rounded versions of it" (*their claim*, inferred from the website's PnL) (`references/prosperity3-frankfurt-hedgehogs/README.md:882-884`). Small orders from other bots sit inside them. So the midpoint of the walls tracks the value far better than the mid does:

$$\text{wall mid} = \frac{\text{bid wall} + \text{ask wall}}{2}$$

Take the Kelp tick of chapter 2's order book: 2 lots bid at 2,029 and 23 at 2,028, 6 lots offered at 2,030 and 25 at 2,032. The mid is 2,029.5. The walls are the 23 and the 25, so the wall mid is 2,030, and the template's rule, the first level from the top with at least 15 lots, picks the same two (`data/order_book.csv`). That is a fact about Prosperity's generator, not a law of markets, and the next sections are about how to check it on new data.

Noise against signal: the signature plot

A price series mixes two things: moves that last, which a fair value should follow, and noise that comes back, which it should ignore. One plot tells them apart without any model. Take the price's change over h ticks, square it, average it, and divide by h. For a random walk, whose moves all last, the variance grows in step with h, so the ratio is flat. Noise that reverts adds the same amount to every change however long, so it inflates the short horizons most and the ratio falls with h. Moves that continue, trends, make it rise. This is the *signature plot* (Bouchaud et al., ch. 2 §2.1.2–2.1.3, eq. 2.8 and Fig. 2.1, PDF pp. 43–46). The book plots its square root, a volatility; this chapter plots the variance itself, so a ratio of 17.3 here is about 4.2 on the book's scale.

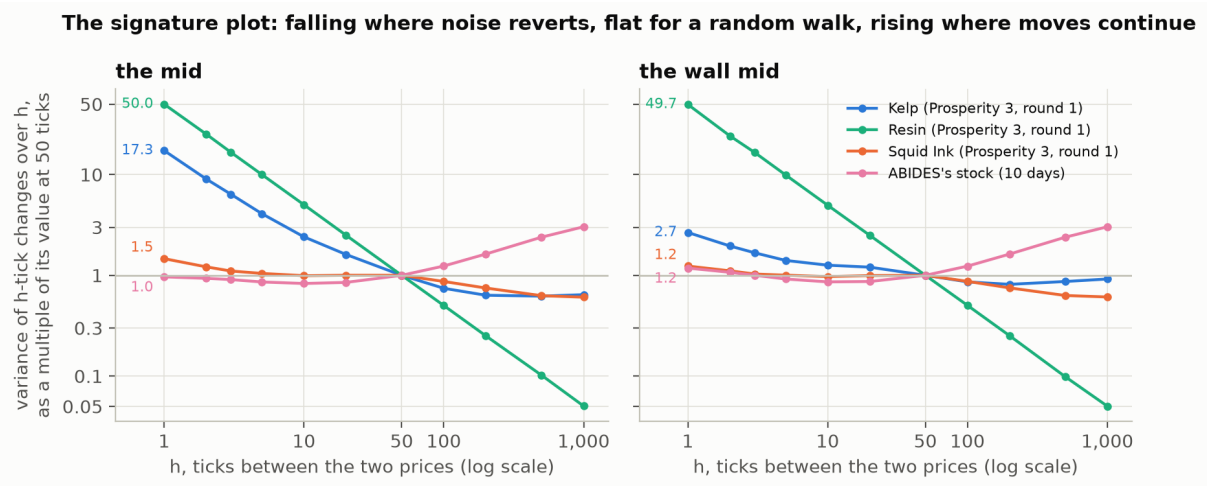


Figure 6: The signature plot for Prosperity 3's Kelp, Resin and Squid Ink (round 1, three days) and ABIDES's one stock (ten days), for the mid and the wall mid, each scaled to its value at 50 ticks. Data: `data/signature.csv`, from the engine's loader.

The plot reads like a diagnosis (Figure 6). Kelp's mid has 17.3 times the one-tick variance its 50-tick changes imply: most of what moves it each tick is the small-order flicker. Its wall mid has 2.7: most of the noise is gone, though not all. Resin's mid and wall mid are both about 50, because Resin's value is fixed at 10,000 and every move comes back. Squid Ink is close to flat, a random walk that jumps, and so is ABIDES at short horizons; ABIDES then rises to about three times its 50-tick value at 1,000 ticks, because its moves continue (and two of its ten days drift) (`data/signature.csv`; `IDEAS.md`, "An independent generator: ABIDES's RMSC04"; `arena.diagnose 1 --data data/abides`, its note "drifts on 2 of 10 days"). The value at one tick is the diagnose tool's `sig` column, which computes the same ratio day by day and averages it (17.9 for Kelp's mid).

The plot says which estimate to trust and at what timescale. A product whose wall mid is flat needs no smoothing: its moves are real. A product whose line falls needs a fair that ignores the falling part.

The micro-price

There is a third kind of estimate, from the sizes at the touch rather than their prices. If the bid holds 2 lots and the ask 6, the next trade is more likely to empty the bid, so the price is more likely to tick down than up. Stoikov writes the imbalance as I, the bid's size over the bid's and

the ask's sizes together, the bid's share of the size at the touch, and the *weighted mid*, which leans the mid toward the side with less size:

$$W = I\,P^a + (1 - I)\,P^b = \text{mid} + S\left(I - \tfrac{1}{2}\right)$$

with S the spread (Stoikov, *The micro-price*, eqs. 2–3, PDF p. 3). On the Kelp tick above, I is 2 over 8, a quarter, and the weighted mid is 2,029.25, a quarter tick below the mid. This is what most teams call the micro-price, and the template has it as `fair: weighted_mid`. Stoikov's own micro-price is different: the mid plus an adjustment learned from data, the expected mid far in the future given today's imbalance and spread (eq. 4, PDF pp. 3–4). It equals the weighted mid only in a special case (appendix 2, PDF p. 9), and on his two stocks it sat between the mid and the weighted mid (PDF p. 6).

On Prosperity's books it doesn't help, and the reason is the walls. The imbalance predicts the plain mid's next move well (a ROC area of 0.83 on Kelp), but what it predicts is the mid's own flicker, which the wall mid has already removed. As a forecast of the wall mid 1 to 50 ticks ahead, adding the imbalance improves on the wall mid by under 1% (`IDEAS.md`, "Book imbalance and the micro-price"). As Kelp's fair value, the weighted mid loses: the template's total on Prosperity 3's round 1, Resin's 53,976 credits included, is 67,630 against 72,708 with the wall mid under the default fill rule, and 56,824 against 63,296 under the strict one; with the plain mid it is 65,054 and 56,729 (`data/kelp_fairs.csv`; `IDEAS.md`, "The weighted mid as fair value"). The literature's regime is a book whose touch *is* the market maker's quote, often with a one-tick spread. Prosperity's touch is small orders inside the walls.

Smoothing, and what a filter would choose

The second choice is how much history the estimate uses. The template's `smoothing` is an exponential moving average: each tick's fair is `smoothing` times the last one plus the rest times the new price, so 0 uses each tick as it is and values near 1 move slowly.

There is a principled starting value. Model the price as a random walk seen through noise, and the best estimate of it is a Kalman filter, which in steady state is exactly such an average, with a gain set by the ratio of the noise to the walk. Both can be read off the same squared changes as the signature plot. The diagnose tool fits them and prints the `smoothing` that matches, its `smooth` column: 0.43 for Kelp's wall mid, 0.97 for Resin's, 0.03 for Squid Ink's (`arena.diagnose 1 --data p3`; `IDEAS.md`, "The Kalman-implied smoothing").

The backtest agrees, with a caveat chapter 10 explains. Smoothing Kelp's fair at 0.43 lifts the template's round total, Resin included, from 72,674 credits to 76,802 under the default fill rule, and 0.6 is the best at 76,806, almost the same. Under the strict rule no smoothing is best, 63,302 against 60,751 at 0.43 (`data/kelp_fairs.csv`). Under the default rule the formula lands in the flat part of the PnL curve on every Prosperity product, within 0.1% of the best; on a simulated noisy Kelp and on ABIDES it lands below the best, so it is a place to start a grid, not an answer (`IDEAS.md`, "The Kalman-implied smoothing").

Robust beats clever

The story's lesson is about robustness. "The largest level" is an argmax, and one outlier captures it: anyone who can place a large order far from the touch moves your fair value. "The first level from the top with at least 15 lots", the template's `sized_mid`, ignores anything behind the real wall. On products tested without rivals it stayed within 2.3% of the old rule, and with rivals it

saved everything (`IDEAS.md`, "Wall-mid definitions"). When other teams' bots share the book, or anyone has a reason to mislead you, prefer the estimate a single order can't move. Chapter 9 meets the deliberate version, a spoofer.

What the fair is for

Ryan Challman's lag shows the last point. A fair value isn't graded on how close it is to some true price. It is graded on the fills it produces: which quotes rest where, and which trades take them. A fair that lags a rising price leaves its asks low, so they fill on every rise; that is a bet on the drift, and it pays until the drift turns. A fair that follows every tick has no such bet and, on their vouchers, no edge.

So the right timescale is the one over which deviations from the fair *come back*, at the horizon you trade them. Chapter 7 makes this a parameter, the smile window, which turned a loss of 267 thousand credits into a gain of 442 thousand on one option (`IDEAS.md`, "Prosperity 2 round 4's coconut coupons with the template's options modes"). Before trading against any deviation, check that it reverts there: the signature plot, its autocorrelation, or whether fading it made money on each day separately.

> **What would differ on another market**
>
> On ABIDES, where value agents hold private views of a hidden price, every estimate we tried, the mid, the wall mid, the sized mid and the weighted mid, was about 35 ticks from that price on average, and smoothing made each one worse; its signature plot is flat at short horizons, so there is no noise to remove there (`IDEAS.md`, "An independent generator: ABIDES's RMSC04"). Inside `arena.sim`, with a maker quoting a tick of noise around the value, the Kalman smoothing was 0.885 and the best was 0.97: the more noise a book carries, the more smoothing pays, and the further the formula falls short (`IDEAS.md`, "The Kalman-implied smoothing"). Prosperity's walls are the easy case.

> **If you know ML**
>
> A fair value is an estimator of a latent variable from noisy observations, and the usual lessons hold. Prefer a robust statistic when outliers can appear: a trimmed rule over an argmax. Match the smoothing to the signal's timescale, as you would a filter's bandwidth; the signature plot is the variogram you would draw for it. Where it breaks: the outliers here can be adversarial, placed by someone who profits from your error; the estimator's loss isn't squared error but the PnL of the fills it causes, so a biased estimator can win; and the latent "true value" may not exist on a market where informed traders, not a generator, set the price.

> **In the repo**
>
> - `fair_value` and `price` in `python/bots/template.py`, chosen per product by `PARAMS[...]["fair"]` (`mid`, `wall_mid`, `outer_mid`, `sized_mid`, `weighted_mid`, `hysteresis_mid`, `fixed`), with `wall_size`, `smoothing` and `imbalance` (the linear micro-price, 0 by default).
> - `uv run python -m arena.diagnose DAYS --data D`: the `mid sig` and `wall sig` columns (the signature at one tick), `smooth` (the Kalman smoothing), and `--suggest` for a PARAMS entry.

- `uv run python -m experiments.day.imbalance --data D --days DAYS` ranks candidate fairs by how they track the wall mid ahead.
- The rows: “Wall-mid definitions”, “The template against the teams’ P4 traders in one market”, “Book imbalance and the micro-price”, “The weighted mid as fair value”, “The Kalman-implied smoothing”.

On the Arena

Written after the event, from the round handovers: which fair value each Arena product used and why, what its signature plot and the diagnose tool’s notes showed on the first download, whether its walls were an exchange bot’s quotes or other teams’ orders, and whether `smoothing` was set from the Kalman column or a grid.

Go deeper

- Stoikov, *The micro-price* (9 pp.): the weighted mid’s flaws (PDF p. 3), the micro-price as a limit of expected mids, and how it compares on two stocks (PDF p. 6).
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 2 §2.1.2–2.1.4 (PDF pp. 43–47): the signature plot, microstructure noise and the bid–ask bounce; ch. 7 §7.1–7.3 (PDF pp. 137–145): what the book’s volumes say about the next move.

Why does a market maker earn anything?

You'll need the book, the spread, taking and making, and fills (chapter 2), and the fair value and the wall mid (chapter 3). Fill models are chapter 10's; here it is enough that a backtest has to assume which quotes fill.

The story

Put our template alone into Prosperity 3's round 1 and it makes 72,674 credits over three days. Put a second copy of the same bot into the same market, quoting the same prices, and each copy makes about 41 thousand: 41,352 and 41,848. With two copies beside it, each makes 27 to 30 thousand (Figure 7). Nothing about the market changed except that someone else was offering the same thing at the same price (`data/copies.csv`; the method in `IDEAS.md`, "The template against copies of itself").

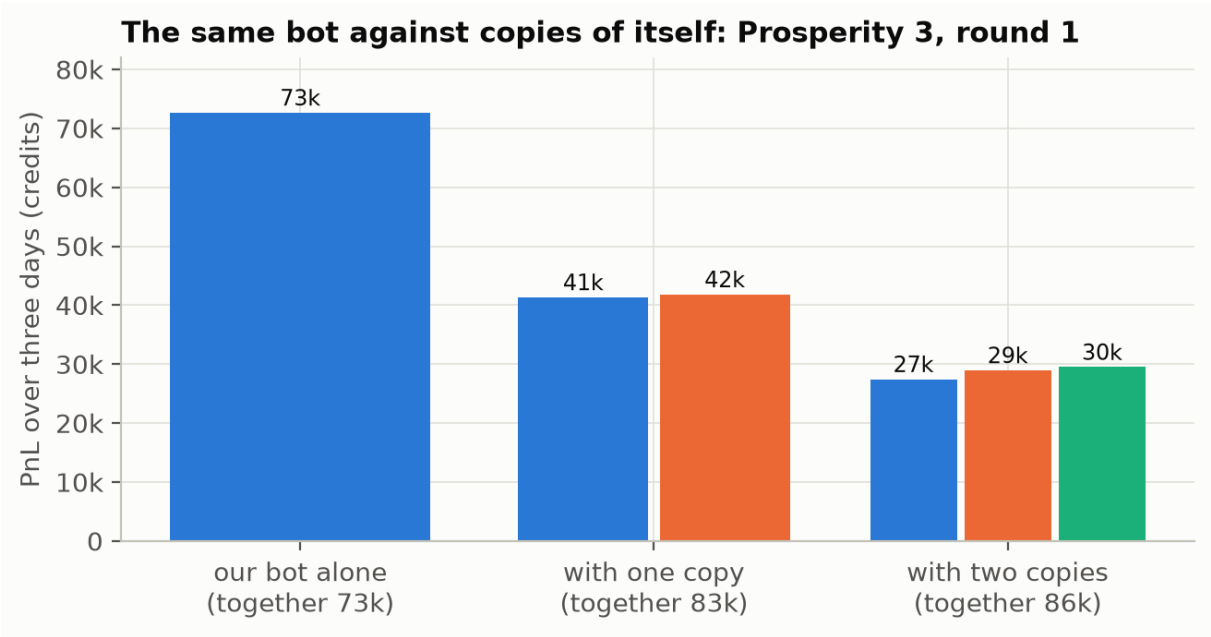


Figure 7: The same bot alone, against one copy of itself and against two, on Prosperity 3's round 1. Data: `data/copies.csv`, from `arena.backtest ... --vs python/bots/template.py`.

Other teams' bots do the same. On Prosperity 4's Osmium, alone and with orders that live one tick (the default mode), the template makes 55,902 credits in round 1. Against one of three published teams' bots it keeps 27,658 to 32,492, about half; against all three at once, 15,632, under a third (`data/osmium_rivals.csv`). Measured on 13 October, the spread on Osmium was a pie of 55 to 62 thousand credits a round, and the bots in the market split it (`IDEAS.md`, "The template against the teams' P4 traders in one market").

In round 2 of the same competition, the organisers offered "extra market access" to every team whose bid was above the median bid: about 20% more quotes in their feed, but not 20% more trades (*their claim*, as are the bids below). Baguette Fromage, a team whose round 1 income came from market making and from quoting on empty sides of the book, read it as a trap. More market makers' quotes in the book could only tighten the spread, and fill the empty sides they were earning on. "Why would a rational team want to PAY for more market maker competition, let alone have more market maker competition at all (even if for free)?" They bid zero. The median bid, the organisers announced later, was 50, so at least half the field bid 50 or more for the right to face more competition (`references/prosperity4-baguette-fromage/README.md:500-519`). We never tested their reasoning; it follows from this chapter's idea.

A market maker's income is a rent, and rents are shared.

The idea

The spread is a fee for immediacy

Someone who wants to trade *now* crosses the book and pays the spread: they buy at the ask, above the mid, or sell at the bid, below it. Someone willing to wait posts an order and lets the hurried come to them. The market maker is the one who waits on both sides at once. Each time a taker buys from its ask and another sells to its bid, it has bought low and sold high by the width of its quotes, without needing any view of where the price is going.

The Varde rehearsal shows this at small scale (Figure 8): the template's first 400 steps on one product of a mock round we ran with its own exchange, which quotes in cents and scores in euros; the figure shows euros. Most of its fills are passive, sells above the mid and buys below it, with some takes among them; its position swings between short 9 and long 6 and keeps coming back to zero; and its profit climbs in small steps, about €4 over 400 steps, while the price itself drifts down.

The position matters as much as the fills. A market maker that bought and never sold isn't earning a spread; it is holding a bet on the price. The income exists only because buys and sells alternate closely enough that the position keeps returning to zero. Chapter 6 is about what happens when it doesn't.

One tick of the template's quotes

The template does this with three steps each tick, in `trade_product`: *take* any book order priced at least `take_edge` better than the fair, *clear* some of the position at prices near the fair, then *make*: quote both sides around the fair. Take the Kelp tick from chapter 2 again, with no position: 2 lots bid at 2,029 and 23 at 2,028, 6 offered at 2,030 and 25 at 2,032. Kelp's settings are `fair: sized_mid` and `make_edge` 0.5, the rest the defaults (`python/bots/template.py`, `PARAMS["KELP"]`).

The fair is the sized mid, 2,030. Nothing is a take: the best ask, 2,030, is not a credit below the fair, and the default `take_edge` is 1. Nothing to clear with no position. The make step puts the bid at the fair less half a tick, rounded down to 2,029, and the ask at the fair plus half a tick, rounded up to 2,031; neither is more than one tick inside the best quote on its side (`make_inside` 1), so they stay. With `make_size` unset each quote is the full room under the limit, 50 lots a side. Run through the template's own functions, the plan for that tick is exactly that: a bid of 50 at 2,029 and an ask of 50 at 2,031, no takes.

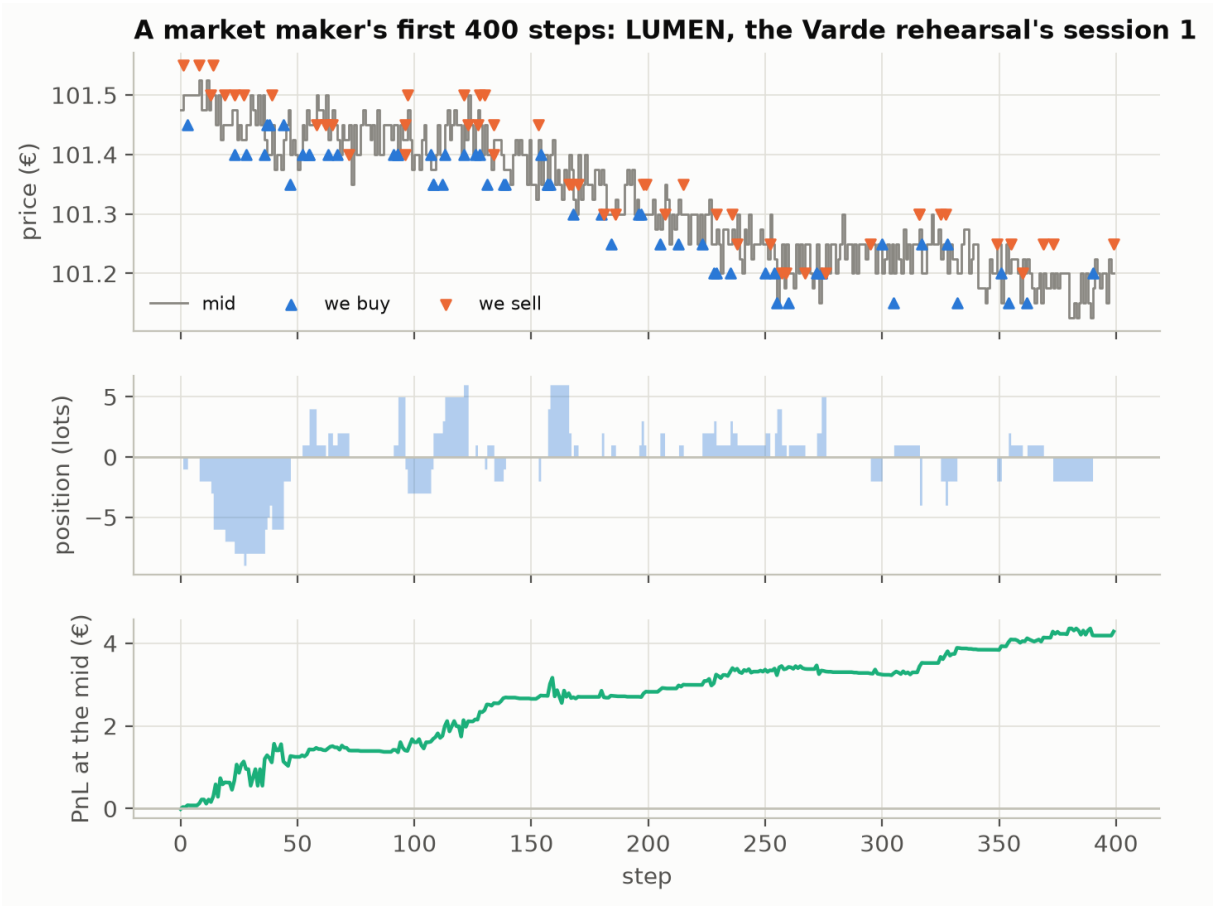


Figure 8: A market maker's first 400 steps: LUMEN, the Varde rehearsal's session 1. Top, the mid and our fills; middle, our position; bottom, PnL valued at the mid. Data: `data/round_trip.csv`, from the rehearsal portal's fills.

So the bot joins the best bid and sits one tick behind the best ask. If a seller hits 2,029, the bot has bought a credit below its fair; if a buyer lifts 2,031, it has sold a credit above. Either way the fill is worth one credit a unit at once, against the fair. Whether it is still worth that later is the next question.

What can go wrong: the price moves after you trade

If that were the whole story, every market maker would be rich. The catch is what happens after a fill. When a taker buys from you, you are now short. If the taker knew something, the price rises, and the short loses more than the spread earned. So the question is not "how wide is the spread" but "how wide is the spread compared with how far the price moves, on average, in the direction of the trades".

Trades, Quotes and Prices (eq. 17.13, PDF p. 345) gives the gain of a patient market maker, per unit traded:

$$\frac{\mathbb{E}[\mathcal{G}]}{\nu_0 T} = \frac{\mathbb{E}[s]}{2} - \mathcal{R}_\infty$$

In it, s is the spread and $\mathcal{R}_\infty$ is the long-run *impact*: after a buy, how far the mid ends up above where it was, on average, and the same for sells with the sign flipped (the book's eq. 1.9, PDF p. 35). Half the spread is what each fill earns at once; the impact is what the market takes back later. On the large stock markets the book studies, the two nearly cancel, and without the exchange's rebates (what an exchange pays back to the orders that rest) the plain strategy loses (§17.2.2): making markets is a competitive business with thin margins.

Measuring the margin before writing a bot

Both sides of the formula can be measured from the recorded trades alone, with no bot at all. The diagnose tool, `arena.diagnose`, does it per product. For every market trade it notes the taker's side, +1 for a trade at or above the best ask and −1 at or below the best bid, then two numbers against the wall mid: what the taker paid, the side times the price less the wall mid (the tool prints its negative as the taker's `edge`), and how far the wall mid moved their way 50 ticks later, standing in for the long run. The market maker who took the other side of every trade earns the first and gives back the second (`python/arena/diagnose.py`, its docstring's "Market trades").

The two cases look like this (Figure 9). On Prosperity 3's Kelp a taker pays 0.93 credits a unit, and 50 ticks later the price has moved 0.01 *against* them: the market maker keeps 0.94 a unit. Resin keeps 1.41, Squid Ink 0.95. On ABIDES a taker pays 0.99 cents a unit and the price then moves 1.31 their way: the market maker loses 0.32 a unit (`data/margins.csv`). On Prosperity 4's round 1 the margins are larger in credits because the spreads are wider, 7.93 on Osmium and 3.78 on Pepper Root, but as a share of the spread they are the same kind of number: 0.49 and 0.37, against Kelp's 0.41 and ABIDES's −0.21 (`data/margins.csv`).

That one measurement says whether plain market making can work on a market at all. Prosperity is the easy case, the textbook's "zero-intelligence" market where takers know nothing: trades barely move the price. A second sign agrees. On Prosperity a trade's side says almost nothing about the next trade's: the correlation of consecutive signs is 0.09 to 0.13, against 0.37 to 0.93 on NASDAQ stocks (`IDEAS.md`, "Does market making pay here, by the textbook's test?"). On ABIDES it is 0.36: trades come in runs, as they do when someone large is working an order (`data/margins.csv`). Chapter 5 is about the market on the other side of that number.

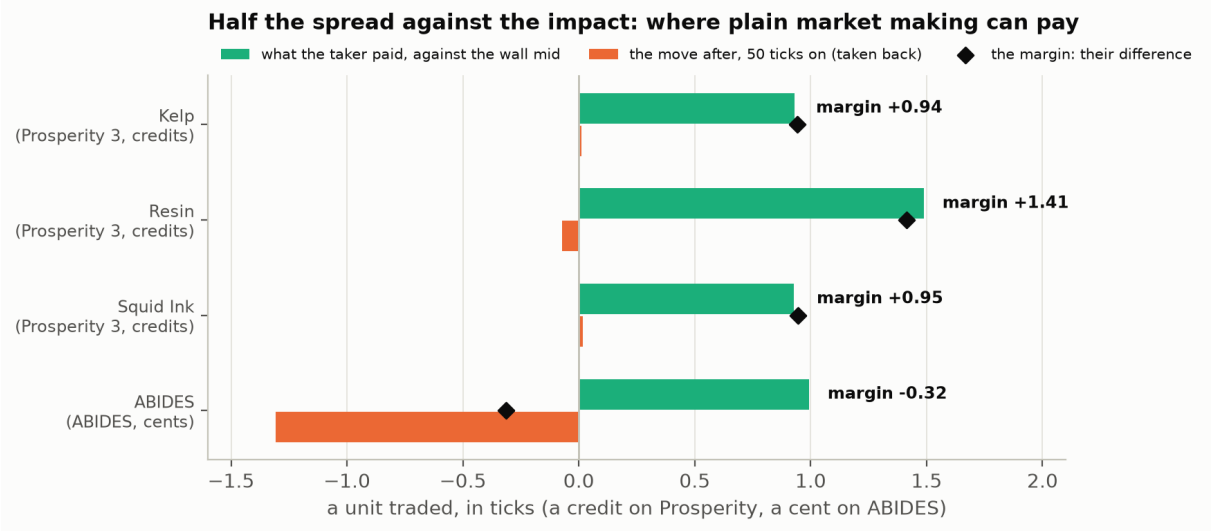


Figure 9: The two parts of the margin per unit, for Prosperity 3's round 1 products (credits) and ABIDES's stock (cents), against the wall mid. Data: `data/margins.csv`, from `arena.diagnose 1 --data p3` and `--data data/abides`.

The margin is an upper bound for one bot, not a forecast. It is what the *average* taker leaves on the table against the wall mid, before anyone competes for it and before any fill rule. The template's own passive fills on Kelp earn 0.69 credits a unit and keep it for 500 ticks (chapter 5's markout figure), less than 0.94; we haven't measured why.

So where does the income come from?

Three things decide it, and you control only some of them:

- **How much uninformed flow there is,** traders who cross the spread for reasons that don't predict the price. They are the market maker's customers. You can't create them.
- **How many others compete for them.** The story: a second bot at your price takes a little under half. Quoting a little better than the others wins the trades, but each step inside the spread gives away margin.
- **How well you avoid the informed.** Quoting further from the fair, pulling quotes when the book looks dangerous, or skewing them against your position (moving both quotes so the side that adds to it is the less attractive) all trade a little income for safety. Chapter 5 is about the informed, chapter 6 about the position.

Competition is the spread's other side

Baguette Fromage's reasoning is the textbook's, in reverse. If market makers earn more than their risk, others join, undercut each other's quotes and narrow the spread until the margin is thin. The large stock markets are at the end of that process; Prosperity's bots aren't, which is why its margins are a quarter to a half of the spread. In a contest every team is a potential market maker in the same book, if the book is shared. Each one that quotes at your price splits the flow, and each one that quotes inside you takes it.

The mechanism is the matching rule of chapter 2. An incoming order trades with the best price first, so a bid one credit higher than yours is hit before yours is. Quoting a tick inside the others, *pennying*, wins the next trades and gives up a tick of margin on each; the others can do the

same, and the margin goes down a tick at a time until one of them would rather wait. That is why the template's make step has two limits, a distance from the fair it won't come inside (`make_edge`) and a number of ticks inside the best quote it won't go beyond (`make_inside`): it competes for the trades, but not past the point where they stop paying. On the Kelp tick above, the bot joined the best bid rather than bidding 2,030, which would have bought from the 2,030 ask at its own fair and earned nothing.

Osmium shows the rest of the arithmetic. Against one team's bot the template kept 49 to 58% of what it made alone, and against three, 28% (`data/osmium_rivals.csv`). Three rivals didn't leave it a quarter each: they quote at different distances, take differently and hold different positions, so the pie isn't cut evenly, and a replay can't say how the takers themselves would have reacted to four market makers instead of one.

One caution about Figure 7: between them the copies make a little more than one bot alone (83 and 86 thousand against 73). Don't read anything into that: a replay can't let the market react to two bots instead of one, so the total isn't something a real market would pay. Read each copy's share, which is what would be scored. And a share depends on how the exchange breaks ties between two orders at the same price, which a replay also has to assume.

What would differ on another market

On ABIDES the margin is negative, −0.32 cents a unit, because value and momentum agents move the price after their trades; the template as committed loses there under every fill model (chapter 5). On the Varde rehearsal's exchange a fee on every order placed, not just every lot traded, took 28% of the gross: a cost the margin in this chapter doesn't see (chapter 12). Prosperity 4's Osmium has a margin eight times Kelp's in credits and a similar share of its spread. The formula is the same everywhere; the measurement is per market and per product.

If you know ML

Measuring $\mathbb{E}[s]/2 - \mathcal{R}_\infty$ from the data is estimating a baseline's expected reward before you train anything: if it is negative, no amount of tuning a plain market maker will fix it. The copies are a multi-agent effect: the reward is shared, so a policy that looks good alone is worth less in a population of similar policies. Where it breaks: the margin is measured on other traders' trades, and your quotes change who trades with you. A bot that quotes tighter than everyone gets more of the informed flow too.

In the repo

- The margin, per product: `uv run python -m arena.diagnose 1 --data p3` (the `edge`, `R(1)`, `R(50)`, `slow` and `fast` columns; its notes say "market making pays" or "thin or negative market-making margin", as a share of the spread; `--csv` writes the same against the mid as well).
- Copies and rivals in one market: `arena.backtest BOT DAYS --vs OTHER_BOT` (repeatable; `--requote` for bots written to cancel every tick).
- The rows: "Does market making pay here, by the textbook's test?", "The template against copies of itself", "The template against the teams' P4 traders in one market".

On the Arena

Written after the event, from the round handovers: each product's margin from `arena.diagnose` on the first download, whether the market was shared with the other teams, and what the copies and rivals test said about how much of the spread our bot could keep.

Go deeper

- Harris, *Trading and Exchanges*, ch. 14 "Bid/Ask Spreads", to the end of §14.4 (PDF p. 323): why spreads are as wide as they are.
- Bouchaud et al., *Trades, Quotes and Prices*, ch. 16 §16.1–16.2 and ch. 17 (PDF pp. 317–330, 338–350): the theory of the spread, and the profitability of market making.

Who is on the other side?

You'll need fills and passive orders (chapter 2), the fair value and the wall mid (chapter 3), and half the spread against the impact (chapter 4). Fill models and resting orders are named here and explained in chapter 10; the grid and the permutation test in chapter 11.

The story

Everything in the template was tuned on Prosperity data. So we took it, unchanged, into ABIDES, an independent market simulator. Its one stock is traded by 1,000 noise agents, 102 "value" agents with noisy private views of a hidden price that slowly returns to a mean, 12 momentum agents that follow moves, and two market makers that adapt (`IDEAS.md`, "An independent generator: ABIDES's RMSC04"). ABIDES prices are in cents.

The template lost. Over ten days it made −21,798 cents (about −$218) under the most optimistic fill model, −57,872 under the strict one, and −362,428 (about −$3,624) with orders that rest. A grid of 36 of its settings, four fair values by three quoting distances by three smoothings, found one cell that made money under the strict rule, +29,196, and none with resting orders: the best of them lost 155,901. Under the optimistic rule seven cells made money, the best +310,847, and that cell lost about a million resting. Doing nothing beat every cell with resting orders (`arena.backtest` and `arena.grid` on `data/abides`, re-run; `IDEAS.md`, "An independent generator: ABIDES's RMSC04").

Then the test that explains it. We made twenty copies of the same ten days with each day's ticks put in a random order, the same books and the same trades with no sequence, and ran a small search over the bot's quoting distance on each. On the shuffled days the best setting made **+4,540,545 cents** on average, about +$45,000 (Figure 10). On Prosperity's Kelp the same test changes almost nothing: 27,390 credits on the real days, 27,326 on the shuffled ones (`data/shuffle.csv`; `IDEAS.md`, "An independent generator: ABIDES's RMSC04").

Shuffling keeps every book and every trade and destroys the sequence: *what happens after* each fill, including moves that keep going. On ABIDES that was where the loss lived. The order of events was the adverse selection.

The idea

The fills you get are the ones you didn't want

A resting quote doesn't choose who trades with it. It is filled whenever someone finds it worth taking, and the people most eager to take it are the ones who know the price is about to move through it. A bid gets hit just before the price falls; an ask gets lifted just before it rises. This is *adverse selection*: conditional on being filled, a market maker's fills are worse than its quotes looked.

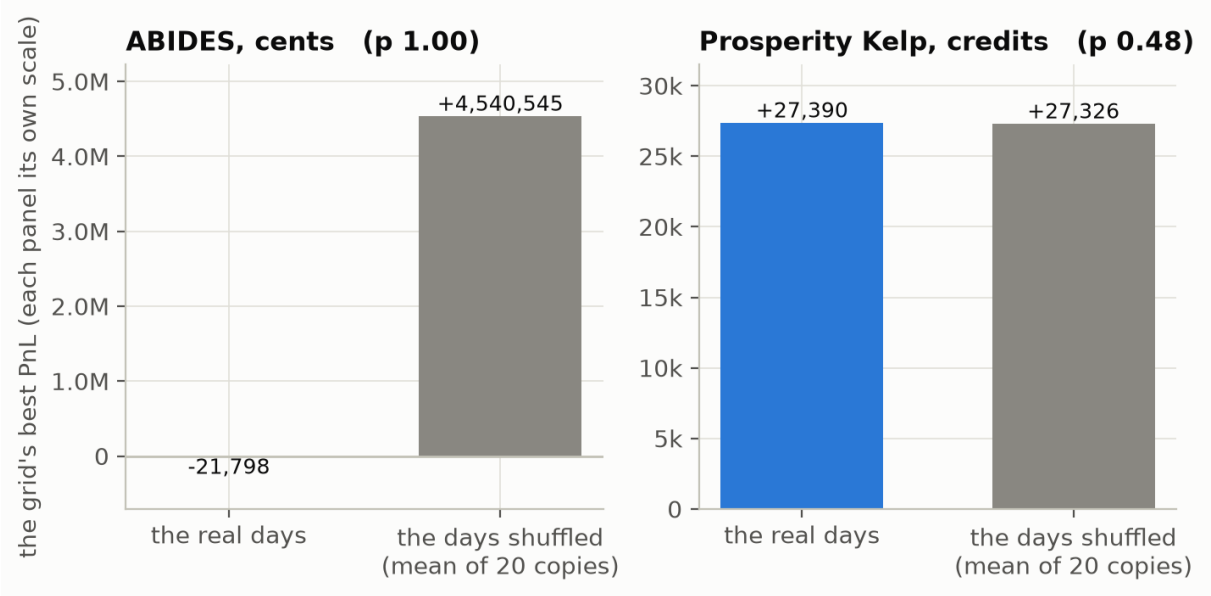


Figure 10: The grid's best PnL on the real days and on shuffled copies: ABIDES (cents, ten days) and Prosperity 3's Kelp (credits, three days); each panel on its own scale. Data: `data/shuffle.csv`, from `arena.grid ... --permute 20`.

The measurement is the *markout*, a fill's profit some ticks later, valued at a reference price, the mid or, on a book like Prosperity's, the wall mid:

$$\text{markout}(h) = \text{side} \times (\text{ref}_{t+h} - \text{fill price})$$

with side $+1$ for a buy and -1 for a sell (`python/arena/analysis.py`, `markouts`). At the moment of the fill the markout of a passive fill is the edge it was quoted at. On an uninformed market it stays there as h grows. Under adverse selection it falls, and if it falls below zero the market maker pays to trade.

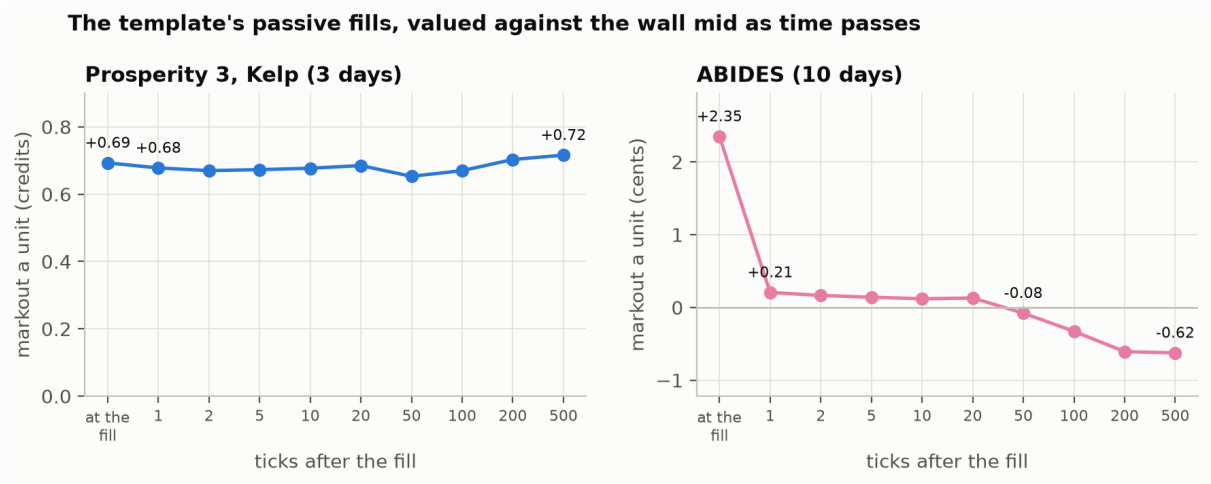


Figure 11: The template's passive fills, volume-weighted over the days, valued against the wall mid at each number of ticks after the fill: Prosperity 3's Kelp in credits, ABIDES in cents. Data: `data/markouts.csv`, from `arena.backtest ... --out`.

The whole difference between the two markets is in one chart (Figure 11). On Kelp the template's passive fills are worth 0.69 credits a unit when they happen, 0.68 a tick later and 0.72 after 500

ticks: nothing takes the edge back. On ABIDES they are worth 2.35 cents a unit when they happen, because the template quotes about two cents from its fair there (its default `make_edge`, 2), and 0.21 a tick later: the price has moved two cents against them in one tick. After 50 ticks they are below zero, −0.08, and after 500, −0.62 (`data/markouts.csv`). Split by what the market did just before, ABIDES's fills against the last move lost on every day under every fill rule, and fills with it gained (`IDEAS.md`, "An independent generator: ABIDES's RMSC04").

Reading a markout curve

The curve is cheap to make and easy to misread, so three things about it. First, its shape says more than any one point. A curve that stays where it started is uninformed flow. One that falls and keeps falling is informed flow, and the horizon where it crosses zero is roughly how long the informed traders' knowledge lasts. One that falls at once and then holds is a cost paid at the moment of trading, a jump or a stale quote, rather than someone's view.

Second, the reference price matters at short horizons. Measured against Kelp's plain mid, the same fills are worth 0.44 credits a unit at the fill and 0.67 a tick later. Nothing happened to the fills in that tick: the mid's flicker, chapter 3's noise, is in the first number and gone from the second. Against the wall mid the curve is flat from the start (`data/markouts.csv`). On ABIDES the two differ at short horizons the other way: against the mid the fills are already −0.32 cents a unit one tick after, −0.18 at 2 ticks and +0.12 at 20, against the wall mid's +0.21, +0.17 and +0.13; from 50 ticks on the two agree (`data/markouts.csv`). So read markouts against the fair value the bot trades around, or past the horizon where the mid's noise has gone.

Third, a backtest's markouts depend on its fill rule, because the rule decides which fills exist. The doubtful fills, trades printed exactly at our price between other bots, are the ones a strict rule leaves out, and DeLise's argument is that limit-order fills come with adverse price moves, so a fill rule that treats fills as random arrivals overstates a market maker's profit; of the three fill techniques he replays against a live simulator, filling always on an adverse move plus a calibrated share of the rest comes closest to it. The analysis tool, `arena.analysis`, prints the curve for each product with takes and passive fills apart; the passive ones are the market maker's, and the ones to watch.

Why there is a spread at all

Adverse selection isn't only a risk to a market maker; in the classic model it is the reason the spread exists. Glosten and Milgrom imagine a market maker who faces two kinds of taker and can't tell them apart: the uninformed, who trade for their own reasons, and the informed, who know where the price will end up. To break even it sets its ask at what it expects the value to be *given that someone buys*, and its bid at the value given that someone sells. Each trade is news, so the two quotes sit apart (Bouchaud et al., ch. 16 §16.1.1, eq. 16.1, PDF p. 319).

In the model's orderly case, where the uninformed traders' views are spread at least as widely as the value can move and only a small share of traders are perfectly informed, it gives the spread in one line:

$$s \approx 2\,\phi\,\sigma$$

with ϕ the share of trades that are informed and σ the scale of the value's moves (Bouchaud et al., eq. 16.6, PDF p. 321). The spread grows with the share of informed traders. And if the uninformed are too few, or trade too little when the spread is wide, there is no spread at

which the market maker breaks even: it withdraws, and the market breaks down (§16.1.4, PDF pp. 322–323). The uninformed are what pays for everything.

Read against chapter 4's measurement, Prosperity's books are what this model calls a market with near zero: the spread is wide and the price doesn't follow the trades, so the market maker keeps most of it. ABIDES has informed traders and a one-tick spread that can't widen to pay for them, and the margin is negative.

Two kinds of market

Prosperity's products are the friendly kind. On Kelp the takers look close to random: the template's position had a correlation of -0.10 to $+0.18$ with where the price went next, about nothing. Kelp's position sits long much of the day for a different reason: only 34 to 36% of its trades at the touch are buys, so more of its takers sell, and the bot buys what they sell (`IDEAS.md`, "`skew` against Kelp's long position"). That position is the flow, not a signal; chapter 6 tests it with a mirrored day.

ABIDES is the other kind: some traders know something, their trades move the price after them (chapter 4's impact, 1.31 cents within 50 ticks), and moves continue: after the wall mid moves 5 ticks over 20, it moves on by about 3 more over the next 50, on all ten days (`IDEAS.md`, "An independent generator: ABIDES's RMSC04"). The template's whole playbook, quote close to a fair value and wait, was built for the first kind and loses on the second. A new market may be either, or each product may be different, which is why the repo runs a test for it before anything else is tuned: `experiments.day.premise` combines the shuffle test, the impact and a forecast into a verdict per product, Prosperity-like, mixed or agent-like (`IDEAS.md`, "The premise check's verdict, calibrated on Prosperity").

Being picked off

The informed need not be people. A price that jumps does the same to a resting quote. On the jumpy basket legs of Prosperity 2 and 3, plain market making with orders that live one tick earned a few thousand credits each: Roses, on Prosperity 2's round 3, 2,685 over three days. With orders that rest, each quote stays in the book until it fills or is cancelled, and when the next snapshot jumps straight through it, it fills at its own stale price. Roses' wall mid moves more than half the spread on 63% of ticks; resting, the same quotes lost 1,477,350 credits, having bought and sold well over 100 thousand units that way on one day (`arena.backtest python/bots/template.py 3 --data p2 --set "PARAMS.ROSES={'limit': 60, 'fair': 'wall_mid', 'make_edge': 1}"` and `--resting`, re-run; `IDEAS.md`, "Market making on products that jump, with resting orders").

Nobody chose to trade with those quotes because they knew something. The replay simply moved the price past them, and a resting order is an option to trade at a fixed price, given free to whoever arrives after the price has moved. The markout of those fills is what the jump was. The diagnose tool prints the share as its `jumps` column, and from 5% its notes say "if orders rest, take only".

When someone is informed, follow them

Adverse selection has a mirror image. If one trader is reliably informed, their trades are a signal. On Prosperity 3 one bot, named Olivia once round 5 showed names, traded Squid Ink six times over the round's three days, 15 lots each time (one of them printed as two trades, with two

buyers). Each buy was within 1.5 credits of that day's lowest wall mid, and each sell within 2 of its highest (Figure 12). A thousand ticks after each, or at the day's close if that came first, the price had moved her way by 4.5 to 86.5 credits; the smallest is a buy 424 ticks before the day's last tick, and a sell 719 ticks before it is cut short the same way (`data/olivia_trades.csv`). Across Kelp, Squid Ink and Croissants none of her trades in round 5 was followed by a move against her at 100, 1,000 or 5,000 ticks; one Kelp buy late in a day was back to no move by the close (`IDEAS.md`, "Following informed counterparties"; `experiments.day.counterparties`).

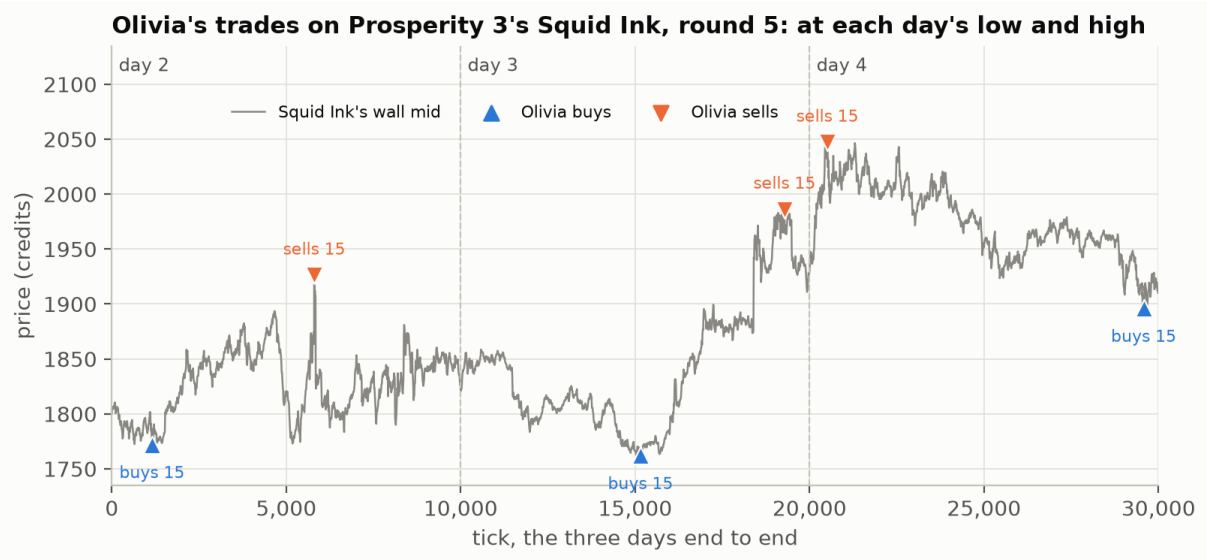


Figure 12: Squid Ink's wall mid over Prosperity 3's round 5, the three days end to end, with Olivia's buys and sells at their trade prices. The line is drawn every 10th tick and every tick within 50 of a trade. Data: `data/olivia_trades.csv` and `data/olivia_prices.csv`, from the engine's loader.

Frankfurt Hedgehogs had spotted the pattern before names were shown, from trades of exactly 15 lots at a running daily low or high, and their final Squid Ink strategy did nothing but follow it (`references/prosperity3-frankfurt-hedgehogs/README.md:270-319`). Our template's `copy` mode does the same with a name: copying Olivia on top of plain market making took our Croissants from 1.7, 1.4 and 2.1 thousand credits a day to 13.5, 25.9 and 20.9 thousand, and Squid Ink from 7.2, 9.5 and 6.5 to 8.9, 14.7 and 8.8. On Kelp it lost: her moves there are small next to the market making they crowd out (`IDEAS.md`, "Following informed counterparties"; `experiments.day.counterparties`, re-run). Those settings were chosen with all three days in view; chosen leaving one day out by how often a name is right, only Olivia was picked, and following her added 54.1 thousand credits over the held-out days, a loss on Kelp included. That rule was written after a first one, choosing names by their mean move, had lost 399.5 thousand on the same held-out days, so it is not a clean test.

The warning is chapter 11's story. Prosperity 4's round 4 named its bots only as "Mark" and a number, and several teams each found a different "informed" Mark. Ryan Challman's team switched theirs off when they failed on a held-out day, and in our test no Mark with more than a handful of trades a day kept the sign of its move on all three days and beat half the spread (`references/wiki/contradictions.md`, row 3). A real Olivia is right on every trade; a lucky one is right for two days.

What would differ on another market

On Prosperity the other side is mostly noise: markouts stay flat, the shuffle test changes nothing, and the danger is mechanical, jumps through resting quotes. On ABIDES it is partly informed: markouts fall within a tick, fills against the last move lose every day, and a bot that does nothing is the benchmark. On days generated by `arena.sim` with a maker quoting a tick of noise and then replayed, per-tick fills in its usual, oscillating state gained on every day, and resting fills lost in every state (`IDEAS.md`, "Passive fills' value by state"). Prosperity 3's round 5 is the one market we have where a name tells you who is informed.

If you know ML

Adverse selection is selection bias with an adversary: the sample of fills you observe is chosen by the people who profit from your errors, so it is biased in the worst direction, and more so the more visible your quotes are. The markout curve is a calibration plot over time: how much of the edge the label "filled" really carried. The shuffle test is a permutation test: destroy the temporal order and see whether the result needed it. Where it differs from the usual use: here the order costing you money is the finding, not a null to reject; and copying a trader is a feature found by searching many names, so it needs the held-out days any searched feature does.

In the repo

- Markouts of any run: `uv run python -m arena.analysis results/` (from `arena.backtest ... --out results/`, `--reference wall_mid` for the wall mid); by market state, `uv run python -m experiments.day.quote_markouts`.
- Prosperity-like or agent-like, per product: `uv run python -m experiments.day.premise --data D --days DAYS`; the shuffle test alone, `arena.grid ... --permute 20`.
- Jumps: `arena.diagnose`'s `jumps` column. Informed names: `uv run python -m experiments.day.counterparties`; the template's `copy` mode follows named traders.
- The rows: "An independent generator: ABIDES's RMSC04", "Market making on products that jump, with resting orders", "Following informed counterparties", "Passive fills' value by state".

On the Arena

Written after the event, from the round handovers: what `experiments.day.premise` said about each Arena product on the first download, the markouts of our first upload's fills, whether the trades named anyone, and whether any product jumped through resting quotes.

Go deeper

- Bouchaud et al., *Trades, Quotes and Prices*, ch. 16 §16.1 (PDF pp. 318–327): the Glosten–Milgrom model, the spread as the price of adverse selection, and market breakdown; ch. 11 §11.1–11.3 (PDF pp. 227–240): the response function, the impact after a trade.
- DeLise, *The negative drift of a limit order fill* (28 pp.): why the price drifts against a limit order once it fills, and which backtest fill rules reproduce that drift.

What does holding a position cost?

You'll need position limits (chapter 2), the fair value and the wall mid (chapter 3), take, clear and make and the round trip (chapter 4), and adverse selection (chapter 5). The fill rules `all` *and* `worse` *are named here and explained in chapter 10.*

The story

Run the template on Prosperity 3's round 1 and look at its position in Kelp, the product whose price wanders. It is long most of the day. Over the three days its mean position is between +21.5 and +35.2 lots, depending on the day and on the fill rule (`all`, `worse`, or orders that rest), out of a limit of 50, and it sits at the limit itself on 5 to 15% of ticks (`IDEAS.md`, "`skew` against Kelp's long position"; `experiments.study.kelp_drift`).

That looks like a market maker failing at its one job, staying flat. The textbook fix is a *skew*: move both quotes down when long, so the bot buys less and sells more. First we asked a plainer question. Is something in the bot leaning long, or is the market handing it a long position?

The test was to mirror each day. Every price is reflected around the day's middle, so each rise becomes a fall; every bid becomes an ask; every buyer in the trade record becomes a seller. If the bot has a bias, a fair value that sits a little low or quotes that lean one way, it stays long on the mirrored day. If the position comes from the flow, it flips. It flipped exactly: on day −2 a mean of +27.8 lots became −27.8, and the PnL was 6,724 credits on both days, to the unit (Figure 13). The same held on every day, under `all`, under `worse` and with resting orders (`data/mirror_summary.csv`; `IDEAS.md`, "`skew` against Kelp's long position").

So the bot is symmetric, and the long comes from the people trading with it. Only 34 to 36% of Kelp's trades at the touch are buys (41 to 43% of the volume), against about half on Resin, so the bot buys 1.24 to 1.32 lots passively for each lot it sells. Then we tried the fix anyway. Every tick of skew cost money. This chapter is about why.

The idea

A position is a bet nobody chose

A market maker's position isn't a decision. It is what is left when the fills on one side outnumber those on the other. Holding it costs in three ways.

- **It moves with the price.** A position of a given size gains or loses that many credits on every move of one. Kelp's price rose 28 credits over day −2; held at the day's mean position all day, the rise would have been worth about 780 credits, and a fall as much the other way. That is variance, not an expected loss.
- **It may come from informed traders.** If the traders who sold to us knew the price was about to fall, the position built from their trades loses on average. That is chapter 5's adverse selection, seen through the position instead of through each fill's markout.

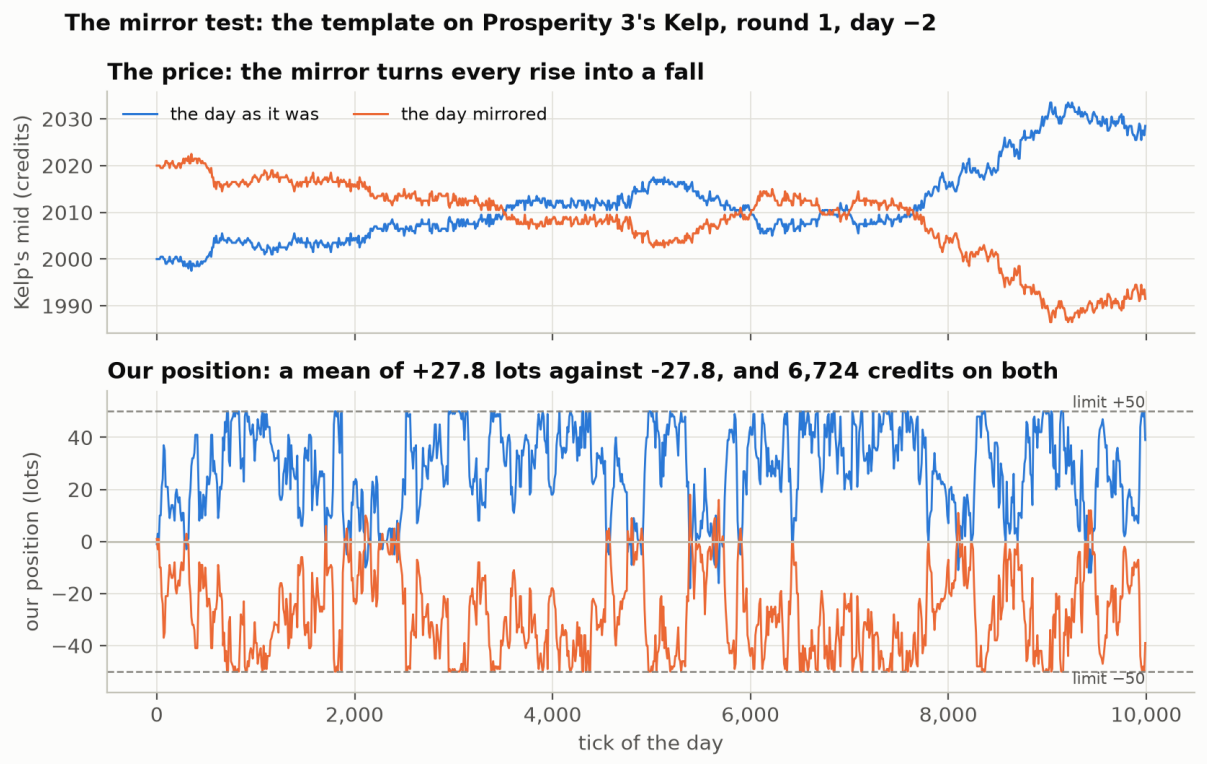


Figure 13: The template's Kelp position on Prosperity 3's round 1, day −2, and on the same day mirrored: top, the price and its reflection; bottom, our position. Data: `data/mirror_test.csv`, from `experiments.study.kelp_drift`'s `mirrored` and the engine.

- **It uses up the limit.** At +50 the bot can't buy. Every seller that would have hit its bid goes elsewhere, and half its business stops until the position comes down.

The second and third are expected costs: one a loss, the other income given up. The data can say whether the second is there, and the third is small here, 5 to 8% of the time at a limit under the default fill rule. The first is a risk, which costs something only if the score charges for risk.

Skew: leaning the quotes

A dealer controls inventory with the tactics Harris tabulates: move both quotes against the position, quote more size on the side that reduces it and less on the side that adds to it, take another trader's quote to change the position at once, or hedge it in a correlated instrument (Harris, Table 13-2, PDF p. 298). The template has the first two as parameters and the third as its `clear` step; the hedge is chapter 8's. The price lever is `skew`:

$$\text{shift} = \text{round}\left(\text{skew} \times \frac{\text{position} - \text{target}}{\text{limit}}\right)$$

So `skew` is the number of ticks both quotes move at the full limit: long 50 with `skew` 2, the bid drops two ticks, so fewer sellers reach it, and the ask drops two, so more buyers do (`python/bots/template.py:851`). The target is zero unless a product has a reason to hold something, like a steady drift.

Avellaneda–Stoikov in words

The famous version is Avellaneda and Stoikov's (2008). Their market maker is risk-averse: it maximises a utility of its wealth at a known time T, when what it holds is valued at the mid. Its quotes centre not on the mid but on a *reservation price*, its own value for the stock given what it already holds (equation 29, PDF p. 6; the same as equation 8, p. 4):

$$r = s - q\,\gamma\,\sigma^2\,(T - t)$$

Here s is the mid, q the position, γ the risk aversion, σ^2 the variance of the mid per unit of time, and $T - t$ the time left. A bigger position is more exposed, a jumpier price moves it further, and the longer it must be held, the more the price can wander. Long, the reservation price sits below the mid, so both quotes drop: a skew. The spread is then set around it, from the same risk term and from how quickly fills fall off with distance (equation 30).

Two later results reframe it. Guéant, Lehalle and Fernandez-Tapia, solving it with a position limit, showed that the time left is an artefact of an approximation near the end: far from it, the skew is a constant times the position (PDF pp. 8 and 11), which is the template's `skew`. Ahuja and co-authors put a mean-reverting price in place of the random walk: far from the end, the best quotes are then constant around the long-run mean, and ignore the inventory (PDF p. 8) (`references/wiki/concepts/optimal-quoting-models.md`).

So the advice depends on its premises. The skew grows with the volatility of the *fair value*, the risk that the thing you hold is worth less tomorrow. Where that is small, as on a pegged product, the theory asks for almost no skew. And the whole model is built on a utility, a penalty on risk. A bot scored on profit alone has a risk aversion of zero, and with none the reservation price is the mid: no skew at all.

Is the position a signal or the flow?

The mirror test says the bot is symmetric. It doesn't say whether the long position is dangerous, because a mirrored day mirrors the danger too. That needs Harris's test for informed counterparties, asked of the position: does it predict the price moving against it? (`references/wiki/concepts/informed-counterparties.md`)

On Kelp it doesn't. The correlation of our position with the change in the price over the next 10 to 1,000 ticks is between −0.10 and +0.18 over the three days and the three fill rules, mostly slightly positive. If the sellers knew something, a long position would come before falls, and the number would be clearly negative (`IDEAS.md`, "`skew` against Kelp's long position").

The same answer comes out of the PnL. Split each tick's change in profit into two parts: what the fills earned at once, and what the position earned by riding the price until the next tick. The second is the position times the next change in the wall mid, summed over the day. On the three days it was +414, −71 and −518 credits, against total Kelp profits of 6,724, 6,247 and 5,727. Over the three days, holding the position added −175 credits to 18,698: about nothing. The rest is the spread (`data/mirror_summary.csv`).

The split has to use the wall mid. Kelp's plain mid flickers around the walls from tick to tick (a lag-1 autocorrelation of −0.48, by `arena.diagnose`; chapter 3's flicker), so against it much of what the fills earned gets booked as the position's gain: 4,119 to 5,441 credits a day (`data/mirror_summary.csv`, `riding_mid`). Chapter 3's noise reaches the accounting too.

Why the skew lost here

With the flow uninformed and the score in profit, a skew can only cost, and the figure below shows how much (Figure 14). Each tick of skew gives up fills the bot would have had, and sells what it holds a little cheaper. In exchange the position shrinks: the mean falls from +24.5 lots at no skew to +9.7 at 3 ticks under `all`. Kelp's profit falls with it, from 18,698 credits to 15,893 at 3 and 11,324 at 8. Under `worse`, where fewer passive fills are counted, it falls faster, from 14,531 to 9,514 at 3 and 6,582 at 8 (`data/skew_grid.csv`; `IDEAS.md`, "`skew` against Kelp's long position").

The theory's own coefficient does no better. Fitted from Kelp's volatility and fill rates, GLFT's closed form asks for 0.6 ticks at the smallest risk aversion tried, 0.001, and costs 514 credits; at 0.01 it asks for 1.9 ticks and costs 952 (`data/glft_skew.csv`; `IDEAS.md`, "The skew coefficient from GLFT's closed form"). And our engine, like Prosperity, values the position left at the end at the mid, so holding to the close costs nothing extra.

The skew a famous model prescribes

The textbook's case was tried by a team that knew it well. In Prosperity 4's round 1, DTU Quant Lab added an Avellaneda–Stoikov skew to their market maker on Osmium, a product pegged at 10,000 with noise around it that reverts quickly (`references/prosperity4-dtu-quant-lab/README.md:85`). In their backtest it cost 16,593 of that edition's credits (*their claim*), and they went back to a fixed fair value of 10,000 (`references/prosperity4-dtu-quant-lab/README.md:91`).

Our grid on the same product agrees on the direction. Re-run today (the cited rows have older numbers from an earlier template: 113.8 thousand at no skew and 93.4 at 8), over Prosperity 4's rounds 1 and 2, six days, the template's Osmium makes 115.9 thousand credits with no skew, 115.5 thousand at 1 tick, 112.6 at 2, 107.1 at 4 and 96.0 at 8; under `worse`, 115.9 thousand falls

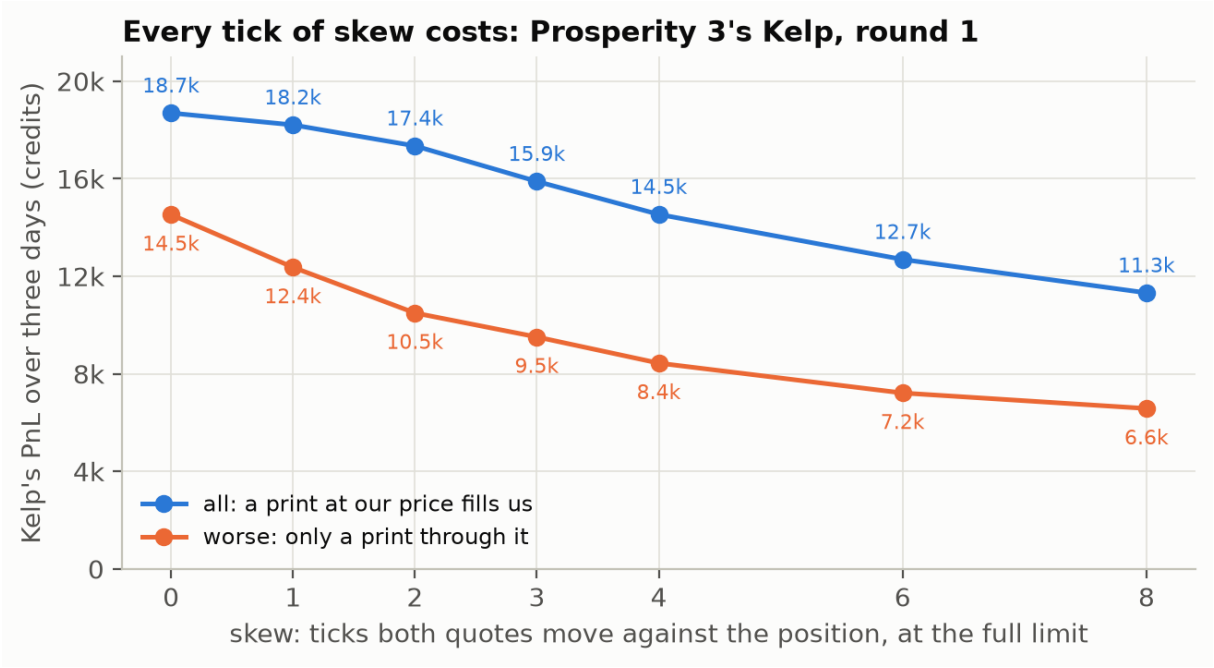


Figure 14: Kelp's profit over the three days of Prosperity 3's round 1 against the template's `skew`, under both fill rules. Data: `data/skew_grid.csv`, from `arena.grid ... --grid PARAMS.KELP.skew=0,1,2,3,4,6,8 --modes all,worse`.

to 83.9. A fixed fair of 10,000, the mean-reverting model's constant quotes, makes 99.2 thousand with no skew and 53.6 thousand at 8 (`data/osmium_skew.csv`; the wiki's `contradictions.md`, row 1). GLFT's closed form, with the variance read off one-tick changes, asks for a skew of 94 ticks there even at a risk aversion of 0.001, and makes 23.7 thousand, where the row's older run had 20 thousand against 114 (`data/glft_skew.csv`). On a pegged price the one-tick variance is noise, not risk: the model's premise, a price that wanders, doesn't hold. We didn't check which variance DTU used; noise in the mid would explain their loss.

The other levers, a line each

- **`size_skew`** shrinks the quote on the side that adds to the position, Harris's "change the sizes" (Bouchaud and co-authors' equation 17.4, PDF p. 342). On Kelp, 0 is best: 0.5 costs 91 credits and 4 costs 2,716. Verdict: **`dropped here`** (`data/size_controls.csv`; `IDEAS.md`, "Inventory control through quote size").
- **`make_size`** caps each quote. A cap of 20 costs 3 credits on Kelp and nothing on Resin; 10 costs 903 and 625; 5 costs 3,657 and 5,836. The flow a tick is smaller than the room, so the cap is no risk lever here: **`dropped here`** as one, kept for Pepper Root's drift bet against rivals (`data/size_controls.csv`; `IDEAS.md`, "Using less than the full position limit").
- **`stop_loss`** flattens a product and stops it for the day once today's profit is that far below its high. It is meant for prices that leave, and chapter 7 shows how mixed that is on Prosperity 3: it does nothing when the fair value can't see the loss, and costs 17 to 20 thousand on some perturbed days when it can. Verdict: **`kept`**, set to about three times a product's largest plain-day drawdown (`IDEAS.md`, "A loss stop per product").

All of these were measured on Prosperity's markets, so each verdict is a prior. The rows' re-test column asks this chapter's question again: a skew only where Harris's test is clearly negative or the score charges for holding. The day's list in On the Day, under "Every round", goes through that column.

What would differ on another market

On ABIDES the takers are informed: their trades move the price by 1.31 cents within 50 ticks (chapters 4 and 5), so a position built from their fills comes before moves against it, the case the skew is for. We never measured a skew there; the template loses on ABIDES before inventory matters. Inside `arena.sim`, a market that reacts to our orders, a preset with Kelp's one-sided flow made the template 23.1 thousand credits against 21.2 thousand without it: flow that knows nothing costs nothing there either (`IDEAS.md`, "Replay against a market that reacts"). Where the close is marked at a price the bot can't see, as in the Varde rehearsal (chapter 10), a position held to the end carries a cost Prosperity never charged.

If you know ML

Skew is a regulariser on the position: it trades expected profit for lower variance, and like weight decay it pays only when the variance it removes would have cost you. The mirror test is an equivariance test: transform the input, check the output transforms the same way. Where the analogy breaks: the data answer back, since skew changes who trades with you, and a score that is a rank, not a profit, can make variance cost after all (chapter 13).

In the repo

- The parameters: `skew`, `size_skew`, `target`, `make_size` and `stop_loss` in the template's `DEFAULTS` (`python/bots/template.py:69-72`, `:146`); the quote shift in `make` (`python/bots/template.py:851`).
- The mirror test, the flow's share of buys, Harris's test and the skew sweep, on Kelp and Resin: `uv run python -m experiments.study.kelp_drift`. The buy share for any download: `arena.diagnose`'s `buys` column.
- The theory's coefficient: `experiments.day.glft_skew`.
- The rows: "`skew` against Kelp's long position", "Skew on a pegged product", "The skew coefficient from GLFT's closed form", "Inventory control through quote size", "Using less than the full position limit", "A loss stop per product".

On the Arena

Written after the event, from the round handovers: whether any Arena product held the template on one side, what Harris's test said about it, whether a skew or a size cap went into a submitted bot, and how the Arena valued a position left at the end of a round.

Go deeper

- Avellaneda and Stoikov, *High-frequency trading in a limit order book* (PDF pp. 3–6): the reservation price (equation 8, p. 4) and its approximation with the spread (equations 29 and 30, p. 6).
- Guéant, Lehalle and Fernandez-Tapia, *Dealing with the inventory risk* (PDF pp. 8–11): why the time left drops out, and the closed form with a position limit.
- Ahuja and co-authors, *Limit order trading with a mean reverting reference price* (PDF p. 8): constant quotes around a level.
- Harris, *Trading and Exchanges*, ch. 13, Table 13-2 (PDF p. 298): how dealers control their inventory.

What do the prices do?

You'll need the fair value, the wall mid and the signature plot (chapter 3), market making (chapter 4), and holding a position, `target` and the loss stop (chapter 6). Options are named here and explained in chapter 8; the fill rules in chapter 10.

The story

Prosperity 2's round 4 added a coconut coupon: a call option on coconuts, with a strike of 10,000 and 250 days to expiry. The template's `smile` mode values an option the textbook way, with the Black–Scholes formula at one fixed volatility (chapter 8), and then adds the option's own average gap from that value, because real prices sit a little off any model. The gap is an exponential average with a span of `smile_window` ticks, started from the first tick's gap (`python/bots/template.py:120`, `:759-760`). The bot buys when the price is half a credit below the result and sells when it is half a credit above.

With the default window of 20 ticks the coupon looked like a gift. Its gap from the 20-tick average had a standard deviation of 2.3 to 2.4 credits, against a half-spread of 0.55: plenty of apparent mispricing, every day. The bot traded it and lost 266,554 credits over the three days. With a window of 5,000 ticks, the same strategy on the same days made 441,929, and its worst day made 114,476 (Figure 15). The same cell won when each day was held out in turn, and under the strict fill rule too, since it only takes (`data/smile_window.csv`; `IDEAS.md`, "Prosperity 2 round 4's coconut coupons with the template's options modes").

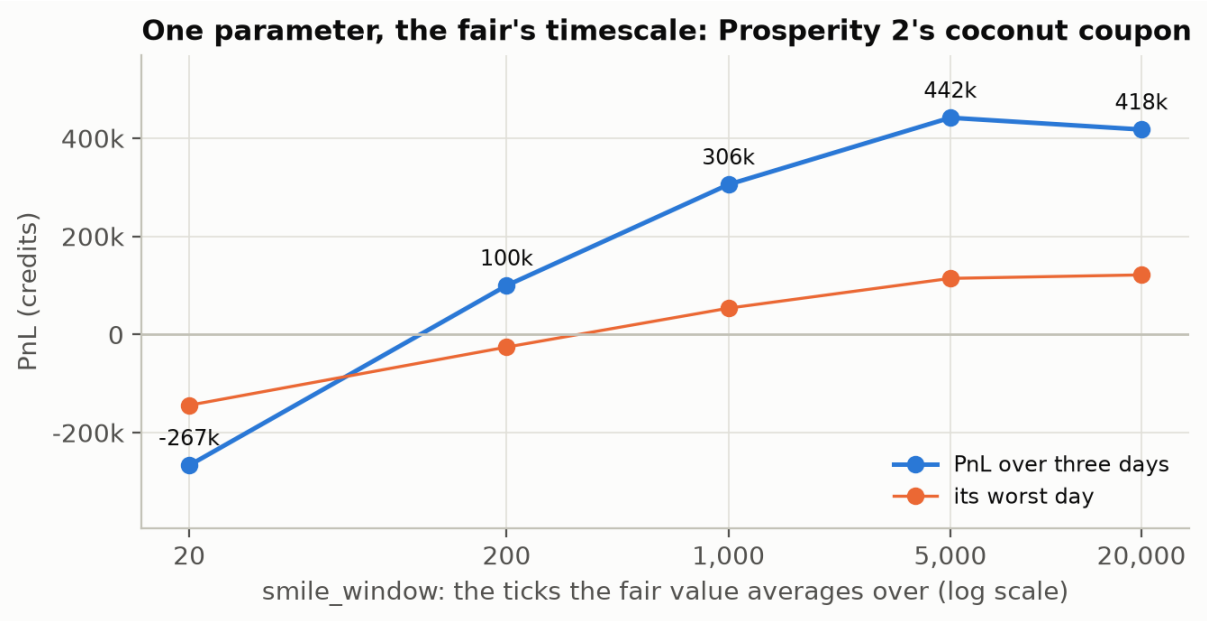


Figure 15: The template's `smile` mode on Prosperity 2's coconut coupon, round 4, three days, at each `smile_window`. Data: `data/smile_window.csv`, from the IDEAS row's `arena.grid` command.

What changed was only how long the fair value remembers: the horizon at which the bot asked whether the price comes back.

The idea

Every pattern has a timescale

Every strategy built on a fair value bets that deviations from it come back, and whether they do depends on the horizon: the same series can bounce back within a tick, wander for an hour, and return to a level by the evening.

The tool is the autocorrelation: how much of a deviation is still there some ticks later. Where it falls fast, deviations are noise to fade; where it stays near 1, they are moves to follow or stay out of. If it falls by the same factor every tick, its lag-1 value says how long half of a gap takes to close (Bailey and co-authors, Proposition 6.2, equation 6.5, PDF p. 21):

$$\tau_{1/2} = -\frac{\ln 2}{\ln \varphi}$$

Here φ is the lag-1 autocorrelation. For an Ornstein–Uhlenbeck process, a price pulled back toward its mean at a steady rate (Ahuja and co-authors' equation 1, PDF p. 3), minus the log of the lag-1 autocorrelation is that rate a tick, so the half-life is the log of 2 over the rate.

The figure below applies it to the coupon (Figure 16). The gap between its price and the fixed-volatility model has a lag-1 autocorrelation of 0.996 to 0.997, a half-life of 163 to 255 ticks. Twenty ticks after any moment, 92 to 95% of the gap is still there. By 1,000 ticks almost none is (`data/coupon_acf.csv`, from `experiments.day.p3_smile`'s residual).

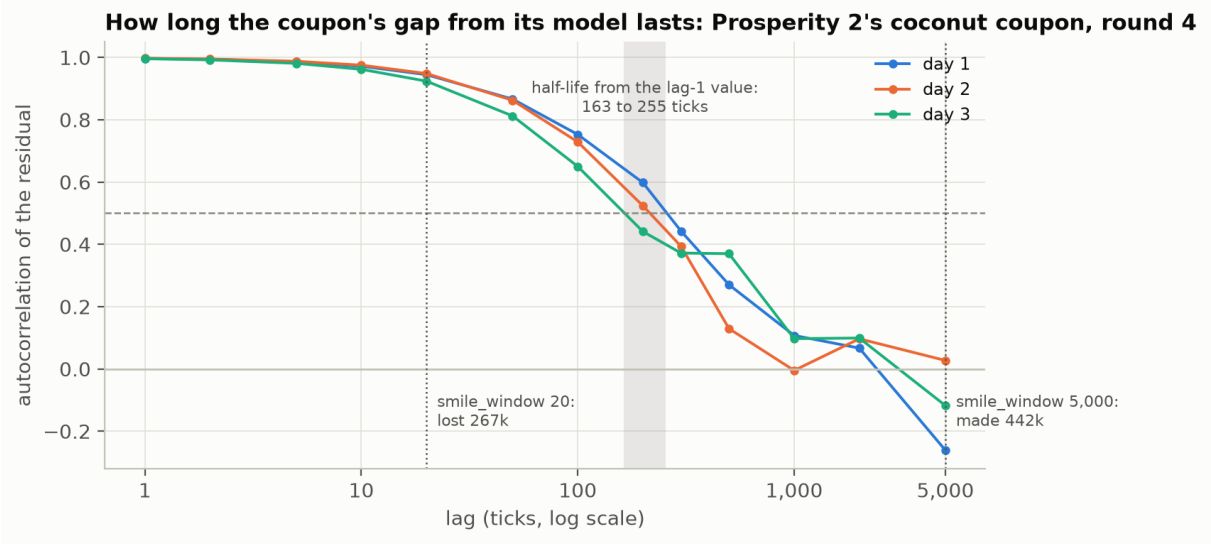


Figure 16: The autocorrelation of the coupon's gap from its model at each lag, on the three days, with the half-life its lag-1 value implies and the two windows of the story. Data: `data/coupon_acf.csv`, from `experiments.day.p3_smile`'s functions with the IDEAS row's settings.

That explains the story. A 20-tick average follows a gap that takes about 200 ticks to halve, so the "fair" moves with the price. When the price runs ahead of it, the bot sells; the price keeps going, and the average catches up with the price rather than the price coming back. What the bot traded

was the *fast part*, the gap less its 20-tick average, and that did revert: its lag-1 autocorrelation of 0.89 means a half-life of about 6 ticks. But it reverted by the average moving, which pays nothing. A 5,000-tick average is about 20 to 30 half-lives long: it stands still while the gap closes, and the bot is paid for waiting. Twenty thousand ticks did a little worse, probably because each day's average gap differs (−0.9, −3.3 and +4.1 credits) (`experiments.day.p3_smile`, part 2).

So before fading a deviation, measure how long it lasts, and make the fair value's memory long against that.

The shapes a price takes, and the notes that find them

The repo's diagnosis tool, `arena.diagnose`, reads each product's data before any bot runs and prints notes, with thresholds calibrated on Prosperity's data (`python/arena/diagnose.py`; `references/wiki/concepts/reading-the-data-on-day-one.md`). Each shape below has a note and a response in the template; the examples are its output on Prosperity 3 and 4.

Shape	`diagnose`'s note	Example	The template's response	Verdict
noise around a value	"mid mean-reverts at one tick"	Kelp, lag-1 −0.48	a fair from the walls, not the mid	kept
a fixed value	"wall mid nearly fixed at 10000"	Resin, sd 0.35	`fair: fixed`, with `fixed_guard`	kept
a level over hundreds of ticks	"wall mid reverts to a level over ~1,000 ticks"	Hydrogel, Velvetfruit	`level` mode	kept
a drift	"drifts up every day"	Pepper Root, +1,000 a day	a `target` toward it	kept
jumps that snap back	"jumps of 6+ spreads snap back"	`ROBOT_DISHES`, 87% of 743	`jump` mode	kept, off by default

Noise around a value. Kelp's mid changes have a lag-1 autocorrelation of −0.48: a move is mostly undone the next tick. The note fires at −0.2 or below and, with "wall mid filters the mid's noise", says to price off the wall mid (chapter 3) (`arena.diagnose 1 --data p3`).

A fixed value. Resin's wall mid has a standard deviation of 0.35 around 10,000, so the template fixes its fair there. A fixed value can't notice it has stopped being true, which is the rest of this chapter.

A level. Prosperity 4's Hydrogel and Velvetfruit wander far within a day, yet each day's mean sits close to the others' (Hydrogel's between 9,989 and 10,003). The note compares the variance per tick of 50-tick changes with that of 1,000-tick ones and fires at 1.8 or above: the price comes back over hundreds of ticks, too slowly for any one-tick number to see (`IDEAS.md`, "Trading toward a known level"). The `level` mode holds the full limit short once the price is `level_band` above the level, and long once it is as far below: a directional bet, not market making. With the level and settings chosen on three days and scored on the fourth, it made 149.9 thousand credits on Hydrogel and 98.9 thousand on Velvetfruit over the four held-out days, against 179.6 and 120.7 thousand chosen on all four. Taking the level from the day so far instead of the other

days cost about a third, 184.9 thousand against 280.2, because a day can open far from its level (`data/level_heldout.csv`). Verdict: `kept`.

A drift. Pepper Root rose about 1,000 credits a day, every day. Bouchaud and co-authors put the general point plainly: market making is easy when prices mean-revert and difficult when they trend (*Trades, Quotes and Prices*, §17.1, PDF p. 341). The template's plain market maker on it made between −3.9 thousand and +57.6 thousand a day; with a `target` of the full limit, 80 long, and a skew of 20 ticks toward it, 75.9 to 78.2 thousand on each of the six days (`data/drift_target.csv`; `IDEAS.md`, "A long bias on a drifting product"). Here chapter 6's skew earns its place, working toward a position the bot wants. Verdict: `kept`.

Jumps. Two notes. "Wall mid jumps past half the spread on 16% of ticks: if orders rest, take only" fires on Prosperity 3's Squid Ink: a price that jumps goes straight through a resting quote (chapter 10). "Snap back" fires on four product-days of Prosperity 4's round 5, all jumping by 100: `ROBOT_IRONING` on day 2 (98% of 58 large jumps reversed the one before), Evening Breath on day 2 (88% of 66), Chocolate on day 4 (86% of 57), and the largest, `ROBOT_DISHES` on day 4, whose wall mid sat on multiples of 100 and stepped off and back: 87% of 743 (`arena.diagnose 5 --data p4`). The `jump` mode holds the limit against each jump, and only once most of the day's jumps have reversed. Over the round's three days it made 958.8 thousand credits against 457.8 thousand for plain market making, 429.1 thousand of the gain on `ROBOT_DISHES`' day; without the gate, fading every jump, day 3 made 184.1 thousand less than plain market making, a loss of 37.3 thousand (`IDEAS.md`, "Fading jumps that snap back"; re-run with `experiments.day.p4_jumps`). Verdict: `kept`, off by default.

Each verdict is a Prosperity result, and the thresholds are rules of thumb from the same data. The runbook's table for each round runs `diagnose` on every new download at T+0:15 (`RUNBOOK.md`, "Each round"; `ON_THE_DAY.md`, "What kind of market is each product?").

When the past stops describing the day

Every response in that table assumes the data's pattern continues. In Prosperity 4's round 3, DTU Quant Lab met the case where it didn't. Their fixed-anchor market maker had worked on Osmium's peg for two rounds, so they used it on a new product, Hydrogel. In the live round, by their account, Hydrogel drifted 70 ticks in one session, and "the fixed-FV anchor kept buying into the drawdown"; the round's algorithmic result was +3,686 credits, rank 2,232 (*their claim*) (`references/prosperity4-dtu-quant-lab/docs/journey.md:16-18`). Elsewhere their write-up gives the drift as 35 ticks or more (*their claim*) (`references/prosperity4-dtu-quant-lab/round_3/README.md:48`).

Their regime detector, built for the next round from the mid's one-tick volatility over 200 ticks, called the handed-out days about 99% "pinned" and the live day 100% "noisy", and their static trader made 17,593 on the handed-out days (*their claim*) (`references/prosperity4-dtu-quant-lab/round_4/algorithmic/trader.py:283-295`). In level, though, those days were as wide as the live one: Hydrogel's wall mid ranged over 142 to 171 ticks on each, standard deviations of 25 to 38 (`data/hydrogel_days.csv`), against a standard deviation of about 30 and a range of 9,915 to 10,031 on the live day as DTU describes it (`references/prosperity4-dtu-quant-lab/round_4/algorithmic/trader.py:261-262`). The anchor survived excursions that large in sample and lost live. A fixed anchor is the purest level bet: it pays while the price keeps coming back, and in-sample days can't say when that stops.

What a regime change costs: the stress test

The stress tool, `arena.stress`, reruns a bot on copies of each day with the prices moved: volatility doubled, a steady trend of ten median spreads over the day, a shift of five spreads at midday that stays, a spike of five that comes back after 100 ticks, or the largest level removed for the middle fifth of the day. Only the path changes (`python/arena/stress.py`).

The figure below runs it on the template, Prosperity 3's round 1 (Figure 17). As committed, the template makes 72,674 credits on the plain days and 66,554 and 62,214 with a trend. Take away one parameter, Resin's `fixed_guard`, which switches the fixed fair to the market's price while the two are more than 5 apart, and the trends take it to 10,444 and 6,228: Resin alone goes from +53,976 to −10,413 and −10,309, a fixed fair trading all day against a price that has left, buying into the fall and selling into the rise (`data/stress_variants.csv`).

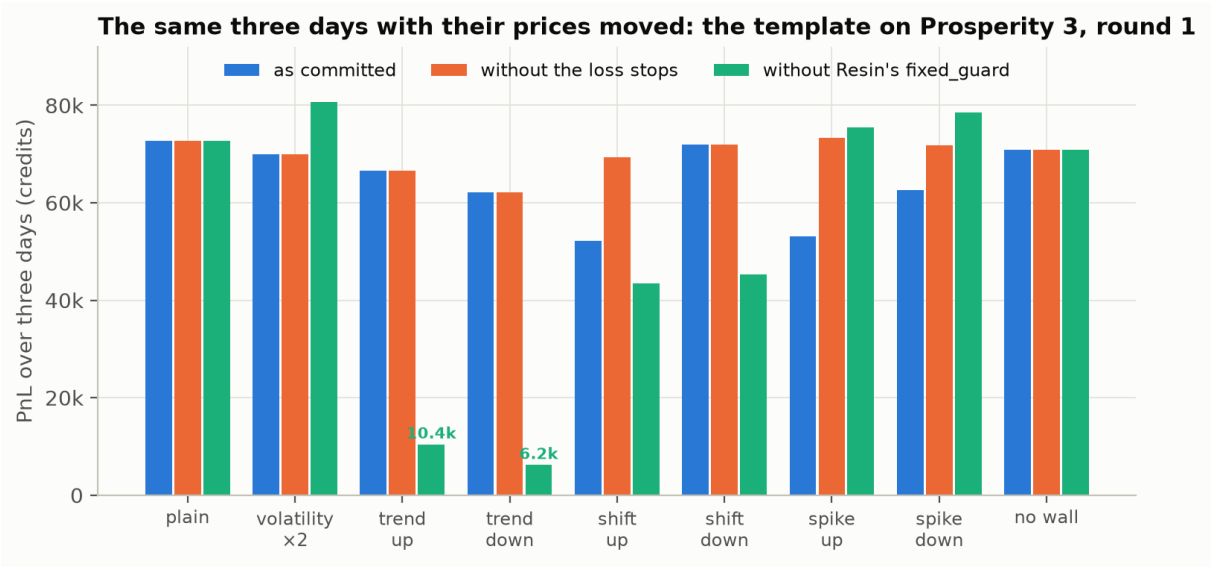


Figure 17: The template on Prosperity 3's round 1 under each of `arena.stress`'s perturbations: as committed, without its loss stops, and without Resin's `fixed_guard`. Data: `data/stress_variants.csv`, from `arena.stress python/bots/template.py 1` with `--set`.

The loss stop doesn't save it. The runs with and without the stops are identical when the guard is off, because the stop judges today's profit at the bot's own fair value, cash plus the position at fair (`loss_stop`, `python/bots/template.py:507`). A fixed fair that the market has left calls every trade a bargain, so on its own books the bot is winning. A stop can only see a loss its fair value can see.

With the guard on, the stops cost money instead. When the price shifts up and stays, Resin makes 32.0 thousand with its stop against 49.1 thousand without, and on the spike days 33.6 and 44.8 thousand against 53.8 and 53.9: the stop fires on the move, and the bot sits out the rest of a day that would have paid (`data/stress_variants.csv`). On Prosperity 4, in sample, the same rule kept every perturbed day's loss within its product's stop; held out, on Kelp and Resin over Prosperity 3's rounds 2 to 5 and ten `arena.sim` days, no stop fired on a plain day (the Prosperity 4 products have no held-out recorded day). It is `kept`, sized to about three times each product's largest plain-day drawdown (`IDEAS.md`, "A loss stop per product"). The first run of this test, before the guard existed, took the template from 81.3 thousand to 19.6 with a trend (`IDEAS.md`, "Stress: what a regime change costs"); the template has changed since, and the re-run above is today's.

The runbook's table for each round runs the stress test at T+1:30 and sizes the stops from it (`RUNBOOK.md`, "Each round"; `ON_THE_DAY.md`, "Stress the bot before submitting").

What would differ on another market

On ABIDES the signature plot is flat at short horizons and rises to about three times its 50-tick value at 1,000 ticks (chapter 3): no fast noise to fade, and moves that continue. Inside `arena.sim`, where the market maker re-centres on a shifted price and the takers follow, Resin's guard held: 214.4 to 223.5 thousand credits over ten days under each shift and trend with it, 66.4 to 112.9 thousand without the guard and the stops. Hydrogel's level bet was the exposure there too: 62.7 thousand on the trend up against 477.9 thousand on plain days (`data/sim_regimes.csv`; `IDEAS.md`, "Regime changes in a market that reacts").

If you know ML

The window is a bandwidth, and the autocorrelation should set it: a smoother shorter than the signal's correlation length tracks the signal instead of averaging out noise. The stress test is a robustness test under a hand-chosen distribution shift. Where the analogy breaks: a trend changes the right action, so robustness to it is paid for with the edge on the days the bot was built for.

In the repo

- The notes: `uv run python -m arena.diagnose DAYS --data D [--profile]` (`python/arena/diagnose.py`, thresholds at the top; the day-one wiki page maps each note to a choice).
- The perturbed days: `uv run python -m arena.stress BOT DAYS [--only trend_up,shift_up] [--size 2] [--set ...]`.
- An option's residual at any window: `experiments.day.p3_smile --window N` (part 2); levels held out: `experiments.day.p4_level_template`; jumps: `experiments.day.p4_jumps`.
- The parameters: `smile_window`, `fixed_guard`, `level`, `level_band`, `target`, `jump_min`, `stop_loss`.
- The rows: "Prosperity 2 round 4's coconut coupons with the template's options modes", "Trading toward a known level", "Fading jumps that snap back", "Stress: what a regime change costs", "A fixed fair that gives way", "A loss stop per product".

On the Arena

Written after the event, from the round handovers: which shapes `diagnose` found in each round's products, what `arena.stress` said before each submission, whether a round's prices changed regime between the download and the scored run, and what the stops and guards did when they did.

Go deeper

- Bailey, Borwein, López de Prado and Zhu, *Pseudo-mathematics and financial charlatanism*, §6.2 (PDF p. 21): the half-life of an autoregressive process, and why serial dependence makes overfitting worse.
- Ahuja and co-authors, *Limit order trading with a mean reverting reference price* (PDF pp. 2–8): market making on an Ornstein–Uhlenbeck price.
- Bouchaud and co-authors, *Trades, Quotes and Prices*, ch. 17 §17.1 (PDF p. 341): why trends make market making hard.

Beyond plain market making

What else can a bot trade?

You'll need the fair value (chapter 3), why making earns (chapter 4), informed traders (chapter 5), what a position costs (chapter 6), levels and jumps (chapter 7), fill models (chapter 10), and held-out days (chapter 11).

The story

In Prosperity 2, a team called Linear Utility went into round 4 in second place. The round added a coconut coupon, a call option on coconuts with a strike of 10,000 and 250 days to expiry. They priced it with Black–Scholes, saw its implied volatility swing around 16%, and traded the swings, as they had traded a basket the round before. To bet on the volatility alone they needed to hedge the coupon's exposure to coconuts. They couldn't fully: the coupon's delta was about 0.53, so 600 coupons, their limit, moved like about 318 coconuts, and they were allowed 300. About 18 deltas were left over, and they chose to hold them: "holding delta with vega was still positive ev (but higher var)" (`references/prosperity2-linear-utility/README.md:162-166`).

The variance came. They "got unlucky and lost money from our delta exposure", made only 145 thousand in the round, in that edition's seashells, and fell to 26th. Their verdict: "we were already second place and thus should've went for lower var to try and keep the lead" (`references/prosperity2-linear-utility/README.md:168-170`).

What failed was a piece of exposure they knew about and kept. Market making earns a little on every trade and holds as little as it can; every strategy in this chapter decides to hold a position, because something it measures says the price will come to it. The question each time is what the bot is really betting on.

The idea

A target instead of a quote

The template's market maker (chapter 4) has no view: it wants a position of zero and leans on its quotes to get back there. Each of the template's *target modes* replaces that zero with a number. Some rule outside the book says "hold +60 now", and the bot's execution moves toward it, normally at once, crossing the spread up to `clear_edge` from fair. With `cross_excess` set, it crosses only for the part of the gap beyond that many lots and works the rest passively (only Hydrogel sets it, at 25) (`python/bots/template.py`, `clear`; `IDEAS.md`, "Passive-first execution toward a target"). Market making can carry on around the target with whatever room is left.

The table lists the modes, what each bets on, and what Prosperity data said.

Mode	Bets that	Measured on	Verdict
`level`	a price returns to a known level	P4 Hydrogel, Velvetfruit	kept

Mode	Bets that	Measured on	Verdict
`basket`	a basket's spread against its legs returns to its usual value	P3 and P2 baskets	kept, taking only; hedging dropped here
`smile`	an option is mispriced against its underlying	P3 and P2 options	kept, take-only; the hedge dropped here
`follow`	an option carries its underlying's level bet, with more room	P4 options	kept
`copy`	one named trader knows where the price is going	P3's Olivia, P4's Marks	kept for Olivia; the Marks dropped here
`jump`	a jump in the price snaps straight back	P4 round 5	kept, off by default

Every row is a Prosperity result, so every verdict is a prior to re-test on the Arena's data, and "dropped here" means dropped on Prosperity's days, nothing more (`IDEAS.md`, rows by mode, below).

Levels

Chapter 7 introduced the level. In brief: on Prosperity 4's Hydrogel and Velvetfruit each day's mean sat close to the others': Hydrogel's between 9,989 and 10,003 credits, against standard deviations of 25 to 38 within a day (`IDEAS.md`, "Trading toward a known level"). So the template holds its full limit short once the price is more than `level_band` above `level`, and long once it is as far below. Choosing the level and settings on three days and scoring the fourth made 149.9 thousand credits on Hydrogel and 98.9 thousand on Velvetfruit over the four held-out days. Taking the level from the day so far instead of from the other days cost about a third: 184.9 thousand against 280.2 with the same settings (`data/level_mode.csv`, from `experiments.day.p4_level_template`). A level bet is only as good as the level.

Baskets and their spread

A *basket* is a product whose value is a fixed bundle of others. Prosperity 3's `PICNIC_BASKET1` contained 6 Croissants, 3 Jams and 1 Djembes; it traded on its own book, next to the books of its three legs. The quantity to watch is the *spread*, the template's `basket` mode's signal:

$$\text{spread} = \text{basket} - \sum_i w_i \times \text{leg}_i$$

with each w_i the number of units of leg i in the basket and every price a wall mid (`python/bots/template.py`, `basket_target`). The template then treats the spread as a level: short the basket when the spread is far above its usual value, long when far below.

The figure below (Figure 18) shows why that works on Prosperity 3. Over a day the basket and its legs wander together by hundreds of credits, while their difference swings around a premium and keeps returning to it within the day. But the premium moves between days: a mean of 70, then 44, then 32, against swings of 79 to 89 (`data/basket_days.csv`).

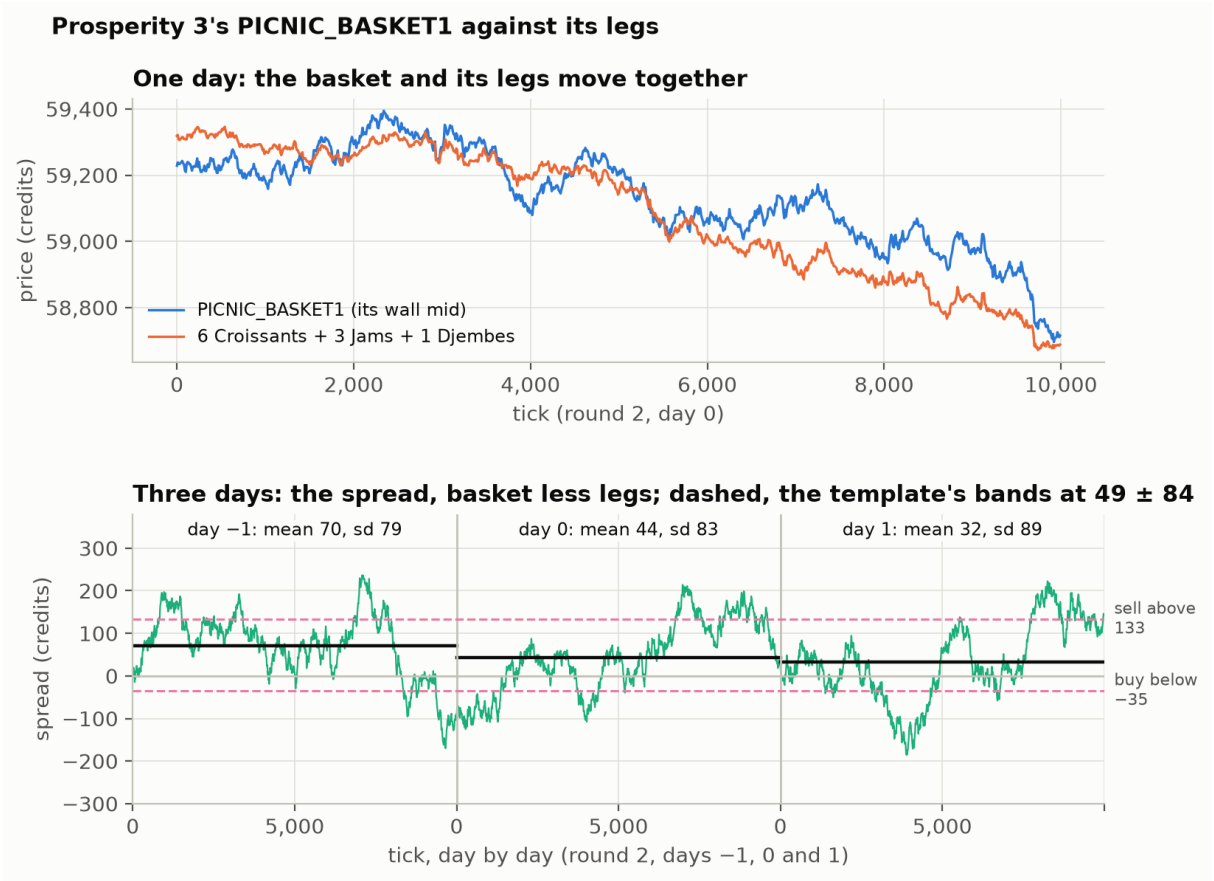


Figure 18: Top: Prosperity 3's `PICNIC_BASKET1` and its six Croissants, three Jams and one Djembes over day 0 of round 2. Bottom: their difference over the round's three days, each day's mean in black, and the template's bands at its level 49 plus and minus 84, dashed. Data: `data/basket_spread.csv`, from the days themselves.

With each day's level and band taken from the other two days, the baskets made 70.7 thousand credits over Prosperity 3's round 2, against 20.2 thousand for plain market making on the same products; chosen on all three days, 139.2. On Prosperity 2's gift basket, 294.3 thousand held out against 20.3 (`IDEAS.md`, "Trading a basket's spread"; re-run with `experiments.day.baskets`).

First, *hedging with the legs never won.* The spread closed mostly by the basket moving, so the legs only added costs: 122.2 and 122.4 thousand with half and with a full hedge, against 139.2 without. Second, the baskets only take. Their prices jump several ticks at once, so resting quotes on them were picked off: −321 thousand credits on Prosperity 3 under `--resting` (`IDEAS.md`, "Trading a basket's spread"). Baskets nobody announced, found by search on Prosperity 4's round 5, didn't hold on held-out days (`IDEAS.md`, "Residual baskets beyond an identity").

Options in a page

A *call option* is the right, not the obligation, to buy an underlying at a fixed *strike* price on a fixed day. If the underlying ends above the strike, the call pays the difference; otherwise nothing. So its value today depends on how likely the underlying is to end above the strike, and by how much, which depends on how much the underlying moves: its *volatility*.

Black–Scholes turns that into a price. It assumes the underlying's log price wanders like a random walk with a fixed volatility, σ, and asks what a portfolio that copies the option's payoff would cost. The answer is the underlying's price times one normal probability less the strike times another: no forecast of direction enters it, only the price, the strike, the time left and the volatility. Run it backwards and a market price gives an *implied volatility*: the volatility at which Black–Scholes reproduces the price. It is the market's price restated in units that compare across strikes.

If the model were exactly right, every strike would show the same implied volatility. In Prosperity 3 they don't: plotted against *moneyness*, the log of the strike over the price scaled by the square root of the time left, the implied vols trace a curve, the *smile*. The figure below (Figure 19) shows Prosperity 3's volcanic rock vouchers on all five of their distinct days. One parabola fits them all; fitted on four days, it misses the fifth by under one point of volatility on average (its worst point by 2.2 to 3.6) (`data/voucher_smile_fit.csv`; `IDEAS.md`, "Options: trading implied-volatility deviations from a fitted smile, delta hedging").

The last idea is *delta*: how many units of the underlying the option moves like. Under Black–Scholes, with no interest,

$$\Delta = N(d_1), \qquad d_1 = \frac{\ln(S/K) + \sigma^2 T/2}{\sigma\sqrt{T}}$$

with S the underlying's price, K the strike, T the time to expiry in years and N the normal distribution's cumulative probability (`python/bots/template.py`, `call_delta`; DTU's helpers, `references/prosperity4-dtu-quant-lab/round_3/algorithmic/trader.py:426-430`). Delta runs from near 0 far out of the money to near 1 deep in it, as Harris describes in words (*Trading and Exchanges*, §17.3.1.3, PDF p. 368). A trader who wants to bet on volatility alone holds minus delta units of the underlying per option: that is the hedge Linear Utility couldn't complete.

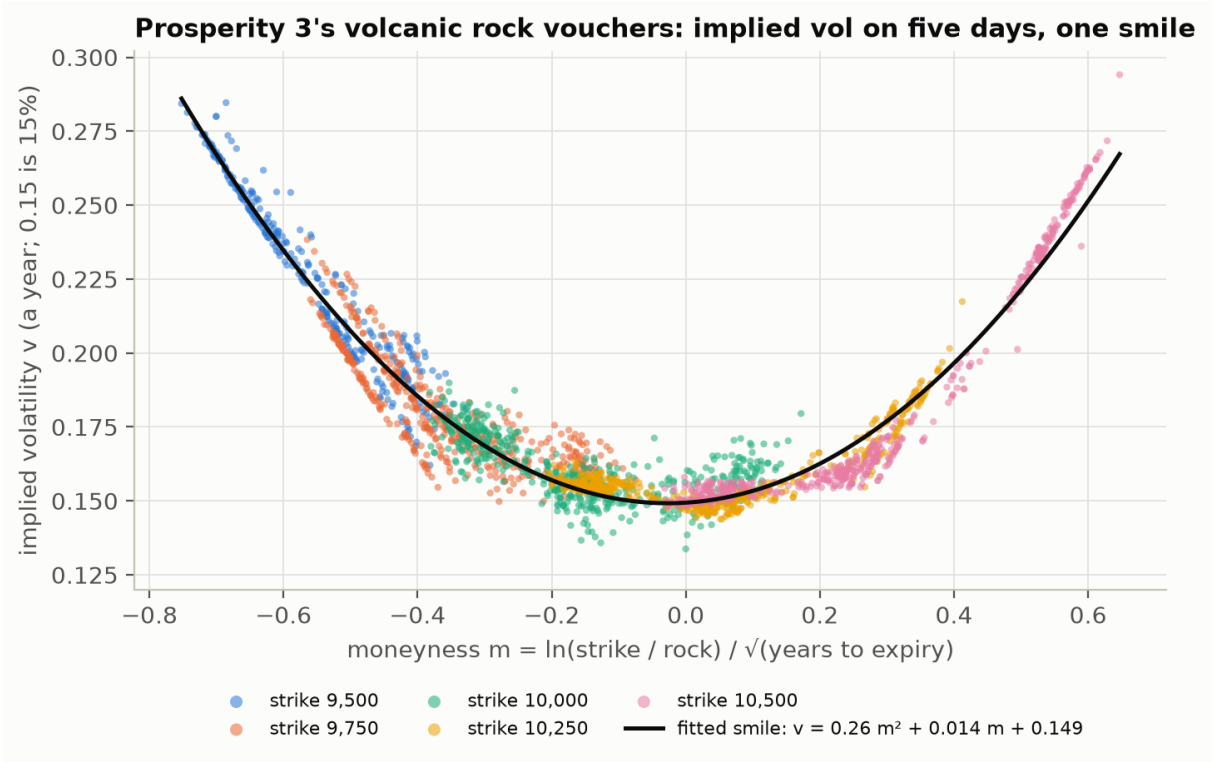


Figure 19: Implied volatility of Prosperity 3's five volcanic rock vouchers against moneyness, on the five distinct days with vouchers, and the parabola fitted to all of them. Data: `data/voucher_smile.csv`, from `experiments.day.p3_smile`'s points.

What paid on Prosperity's options

The template's `smile` mode prices each voucher at Black–Scholes on the fitted smile, plus the voucher's usual offset from it, and takes any book order that strays from that fair. On the two strikes near the money it made 528.8 thousand credits over the five days, every day positive, the same under every fill model. The controls took that apart. One flat volatility for every strike did as well (567.0 thousand with quotes, against 559.4 with the smile), and no option model at all lost 76.4 thousand. The controls suggest the edge was pricing each voucher off the rock's latest price, and that the smile's shape added nothing. The delta hedge cost 49.9 to 256.4 thousand in every variant, and quotes left resting on the vouchers lost 0.6 to 2.6 million (0.6 to 1.5 million unhedged) (`IDEAS.md`, "Options: trading implied-volatility deviations from a fitted smile, delta hedging"; re-run with `experiments.day.p3_smile --engine rust`).

Prosperity 4's vouchers had a flat volatility within each day, so each was the underlying at a fixed delta; the template's `follow` mode lets them carry the underlying's level bet with more room: 768 thousand credits held out over four days, Velvetfruit and its vouchers together (the vouchers alone 661 thousand) (`IDEAS.md`, "What the P4 vouchers are"). On Prosperity 2's coupon, Linear Utility's own product, the residual drifted instead of reverting, and `smile` paid only with an offset averaged over 5,000 ticks (chapter 7). And CMU Physics' fitted smile stopped fitting on the day they were scored; a short rolling window of implied vol took their backtest from about 80 thousand to 200 thousand a day, *their claim* (`references/online-snapshots/chrispyroberts_imc-prosperity-3.README.md:236`). On three markets, the smile was the least useful part of the textbook.

Copying the informed, and fading jumps

Two modes read something other than prices. The `copy` mode follows named traders (chapter 5's Olivia): each of her trades sets the target to the limit her way. On top of plain market making it took Prosperity 3's Croissants from 1.4 to 2.1 thousand credits a day to 13.5 to 25.9 thousand. Choosing whom to follow is the hard part. Picking names by their mean move on two days and trading them on the third lost 399.5 thousand; picking only names that were right on 80% of trades on each training day chose Olivia alone, and made 54.1 thousand held out, over Prosperity 3's round 5. The hit-rate rule was written after the mean rule failed, so it is not a clean test (`IDEAS.md`, "Following informed counterparties").

The `jump` mode fades jumps. On one product-day of Prosperity 4, `ROBOT_DISHES`' wall mid sat on multiples of 100 and stepped by 100, and 87% of its 741 jumps were followed by a move the other way (the jump mode's count; the diagnose note, with its own threshold, counts 743). Holding the limit against each jump made 958.8 thousand credits over round 5's three days against 457.8 for plain market making, 429.1 thousand of it on that one product-day. A gate makes it safe to switch on everywhere: it trades only once most of the day's jumps have reversed; without it, fading every jump made 184.1 thousand less than plain market making on one day, a loss of 37.3 thousand (chapter 7). All of these settings were chosen on these three days (`IDEAS.md`, "Fading jumps that snap back").

Edges a replay can't value, and edges that aren't allowed

Some edges a replay can't test. In Prosperity 4's round 1, one side of the book was empty on about 7.6% of ticks, and Baguette Fromage found that a hidden taker would sometimes lift a quote about 100 from fair on that side: an expected 18.6 thousand credits a day on Osmium, *their claim* (`references/prosperity4-baguette-fromage/README.md:352-365`). DU reports an admin saying the generator removed 20% of the bots' orders at random (`references/prosperity4-du-trading/README.md:77`). Our engine can't value it at all: almost no recorded trade printed outside the visible book, and none far enough out, so a quote 20, 50 or 100 from the wall mid fills nothing in any mode (`IDEAS.md`, "Quoting far away when one side of the book is empty").

And some edges are leaks. Hunting for how another team made 1.2 million in one round, Linear Utility regressed that year's prices on the *previous* year's public data: last year's diving gear, times about 3, predicted this year's roses with an R-squared of 0.99, and last year's coconuts predicted this year's with a beta of 1.25 (*their claim*). They called it "fair game", since the data was public, and spent the last round trading on known future prices (`references/prosperity2-linear-utility/README.md:176-182`). A year later the bots behaved like the previous year's to over 95%, so teams hard-coded trades at known timestamps. Frankfurt Hedgehogs reported it after round 2; IMC fixed the bots, banned hard-coding and re-ran the first two rounds (`references/prosperity3-frankfurt-hedgehogs/README.md:967-976`). CMU Physics, 771st when round 1 was first ranked (other teams had hard-coded the live day from the website's sample data, and a Squid Ink spike went against them), were 9th once it was re-run, where the spike went their way: "We got lucky" (`references/online-snapshots/chrispyroberts_imc-prosperity-3.README.md:70-72`).

A leak is the purest edge, and the one most likely to be taken away or to count as cheating; chapter 9 says how the repo keeps such a bot from being uploaded by accident.

If you know ML

A target mode is a forecaster whose output is a position rather than a price. The basket's spread is the residual of a linear model whose weights are given, and trading it is trading the residual's mean reversion, as with cointegration. Implied volatility is a reparametrisation, like logits for probabilities: a monotone transform that makes prices across strikes comparable, so a smile is a calibration curve. Where the analogy breaks: in ML a better-fitting model is usually the better one. Here the better fit (the smile) added nothing over a flat volatility, and the "complete" model, with the hedge, lost money.

In the repo

- The modes and their parameters: `python/bots/template.py` DEFAULTS (`level`, `level_band`, `level_exit`; `basket`, `hedge`, `hedge_ratio`; `smile`, `underlying`, `strike`, `expiry`, `smile_window`, `follow`; `copy`, `copy_sizes`; `jump_min`, `jump_share`, `jump_seen`; `clear_edge`, `cross_excess`, `cross_max`), and `basket_target`, `call_delta`, `smile_fair`.
- One experiment for each: `experiments.day.baskets`, `experiments.day.p3_smile` (and `p4_vouchers`), `experiments.day.counterparties` and `experiments.day.informed`, `experiments.day.p4_jumps`, `experiments.day.p4_level_template`; each takes `--data D` and the days for the day's download (`--days ...`, or positional days for `p4_vouchers`).
- The rows: "Trading toward a known level", "Trading a basket's spread", "Residual baskets beyond an identity", "What the P4 vouchers are", "Options: trading implied-volatility deviations from a fitted smile, delta hedging", "Prosperity 2 round 4's coconut coupons with the template's options modes", "Following informed counterparties", "Fading jumps that snap back", "Quoting far away when one side of the book is empty".
- On the day: `RUNBOOK.md`, "Each round", T+0:30 points at the experiment for each (`python/experiments/README.md`).

On the Arena

Written after the event, from the round handovers: which of these mechanisms the Arena's rounds had (levels, baskets, options, named traders, jumps, empty sides), what each experiment measured on their data, which modes went into the submitted bots, and whether the Prosperity verdicts held.

Go deeper

- Harris, *Trading and Exchanges*, ch. 17 §17.1–17.4 (PDF pp. 360–385): arbitrage of every kind, pure and speculative, with options and their delta (p. 368), pairs trading and the risks that sink arbitrageurs.
- Cartea, Jaimungal and Penalva, *Algorithmic and High-Frequency Trading*, ch. 11 (PDF pp. 290–300): pairs trading and the optimal bands for a mean-reverting spread, which `experiments.day.baskets` tests against the template's.

Who else is in the market?

You'll need the book and resting orders (chapter 2), the wall mid and ***`sized_mid`*** *(chapter 3), why making earns and what a copy takes (chapter 4), adverse selection (chapter 5) and fill models (chapter 10).*

The story

Prosperity's market holds only the exchange's own bots, so nothing in its data shows what another team could do to us. Our engine can put a rival into the same book (the `--vs` option), so we built some. One is a spoofer: each tick it rests a 40-lot order 10 ticks behind the best price on one side, where it rarely trades, switches side every 200 ticks, and takes any order that crosses its own idea of fair (`python/bots/rivals/spoofer.py`). It wants nothing from the 40 lots except to be seen.

Against the template with every product's fair value read from the wall mid, the largest level on each side, the spoofer won. On Prosperity 3's round 1, with orders resting, Kelp went from 16.8 thousand credits alone to −2.1 thousand with the spoofer in the market, and the spoofer made 24.3 thousand across both products, while our bot traded only 1,644 units with it. Kelp didn't lose by trading with the spoofer; it lost by trading with everyone else at the wrong price, because the 40 lots were the largest level and dragged its fair value toward them (`data/adversaries.csv`; `IDEAS.md`, "Adversarial rivals").

For Kelp the cure cost nothing. On `sized_mid`, the first level from the top with at least 15 lots, which the template now uses for Kelp, it made 16.8 thousand with the spoofer there as without it: the far spoof never moved a fair that no longer asked which level was biggest. The committed bot still loses 10.2 thousand to the far spoofer, all of it on Resin (`data/adversaries.csv`).

That is what this chapter is about. Everything earlier treated the market as weather, something that happens to the bot. When other teams trade in the same book, the market is partly people, some of them trying.

The idea

Alone, or in one market

There are two kinds of competition, and they need different preparation. In Prosperity each team trades alone against the exchange's bots, and teams compete only through the leaderboard: nobody can take our fills or see our quotes. In AlgoTrade 2026, the winners write, "all teams traded in the same shared market" (`references/algotrade2026-negative-eevee/README.md:7`), and UChicago's competition was shared too (`references/wiki/concepts/shared-market-and-adversaries.md`). For the Arena it is still unknown: the site calls its leaderboard a market simulation and lists "anticipate competitors" among the skills, but says nothing firmer (`ON_THE_DAY.md`, "Known so far").

In a replay, the only other traders are the recorded ones and whatever we bring. So every number in this chapter is from bots we chose to put in the market, and says what they would do, not what the other teams will.

What a second bot takes

Chapter 4 put copies of the template beside it: alone on Prosperity 3's round 1 it makes 72,674 credits, beside one copy each makes about 41 thousand (`data/copies.csv`; `IDEAS.md`, "The template against copies of itself"). Against three real teams' Prosperity 4 bots, with orders living one tick, the template kept 49 to 58% of its lone Osmium income (55,902 credits) against one of them and 28% against all three (`data/osmium_rivals.csv`; `IDEAS.md`, "The template against the teams' P4 traders in one market").

Theory says the same. In Grossman and Miller's model, as Cartea, Jaimungal and Penalva present it, a trader who must sell i units finds n market makers willing to take them only at a discount below the efficient price μ, and in equilibrium

$$|S_1 - \mu| = \gamma\sigma^2 \frac{i}{n+1}$$

with γ their risk aversion and σ^2 the variance of the price shock before the next trader arrives (*Algorithmic and High-Frequency Trading*, §2.1.1, PDF p. 41). Each extra market maker shrinks everyone's margin. The spread is a rent, and rents are shared.

Edges other than market making decay the same way, often faster. AlgoTrade's winners ran a cross-exchange arbitrage bot that took mispricings "out of the market as soon as they appeared"; "several teams mentioned that their own cross-exchange arbitrage became much worse around round 2, which was also when our first prototypes went live" (`references/algotrade2026-negative-eevee/README.md:23`). They also say which edges survive a crowd: mean reversion "still allowed other teams to enter later; cross-exchange arbitrage did not". The edge worth having in a shared market is the one others can't join late.

Which setting is best depends on who else is there

A rival changes not only how much we make but which setting is best. The script `experiments.day.rival_field` puts six versions of the template (Kelp's `make_edge` 0.5, 1 or 1.5, times Resin's 1 or 2) into 14 markets: alone, beside one archetype (a plain market maker at three edges, a taker, a sniper, a spoofer, a trapper, a copy of us), and beside mixes of three to six of them (Figure 20).

With orders that live one tick, five or six different settings were best across the 13 markets with rivals (six only through a four-way tie), and the one best against a copy alone was neither the best worst case nor the best average. With resting orders the choice matters far more. Resin's edge of 2 waits for prices a rival quoting nearer fair takes first: in a crowd of five bots the committed setting made 0.9 thousand credits where Resin at 1 made 28.5 (`data/rival_field.csv`; `IDEAS.md`, "Our settings against a field of rival archetypes in one market").

What if the rivals adapt too? With the `--game` option, a copy chooses among the same six settings and the two reply to each other in turn. Play always settled and never cycled, and with any fixed crowd behind the two of them, Resin at 1 was the stable reply. The committed Resin 2 is stable only between two copies alone. The verdict is still `open`: the committed edge stays, decided for the Prosperity products. The row's re-test scores candidates against a field

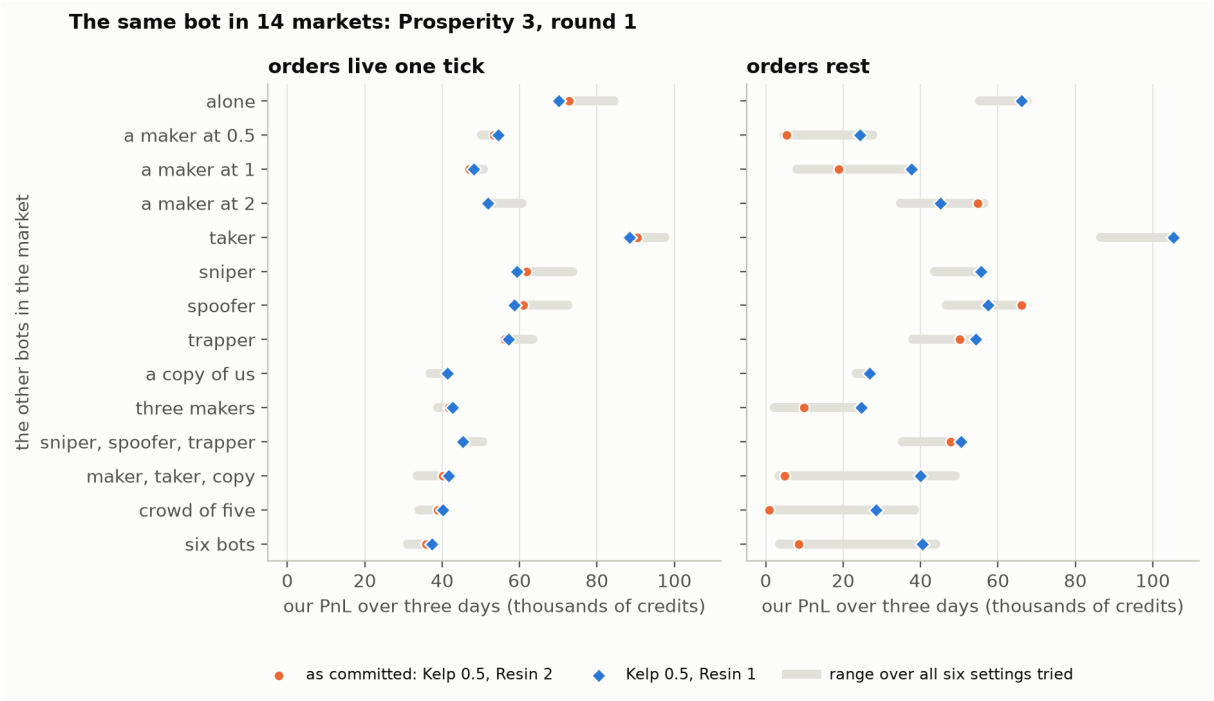


Figure 20: Our PnL on Prosperity 3's round 1 in each of 14 markets, for the committed settings (Kelp 0.5, Resin 2) and for Kelp 0.5 with Resin 1; the grey bar spans all six settings tried. Left, orders that live one tick; right, orders that rest. Data: `data/rival_field.csv`, from `experiments.day.rival_field --engine rust`.

and takes the max-min or the mean choice, never the reply to one copy; `ON_THE_DAY.md` repeats it (`IDEAS.md`, same row; `ON_THE_DAY.md`, "Is the market shared with the other teams?").

The herd: what an LLM writes

Most teams will hand the rules to a language model and start from what it writes. So the repo has a zoo: bots written blind by models from one spec of Prosperity 3's round 1, never edited afterwards (`python/bots/zoo/`). They are the baseline the template must beat and the rivals it may meet.

Alone, the best of them beat the template: Claude Opus's baseline made 99.2 thousand credits to the template's 72.7, most of the difference from Squid Ink (21.8 thousand), which the template doesn't trade. Under the strict fill rule they were level, 62.8 against 63.3. Head to head the Opus bot made 61.9 to the template's 40.5, and in a market of thirteen bots the template made 31.1 thousand and Opus's market maker 125.1. The models know the Prosperity winners' playbook: the Claude bots take, clear at fair and quote one tick inside the best, and five of six hard-code Resin at 10,000 from the spec's hint (`data/zoo.csv`; `IDEAS.md`, "The LLM zoo").

The same model and prompt can also write a disaster. Small local models wrote takers that paid the spread every tick, and quotes that crossed the book after every fill: gemma4's second market maker, its inventory skew in the wrong units, lost 1,979.5 thousand alone, and the template made 585.7 thousand next to it, against 72.7 alone (`data/zoo.csv`). One sample says little about a model, and a field holding such bots is where quoting every product pays most.

Bots built to hurt

The zoo is careless; the bots in the repo's rivals folder are built to hurt. In Prosperity 3's round 1 with resting orders, against the template as committed, the trapper (a large bait one tick inside the best, which sells to whoever steps in front of it) cost us 16.3 thousand and lost 34.3 itself; the spoofer one tick behind the best cost 0.5, the one 10 behind 10.2, all of that on Resin; the sniper, which takes any order one tick through its fair, cost 10.3. A plain copy of the template cost 39.7, more than any of them (`data/adversaries.csv`, from `experiments.day.adversaries --data p3 --days 1 --engine rust`).

What protects the committed bot from the near spoof is Resin's fixed fair, not `sized_mid`. With every product on `sized_mid`, the spoofer one tick behind cost 34.5 thousand (Resin from 45.8 to 12.6) and the far one 20.8 (`data/adversaries.csv`). A far spoof moves a fair read from the largest level; a large order near the touch still moves `sized_mid`, since it is the first level with enough lots.

Turning the attack around is harder than it looks. Our spoofer one tick behind the best, run with orders resting against the four Claude zoo bots, cost them 1 to 56 thousand more than they lost alone, while the spoofer itself lost 25.6 to 96.8 thousand every time. Hurting is easy; profiting isn't (`IDEAS.md`, "Manipulating other bots"). And a spoofer is exactly the kind of bot the gate below keeps from being uploaded.

Gray areas, and the gate

Some strategies trade well; others work by fooling the exchange or other teams: spoofing a fair value that reads size, pushing the price the close is marked at, using an exchange bug. In real markets the first two are illegal. In a competition they are whatever the rules say, and Prosperity's history shows organisers will rule: IMC banned the hard-coding of the previous year's bot behaviour and re-ran two rounds (chapter 8). The Arena's rules allow any AI tool and forbid help from outside the team (`ON_THE_DAY.md`, "Known so far"); whether they have a market-abuse clause is one of the questions for the organisers.

The repo's position is simple: we never game the exchange or other teams outside the rules. Such a variant may still be worth building and backtesting, to know what it would gain and to recognise it if someone else runs it. So the repo keeps one from being uploaded by accident (Figure 21). Any bot file that games the market must set a line setting `GRAY_AREA` to what it does; nothing detects a file that doesn't, so the rule is on its author. A file that sets it can't pass: the validator fails it, with "GRAY AREA:" and its warning in capitals, and `arena.compact` keeps the line and warns, and only `--allow-gray-area`, after an organiser has said yes in writing, turns the failure into a warning (`python/arena/validate.py`, `gray_area_check`; `python/tests/test_validate.py:300`). RUNBOOK's rules for the whole day repeat it: never uploaded without an organiser's yes.

The field is noisy too

Other teams also make mistakes, and their mistakes move the leaderboard as much as their skill does. In Prosperity 3, CMU Physics used Jasper's visualiser, which pushed their submission past 100 MB of memory; the platform restarted it, which wiped the rolling windows behind their basket and volcanic rock trades, "causing our trader to effectively buy and sell these products randomly". Together with an implied-volatility fit that stopped fitting on

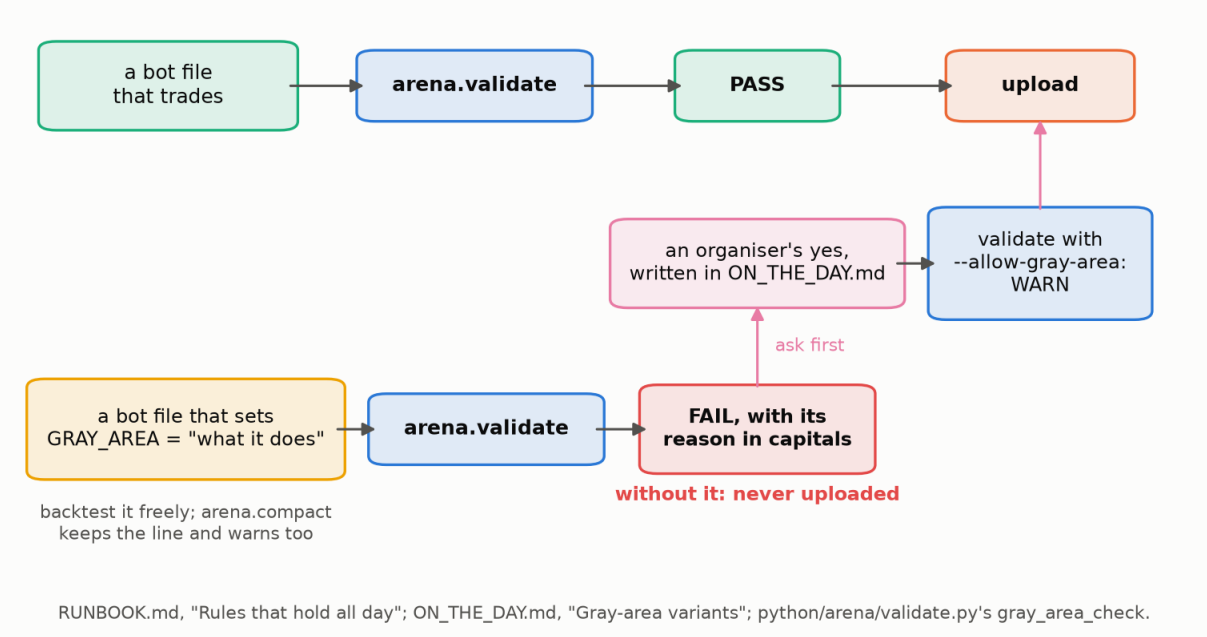


Figure 21: The gate: a bot that trades passes `arena.validate` and may be uploaded; a bot that sets `GRAY_AREA` fails it, and only an organiser's written yes and `--allow-gray-area` let it through. From `python/arena/validate.py` and `RUNBOOK.md`, "Rules that hold all day".

the day, it took them from 7th to 241st in that round (*their claim*) (`references/online-snapshots/chrispyroberts_imc-prosperity-3.README.md:230-236`). Meanwhile Alpha Animals' bug left them short the volcanic rock at its limit all day; it happened to pay, and took them to 2nd (`references/prosperity3-alpha-animals/README.md:135`).

So a rank is a noisy reading of the field: each team's bot, its bugs and its luck. After each deadline the leaderboard shows every team's curve, and `experiments.day.leaderboard_curves` reads from them whether the teams shared one market (`IDEAS.md`, "Reading the leaderboard's curves: alone, on the same data, or one shared market?").

If you know ML

This is multi-agent learning. The best reply to a fixed field is a best response; the max-min choice is a minimax policy; the `--game` option is self-play with fictitious play. Against the far spoofer, `sized_mid` is a robust estimator against a poisoned input: a rule that a large order far from the touch can't move, where a maximum used to be. Where it breaks: an ML adversary usually gets a budget and attacks one input, while a market adversary pays for every attack with its own position, which is why most of ours lost money. And there is no test distribution to hold out: the field is unknown until it trades.

In the repo

- Rivals in one market: `arena.backtest BOT DAYS --vs OTHER` (repeatable; `--requote` for bots written for orders that live one tick); copies, the zoo through `arena.compare`, and `python/bots/rivals/` (trapper, spoofer with `OFFSET`, sniper, taker).
- `uv run python -m experiments.day.rival_field --engine rust` (`--`

`candidate`, `--game`, `--background`), `experiments.day.adversaries`, `experiments.day.undercut`, `experiments.study.zoo`, `experiments.day.leaderboard_curves`.

- The gate: `GRAY_AREA` in the bot file, `arena.validate --allow-gray-area`.
- The rows: "The template against copies of itself", "The template against the teams' P4 traders in one market", "Adversarial rivals", "Our settings against a field of rival archetypes in one market", "The LLM zoo", "Manipulating other bots", "A spoof tell from the data".
- On the day: `RUNBOOK.md`, "Each round", T+1:30 and "After"; `ON_THE_DAY.md`, "Is the market shared with the other teams?" and "Gray-area variants".

On the Arena

Written after the event, from the round handovers: whether the Arena's teams shared one market, what the leaderboard curves said after round 1, what the round's zoo bots made against ours, what the market-abuse rules were, and whether any gray-area variant was built.

Go deeper

- Harris, *Trading and Exchanges*, ch. 11 §11.1.2 (PDF pp. 261–263): quote matchers, which the template is, and the traders who trap them.
- Cartea, Jaimungal and Penalva, *Algorithmic and High-Frequency Trading*, §2.1 (PDF pp. 38–41): Grossman and Miller, and what competition does to a market maker's margin.
- Wang and Wellman, *Spoofing the Limit Order Book: An Agent-Based Model* (PDF p. 4): the spoofer one tick behind the best, in an agent-based market.

How we know

Why is my backtest lying?

You'll need passive orders and fills (chapter 2), the fair value (chapter 3), why making earns (chapter 4) and adverse selection (chapter 5).

The story

In Prosperity 4 a team that calls itself Kayzerjali tuned one product, Pepper Root, in a backtester. Over one window of 100,000 timestamps the backtest said +2,415; the live platform, on the same window, said +436 (*their claim*). They didn't conclude that the strategy was bad. They concluded that the backtester's fill model "diverges severely from the live platform", and rebuilt their loop around it: the live result became the only thing they optimised, and the backtester became a check for bugs before an upload (`references/online-snapshots/Kayzerjali_Prosperity2026/run_cycle.md:3-7`).

A Prosperity 3 team, Frankfurt Hedgehogs, said the same thing more politely. The open-source backtester they used, Jasper's, was "very accurate and highly flexible", but it "could not … accurately model potential fill probabilities after inserting liquidity for assets like Rainforest Resin and Kelp" (`references/prosperity3-frankfurt-hedgehogs/README.md:898-900`).

Both teams hit the same wall, and it is not a bug in anyone's code. It is a question that recorded data cannot answer. This chapter is about that question: what a replay has to assume, how much each assumption is worth, and how to work when you can't know which one is true.

The idea

The question a replay can't answer

A backtest replays a recorded day: every tick's order book and every trade that printed. Our bot reads each tick and sends orders, and the engine decides what would have happened to them. Some of that is easy. An order that crosses the book, buying at or above the best ask, trades with orders that were really there, at their prices: a *take*, and the replay knows the answer.

A market maker's income comes mostly from orders that don't cross, *passive* orders waiting for someone to trade with them. When the recording shows a trade at our bid's price, would it have traded with us? The recording can't say, because we weren't there. That trade happened between two other bots. Had we been there, the seller might have hit us instead, or the other bot first, or not traded at all because our order changed the book it saw.

So every backtest engine answers the question by assumption, and the assumption is a setting. Ours has four main ones (Figure 22):

- `all` (the default, which matches the open-source Prosperity backtester): a recorded trade at our price, or through it, fills us.

- **worse**: only a trade strictly through our price fills us. If someone sold below our bid, they would surely have sold to us first.
- **none**: no recorded trades at all; only orders in the visible book fill us.
- **resting**: orders stay in the book between ticks and wait in a queue behind the orders already at their price.

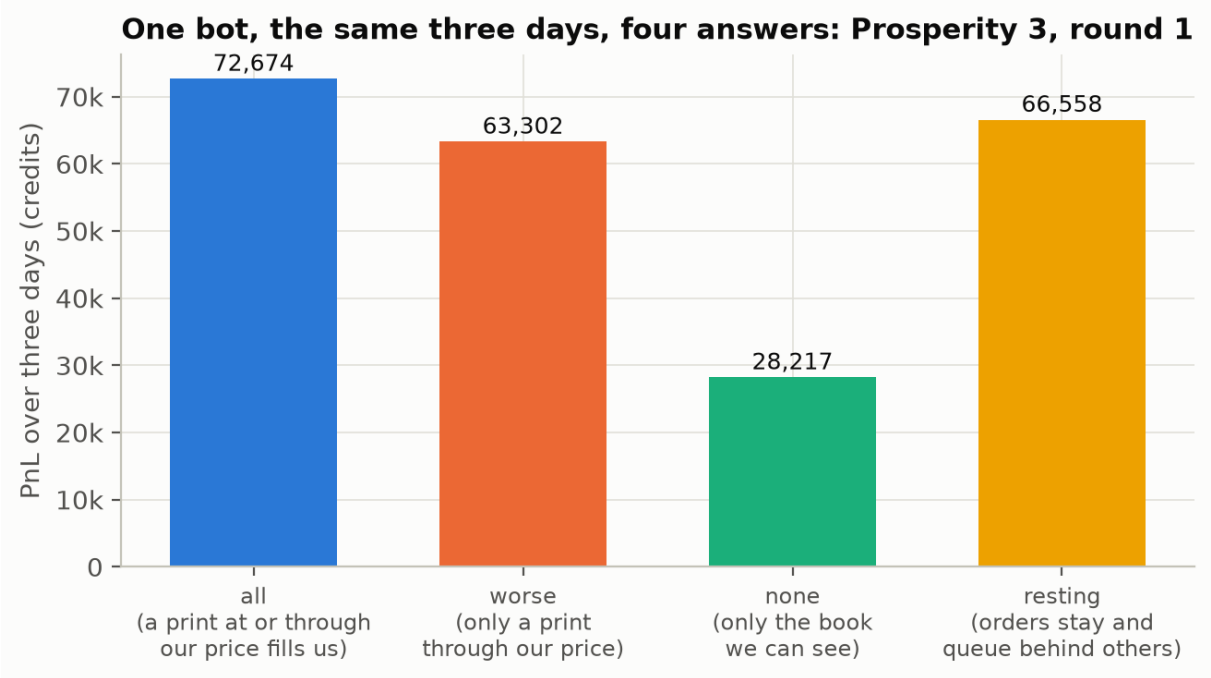


Figure 22: The template on Prosperity 3's round 1, all three days, under each fill model. Resting uses the default queue (`back`). Data: `data/fill_modes.csv`, from `arena.backtest python/bots/template.py 1 --data p3` in each mode.

The bot is the same, and so are the days. The answer goes from 28,217 credits to 72,674. More than half of the default comes from fills that only exist if bots which traded with each other would have traded with us instead (`README.md`, "Fill assumptions matter").

What the market offers at each distance

The replay's assumption can be taken apart, because one half of it is in the data. For a quote at distance d from the fair, the recording says how often a trade printed at d or beyond, and how many lots: that is what the market offered a quote there, before any queue or rival. What it can't say is whether those lots would have come to us.

Figure 23 measures that offer on Kelp. A quote at the wall mid sees a print at or beyond it on 13.9% of ticks, a side; at 1.5 ticks out, 8.0%; at 2 ticks, only 2.4%. Multiply the lots by the edge each one earns and you get what a quote at each distance could earn, under each rule. Under `all`, every print *at* the quote counts, and the best distance is 1.5 ticks: 0.44 credits a side a tick. Under `worse`, only prints *beyond* the quote count, and at 1.5 ticks most of those vanish, since the prints at 1.5 outnumber those beyond it more than two to one. The best distance becomes 1 tick, at 0.29 (`data/fill_curve.csv`, from `experiments.day.fill_curve --data p3 --days 1 --products KELP`).

That is the whole mechanism in one chart. The further out a quote sits, the larger the share of its fills that are the doubtful kind, trades printed exactly at its price between other bots. A rule that counts them rewards moving out; a rule that doesn't punishes it.

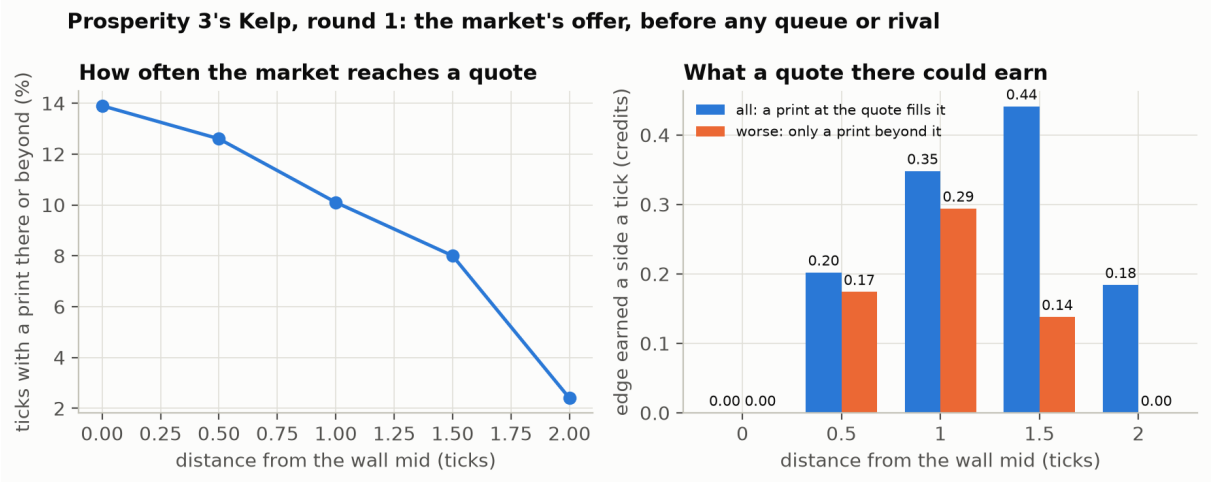


Figure 23: What a quote at each distance from the wall mid could fill and earn on Prosperity 3's Kelp, before any queue or rival: left, how often a print reaches it; right, its edge earned per side per tick under each rule. Data: `data/fill_curve.csv`, from `experiments.day.fill_curve`.

Why the error isn't noise

So the fill model isn't a small, random error. It is biased, and the bias depends on the very parameter you tune. The backtests say the same as the curve. The table gives the template's PnL with three settings of Kelp's `make_edge`, on Prosperity 3's round 1 and on three days of round 5 the choice never saw, in thousands of credits.

`make_edge`	0.5	1	1.5
under `all`	72.7	81.4	84.5
under `worse`	63.3	57.8	52.3
out of sample (round 5), `all`	53.8	61.1	64.5
out of sample (round 5), `worse`	48.8	44.3	38.8

Under `all`, the wider quote wins; under `worse`, it loses; and each rule ranks the edges the same way out of sample as in sample (`data/kelp_edges.csv`; `IDEAS.md`, "Why the zoo beat the template head to head"). Held-out data can't settle it, because the held-out days are replayed under the same assumption. **The fill model decides the edge.** A permutation test agrees: shuffling the order of Kelp's ticks finds the same best edge and the same lift over the median setting (a p-value of 0.55 for the best cell, 0.80 for its lift), so the choice comes from each tick's book and the fill rule, not from anything the price does over time (`IDEAS.md`, "A permutation test of the whole search").

Where we stand in the queue

When orders rest, a second assumption joins the first: our place in the queue. A new order joins the back of its price level. As orders ahead of it trade or are cancelled, it moves up. Trades are visible in the data; cancellations aren't, only the level shrinking. So the engine must guess where each unexplained drop happened, ahead of us or behind. Its three answers are `back` (always behind us, the pessimistic default), `front` (everyone else re-posts behind us every tick) and `prob`, which splits the drops by our place:

$$\Pr(\text{a cancellation is ahead of us}) = \left(\frac{\text{volume ahead of us}}{\text{volume at the level}}\right)^{N}$$

with N the `--queue-power`, 1 by default (`python/arena/engine.py`; the rule comes from the hftbacktest library). On Prosperity's round 1 the three hardly differ: 66,558 credits with `back`, 66,562 with `prob`, 67,070 with `front` (`data/queues.csv`). That is a fact about Prosperity's book, whose levels are redrawn almost every tick, so there is rarely a place in a queue to hold (`arena.diagnose`'s `kept` column, 8% on Prosperity; `ON_THE_DAY.md`, "Which backtest settings match the playground?"). On a book whose levels persist, the three models diverge, and only the real exchange's fills can choose between them.

The other things a replay assumes

Each of these is a setting in our engine, each was measured on Prosperity data or a rehearsal, and none can be read from the data itself.

When an order arrives. The replay matches an order against the book the bot saw. If the exchange applies it a tick later, the price has moved. One tick late, the template on round 1 falls from 72,674 credits to 40,472 under `all`, and from 63,302 to 31,218 under `worse`; two ticks late, to 36,336 and 28,014 (Figure 24). Resin's takes against its fixed fair arrive after the price has already come back. Part of the loss is a race against the limit: a late order is checked again on arrival, against a position its own earlier orders have moved, and one failure rejects every order for the product; one tick late, that happened on 3,248 of day 0's 10,000 ticks. Capping each quote's size at 15 to 20 lots recovers part of it: 45,534 under `all` one tick late (`IDEAS.md`, "Orders that arrive a tick late").

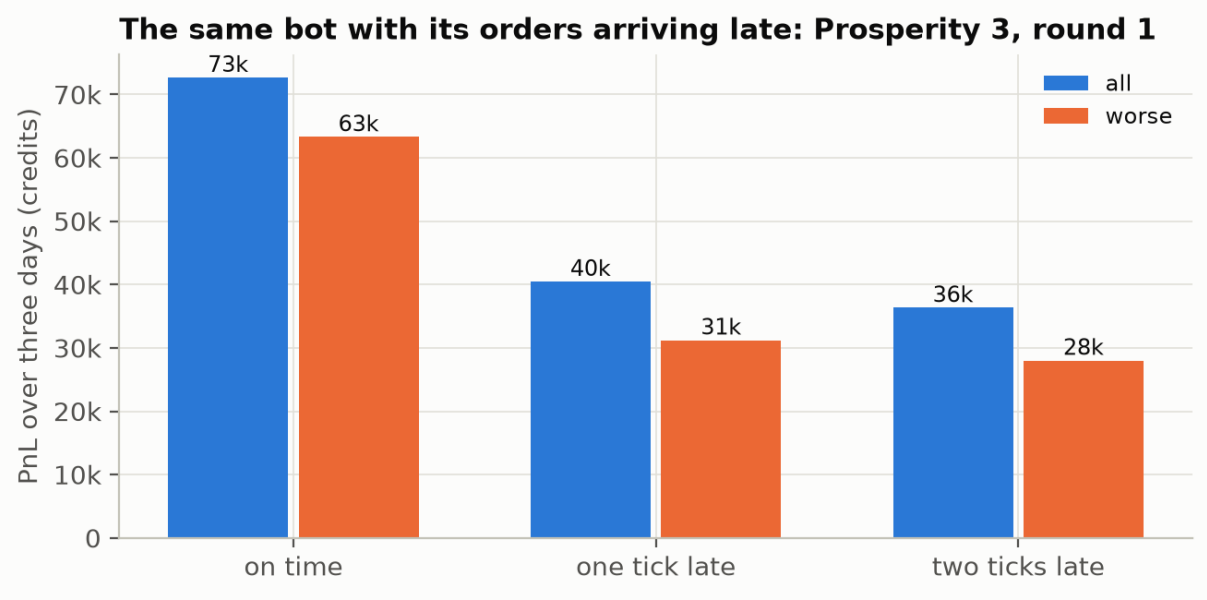


Figure 24: The template with its orders arriving on time, one tick late and two ticks late, under both fill models. Data: `data/delay.csv`, from `arena.backtest ... --delay N`.

Whether what we take stays gone. A replay usually restores the recorded book every tick, so the volume we bought is back for sale a tick later. In our first rehearsal the bot made 42,446, in that round's units, on the hidden days with the book restored, and 31,172 when what it took stayed gone: a quarter of the score from one assumption (`IDEAS.md`, "The kickoff prompts on a disguised round"; `--deplete`).

How much of each trade we could get. If others' hidden orders share every trade with us, we get only a part of what prints at our price. `--fill-ratio 0.5` halves the available liquidity: the template on round 1 then makes 40,184 credits over the three days in the default per-tick mode, instead of 72,674 (`README.md`, "Fill assumptions matter"). The Varde rehearsal's exchange did exactly this, giving resting orders half of each print, unannounced; chapter 12 tells how calibrate came closest with this setting and still refused.

Whether orders rest, on a market that jumps. With orders that live one tick, plain market making on the jumpy basket legs of Prosperity 2 and 3 earned a few thousand credits each. Let the same quotes rest, and the next snapshot jumps straight through them, filling them at their own stale price: Prosperity 2's Roses lost 1,477 thousand credits, having bought and sold 114 thousand units that way on one day (`IDEAS.md`, "Market making on products that jump, with resting orders").

What the position is worth at the end. A bot that ends the day holding something is scored on cash plus that position at some price, PnL = cash + position × mark, and the engine chooses the mark. `--mark` picks the mid, the touch or the last trade. The Varde rehearsal used the volume-weighted average of the last 100 trades and didn't say so (`python/experiments/mock_round/round4/README.md`); the engine has no such mark.

What to do about it

Three habits follow, and the repo is built around them.

1. **Never trust one fill model.** Run every comparison under at least two, `all` and `worse`, and believe only what wins under both (`README.md`, "Fill assumptions matter"). If the ranking flips, the gain is a property of the assumption.
2. **Measure the real exchange early.** One upload of a plain bot tells you how far the portal is from each of your settings, and its fills file can be read with the same fill curve as above, to measure our share of what the market offered (chapter 12).
3. **Prefer the edges a replay can see.** A take against the visible book is filled the same way under every fill rule, though not under every delay; income from passive fills is the part to doubt.

> **i** What would differ on another market
>
> On ABIDES, where informed traders move the price after their trades, every rule loses and they lie further apart: −21,798 cents under `all`, −57,872 under `worse`, −362,428 resting with the `prob` queue (chapter 5). On the Varde Exchange, the closest of our settings on fills was `all` with half of each print. The order of the rules' optimism is the same everywhere; how far apart they are is not.

> **i** If you know ML
>
> A backtest is a simulator, and the fill model is its sim-to-real gap. Two things make it worse than the usual gap. It is *one-sided*: optimistic fills always look better, so a search drifts toward whatever the simulator flatters. And it is *correlated with the parameters*: `make_edge` decides what share of your fills are the doubtful kind, so tuning it is tuning to the gap. Held-out days don't help, as they would against overfitting, because they share the simulator. The fix is the one for any simulator: measure it against the real system, and treat what remains as a range, here the span between `all` and `worse`.

In the repo

- The fill rules: `uv run python -m arena.backtest BOT DAYS --match-trades all|worse|none`, `--resting` with `--queue back|prob|front` and `--queue-power`, `--fill-ratio`, `--delay`, `--deplete`, `--mark mid|touch|last` (`python/arena/engine.py`; `ARCHITECTURE.md`, "Matching rules (Prosperity semantics)").
- What the market offered at each distance, and our share of it from a results file: `uv run python -m experiments.day.fill_curve --data D --days DAYS [--results RESULTS]`.
- Two models at once in a grid: `arena.grid ... --modes all,worse`; named bundles with `--scenario`.
- The rows: "The template's market maker", "Why the zoo beat the template head to head", "Orders that arrive a tick late", "Market making on products that jump, with resting orders".

On the Arena

Written after the event, from the round handovers: which fill rule, queue and delay the Arena's portal matched, how far its scores were from our backtests, and what that did to each round's choices.

Go deeper

- Bouchaud et al., *Trades, Quotes and Prices*, ch. 21 §21.3–21.5 (PDF pp. 413–421): limit against market orders, the value of a place in the queue, and (§21.5, p. 421) why backtests with phantom orders are biased.
- DeLise, *The negative drift of a limit order fill* (28 pp.): why the fills you get are, on average, the ones you wouldn't have wanted, and which fill rule a backtester should therefore use.

How do I know a result is real?

You'll need informed traders (chapter 5), the fill models (chapter 10), the permutation test's first appearances (chapter 5, as the shuffle test, and chapter 10), and `make_edge` *and* `take_edge` *(chapter 4). From machine learning you'll need nothing new: this is the chapter where your background transfers most directly.*

The story

Prosperity 4's last round had fifty products, and Baguette Fromage pointed scanners at them: cointegration tests, lead–lag relationships, book-imbalance signals. "At first it looked like we had found basically unlimited alpha." Whole families of products looked tightly linked; baskets reverted to their legs; products seemed to predict each other "with almost suspicious precision". That made them more sceptical, not less, so they forced themselves to fit on two days and test on the third. "The result was a disaster." Residuals stopped being stationary, hedge ratios changed sign, correlations weakened or inverted. "If you searched hard enough, it really was possible to find absurdly profitable backtests" (`references/prosperity4-baguette-fromage/README.md:866-902`).

The round before, the same data had shown teams the names of the bots behind each trade, "Mark 14", "Mark 22", "Mark 67", and every team went looking for an informed trader like Prosperity 3's Olivia (chapter 5). Several found one, and they didn't agree which: one write-up names Mark 67, another Marks 14 and 55, another 14 and 38 (`references/wiki/contradictions.md`, row 3). Ryan Challman's team fitted a weight per named trader on the training days; on the day they had held out, switching the weights off recovered about 15.8 thousand of profit, in the game's currency (*their claim*) (`references/prosperity4-ryan-challman/README.md:89-91`).

We ran two tests that settle it. First, for each named trader, product and side, the move after its trades on each of the three days: no Mark that traded more than a handful of times a day kept the sign of its move on all three and beat half the spread. Second, choosing names on two days by their average move and trading them on the third, three times over: the chosen names lost 30.5 thousand credits on the days they were tested on (`IDEAS.md`, "Following informed counterparties"). Every one of those signals looked real on the days it was found on.

The idea

A grid is a hyperparameter search, with very few folds

`arena.grid` runs every combination of a few settings and ranks them. The best of many noisy estimates is biased upward, the winner's curse, and the more combinations, the bigger the bias. What is unusual here is how little data there is. A Prosperity round gives three days; the Arena may give a handful. Three days are three folds, they are correlated, and the scored day may come from a different regime.

So the grid never ranks by one number. For each cell it prints `pnl`, the total over all days; `min day`, its worst day, because a setting that wins on average by winning big once is fragile; `nbhd`, the average over the cell and its neighbours, because a lone spike among poor neighbours is luck; and the PnL under a second fill model, usually `worse`, because of chapter 10. A setting near the top on all four is worth more than the single best `pnl`.

Hold out days, not ticks

The honest split is by day (Figure 25): choose on two days, score on the third, and repeat for each day. Splitting ticks at random would put a scored tick between two training ticks that share its price level and its regime, and every method would look good.

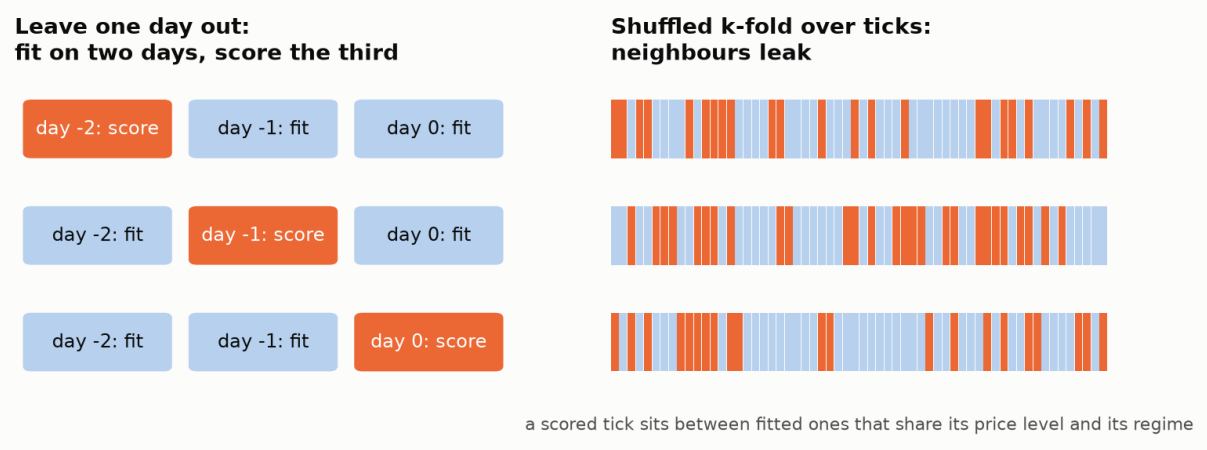


Figure 25: Leave one day out against shuffled k-fold. Days are the unit that can differ; ticks within a day share their regime.

`--holdout` goes one step further: it keeps a part of the data out of the whole choice, the last day or the last 25% of a single day, picks on the rest, and keeps the pick only if it also beats the bot as is on what was kept out. It is a test set in the strict sense, used once.

One grid, read in full

Figure 26 is Kelp's two edges on Prosperity 3's round 1, 24 cells under both fill models.

Under `all`, the case for quoting wider looks strong. The best cell, `make_edge` 1.5 with `take_edge` 1, makes 84,512 credits against the bot as is's 72,674 (`data/fill_modes.csv`), and it is best by total, by worst day (26,956) and by neighbourhood (80,404). Each held-out day chooses it again. Then the second fill model: under `worse`, the best cell is `make_edge` 0 with `take_edge` 0.5, at 63,912, and the wide cells are among the worst (`data/kelp_stats.csv`). Every statistic below is computed under `all` only.

PBO: does the in-sample best stay good out of sample?

Bailey and colleagues' answer to "is this search overfit?" is a cross-validation built for it, CSCV, combinatorially symmetric cross-validation (Figure 27). Cut the days into blocks, here 18, six a day. Split the blocks into two equal halves in every way there is (48,620 ways; the grid draws 4,000 of them). For each split, choose the best cell on one half and see where it ranks among all the cells on the other.

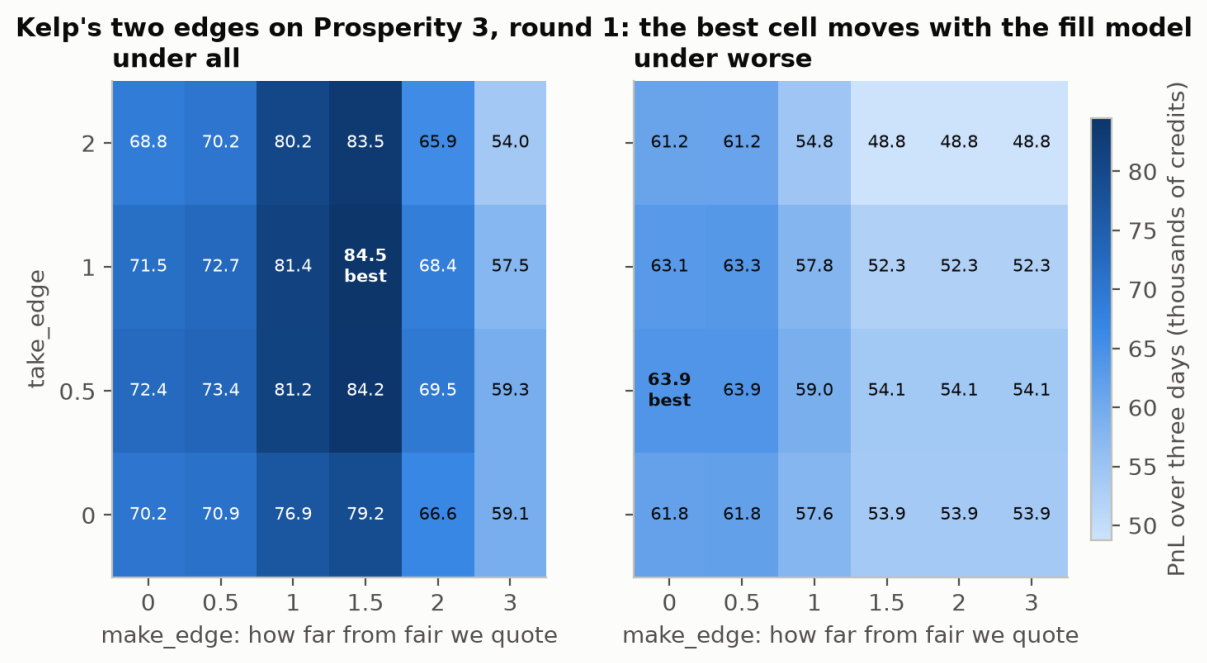


Figure 26: Kelp's `make_edge` against `take_edge` on Prosperity 3's round 1, PnL over three days (both products; only Kelp's settings change), under `all` and `worse`. Data: `data/kelp_grid.csv`, from `arena.grid ... --modes all,worse`.

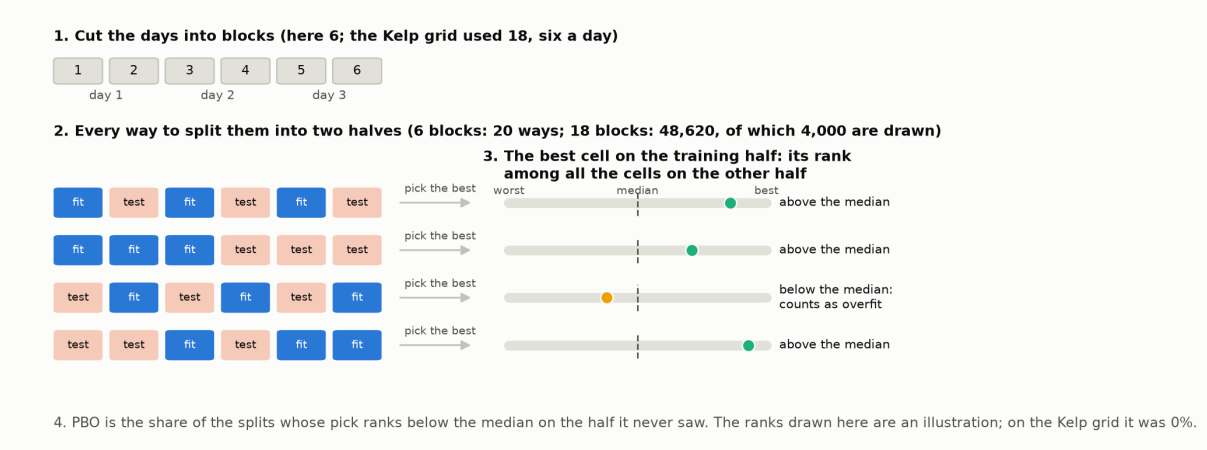


Figure 27: How CSCV estimates the probability of backtest overfitting: blocks, the halvings, and where each halving's pick ranks on the half it didn't see. A diagram; the ranks drawn are an illustration.

$$\text{PBO} = \Pr\left(\text{the in-sample best ranks below the median out of sample}\right)$$

This is the paper's Definition 2.2 (PDF p. 10), estimated by its Algorithm 2.3 (pp. 11–12). Near 0 is good; 50% means choosing was no better than picking at random. On the Kelp grid PBO is 0%: the halvings' picks rank, on average, at 0.95 of the way to the top out of sample, and none loses money there (`data/kelp_stats.csv`). On ten grids of pure noise built for the purpose, it averaged 46.5% (`IDEAS.md`, "Bootstrap tests of the best cell"), so the blocks next to each other don't leak enough to fake a result.

The DSR: is the best Sharpe ratio better than the best of that many with no edge?

A Sharpe ratio is a mean gain over its standard deviation. Run many cells with no edge at all and the best of them will still show a positive one, larger the more cells you try and the more their results vary. Bailey and López de Prado's deflated Sharpe ratio asks how likely it is that the best cell's Sharpe ratio beats that bar:

$$\text{DSR} = Z\left[\frac{(\widehat{SR} - \widehat{SR}_0)\sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3\,\widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4}\,\widehat{SR}^2}}\right]$$

Here Z is the normal distribution's cumulative function, T the number of blocks, $\hat{\gamma}_3$ and $\hat{\gamma}_4$ the skew and kurtosis of the gains, and $\widehat{SR}_0$ the best Sharpe ratio expected from $\hat{N}$ independent cells with no edge (eq. 2 and the line under it, PDF p. 8; eq. 1, p. 7). The cells of a grid aren't independent, so the paper discounts their count by their average correlation (eq. 9, p. 14): the Kelp grid's 24 cells count as 16.1 (`data/kelp_stats.csv`; `python/arena/stats.py`).

`arena.grid` applies it to the best cell's *gain over the bot as is*, block by block, since on the cells' own PnL it would only say that the bot makes money. On the Kelp grid: a per-block Sharpe ratio of 7.44 against a bar of 6.73, and a DSR of 0.74, under the paper's 0.95 (`data/kelp_stats.csv`). So the DSR hesitates where PBO didn't. The reason is in its bar, which rises with the spread of *every* cell's Sharpe ratio, poor ones included: on the wider 64-cell grid of the same two edges, where the DSR was 0.76, it was 1.00 computed over the nine best cells only (`IDEAS.md`, "Bootstrap tests of the best cell"). A wide grid's bad corners can hide a real gain.

SPA and StepM: does any cell, and which, beat the bot as is?

Hansen's test of superior predictive ability starts where the DSR stumbles. It compares every cell to one benchmark, here the bot as is, and asks whether any of them beats it, accounting for all of them. It improves on White's Reality Check, which the poor and irrelevant alternatives can drive to a non-result; Hansen studentizes each cell's gain and lets the data discount the cells that are clearly worse (PDF p. 1). In his own application, to forecasts of US inflation, adding a large set of poor forecasts moved the Reality Check's p-value from 10.6% to 96.3% and his test's from 4.8% to 10% (Table 6, PDF p. 13, and p. 14). Romano and Wolf's StepM answers the next question, *which* cells: it rejects the cells whose gain clears a bootstrap bar, removes them, and recomputes the bar for the rest, holding the chance of naming even one false cell at a set level (Algorithm 4.1, PDF p. 14). Both run on each cell's gain per block of 100 ticks, resampled in runs of random length so that the blocks' autocorrelation survives (Politis and Romano's stationary bootstrap). Each resample's spread is re-estimated from that resample, as Romano and Wolf's Algorithm 4.2 says (p. 15) (`python/arena/stats.py`).

On the Kelp grid, SPA gives a p-value of 0.000 that no cell beats the bot as is, and StepM names nine cells, led by `make_edge` 1.5 with `take_edge` 1, 0.5 and 2, then `make_edge` 1. The grid flags the disagreement with the DSR as expected (the wiki's contradictions, row 18) (`data/kelp_stats.csv`).

Why we read them at 4%

A test is only as good as its rate of false alarms, which we measured. On a thousand grids a row of pure noise (24 cells, 300 blocks, the cells correlated 0.7 with each other, six kinds of noise), StepM at its nominal 5% named a noise cell in 5.9 to 8.3% of grids, not 5%. Read at 4%, it did so in 4.5 to 5.2%, except where the gains were strongly autocorrelated, 7.0%. The price is power: a cell with a small real gain was found in 47.7% of grids rather than 51.1%. So the repo aims at a 5% error and reads both tests at 4%, `TEST_ALPHA` in `python/arena/stats.py`; with `--holdout` the pick's rate of false alarms falls to 2.2 to 4.0% (`IDEAS.md`, "Bootstrap tests of the best cell").

The pick

Every statistic so far was right about the data it saw. None could see that the data was replayed under an assumption. That is what the second fill model is for, and it is built into the one line the repo acts on, the *pick*: StepM's best cell that also beats the bot as is under every other mode. On the Kelp grid, only one of StepM's nine cells beats the bot under `worse`, and the pick is a small step, `make_edge` 0.5 with `take_edge` 0.5: +736 credits under `all`, +596 under `worse`. With `--holdout last`, the choice is made without day 0 and then checked on it: the same cell, +48 under `all` and +109 under `worse` on the day it never saw, so it is kept (`data/kelp_stats.csv`).

We chose this rule by testing ten ways of choosing a cell on days the choice never saw. Over six two-parameter grids on Prosperity 3 and 4, trained on one round and scored on the product's later days, picking the best PnL gained 12.2 thousand credits under `all` and lost 19.8 thousand under `worse`; on fresh simulated days, SPA with the `worse` gate lost in 2 of 120 runs, while rules that gained as much lost in 10 to 22. On the real days, from the committed settings, that rule never moved at all, so its record there is one of never acting, not of never losing (`IDEAS.md`, "Selection rules on unseen days"). That is a Prosperity result, to be re-tested on the Arena's own days once two rounds exist.

The permutation test: does the choice need the order of events?

Chapters 5 and 10 used this test, the first time as the shuffle test. Here is how it works. Make copies of the days whose ticks are shuffled, so that each tick keeps its book and its trades but the order between them is gone: no trend, no level the price returns to, no signal that leads it. Run the *whole search* on each copy, and compare two statistics with the real run's: the best cell's PnL, and its lift over the median cell. The p-value is the share of copies, the real days counted as one, at least as large (the grid's docstring; an adaptation of Masters' test, his book unread).

Figure 28 shows both outcomes. On Kelp's 64-cell grid of edges, the copies' searches find as good a best cell and as large a lift (the real lift 23,868 credits, the copies' mean 25,833): the choice of edge comes from each tick's book and the fill rule, not from anything the price does over time. On Prosperity 4's Hydrogel, gridding the band around its level and how hard to clear, the real lift is 124,762 against the copies' mean of 28,585 and their largest, 70,547: choosing

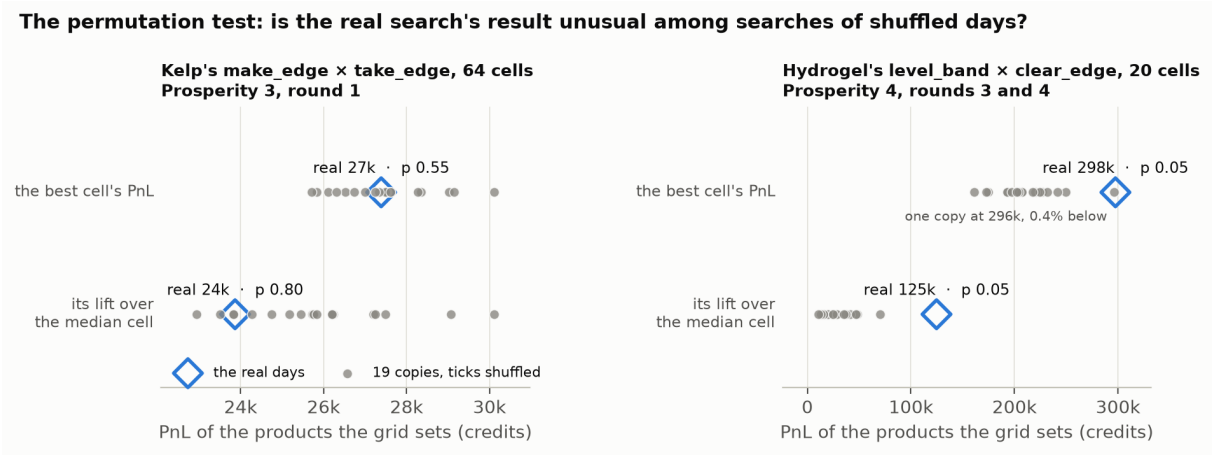


Figure 28: The permutation test on two grids, 19 shuffled copies each. Kelp's real search sits inside its copies (p 0.55 for the best cell, 0.80 for its lift); Hydrogel's lift sits far beyond every copy, and its best cell just beyond them (both p 0.05, the smallest 19 copies allow). Data: `data/permutation_null.csv`, from `arena.grid ... --permute 19`.

those settings exploits the price returning to its level, the positive control. The best cell's own margin is thin: 297,606 against one copy's 296,299, 0.4%, and with the ticks shuffled in runs of 100 rather than one at a time its p was 0.15 (`data/permutation_null.csv`; `IDEAS.md`, "A permutation test of the whole search").

When the profit is on one day

One failure no test above catches is concentration. Ryan Challman's `ROBOT_DISHES` made about 389 thousand, in the game's currency, on one training day against 11 and 4 thousand on the other two, 96% on one day; its lag-1 autocorrelation of returns was −0.29 on that day and about zero on the others (*their claim*). They didn't gate the strategy to that day's regime or size it to that day's number; they cut its weight from 0.12 to 0.06, as a hedge against the scored day not looking like it (`references/prosperity4-ryan-challman/README.md:115-141`). The `min day` column and the held-out days are there to show you such a profile before you size to it; chapter 13 is about the sizing.

> **What would differ on another market**
>
> On ABIDES, the same permutation test points the other way: shuffling an agent market's ticks removes the informed traders' moves, and plain market making on the copies made 4.5 million cents on average against −21,798 on the real days (`data/shuffle.csv`), so p is 1.0 and the order of events is everything. On the Varde rehearsal there were three public sessions, too few for any of these tests; the portal's standard error was the only measure of noise (chapter 12). The Arena may give one day of data a round: then `--holdout 25%` replaces the held-out day.

> **If you know ML**
>
> Everything here is model selection under multiple comparisons: held-out folds, a correction for the number of trials, bootstrap tests against a baseline, permutation nulls, and calibrating a threshold on synthetic nulls. Three things are harsher than usual, and that is where the

analogy breaks. There are three folds, not a thousand. The folds are correlated and the test day may come from another distribution, which every block-level statistic here is blind to, since its halvings and resamples mix blocks of the same days. And the simulator that scores every fold shares one bias (chapter 10), which no statistic computed inside it can detect, so the second fill model is part of the validation, not an extra.

In the repo

- `uv run python -m arena.grid BOT DAYS --grid PARAMS.P.key=v1,v2 --modes all,worse [--holdout last|25%] [--gate first] [--permute 19]`: the columns, the held-out days, PBO, DSR, SPA, StepM, the `pick` and the permutation test (`python/arena/grid.py`; the statistics in `python/arena/stats.py`, twinned in Rust).
- The null rates: `uv run python -m experiments.study.snooping_null_rates --engine rust`; the choice of rule: `uv run python -m experiments.day.selection_rules --part 1 --engine rust`.
- The rows: "Bootstrap tests of the best cell", "Selection rules on unseen days", "A permutation test of the whole search", "Following informed counterparties".

On the Arena

Written after the event, from the round handovers: which grids the rounds ran, what their PBO, SPA, StepM and picks said, whether any pick failed on the private run, and how many days each round gave to hold out. Until then, `RUNBOOK.md`, "Each round", T+1:00, and `ON_THE_DAY.md`, "Evaluation data" and "Which modes gate the grid's pick?".

Go deeper

- Bailey, Borwein, López de Prado and Zhu, *Pseudo-mathematics and financial charlatanism* (34 pp.): why backtest overfitting is the default. Then the wiki's page on validation and overfitting (`references/wiki/concepts/validation-and-overfitting.md`), and *The probability of backtest overfitting* (34 pp.), CSCV in full (pp. 10–12).
- Bailey and López de Prado, *The deflated Sharpe ratio* (pp. 7–9 for the formula, p. 14 for correlated trials).
- Hansen, *A test for superior predictive ability* (p. 1 for the idea, pp. 7–8 for the bootstrap, pp. 13–14 for the application); Romano and Wolf, *Stepwise multiple testing as formalized data snooping* (pp. 14–15, the studentized step-down).

What does the portal tell me?

You'll need PnL as cash plus position (chapter 2), the rehearsals (chapter 1) and a backtest's assumptions (chapter 10).

The story

In the Varde rehearsal, the fourth disguised round we ran before the event, the first upload scored €367.61 on the three public sessions. Before the order fee, which our engine can't charge, the portal's gross was €511; our backtest of the same file, under its best reading of the exchange's rules, had said about €1,330 (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`). And calibrate, the tool that searches our engine's settings for one that reproduces the portal, refused: no setting came within 5% of every figure (Figure 29).

So the session read the portal's results file by hand. The rules had said there was a fee per order placed as well as per lot, and it was easy to underrate: the bot placed and cancelled orders all the time, 28,647 of them, and at half a cent each they took €143.24 of a €510.85 gross, **28%** (`data/sessions.csv`; `python/experiments/mock_round/round4/results/DRYRUN_LOG.md`, "What calibrate printed"). The fills showed a second gap: the portal had filled far fewer lots than any of our settings that traded at all, 8,466 against 11,345 to 18,772. The rule behind it, that only half of each print reached resting orders, was not in the rules the session had; it sat in the rehearsal's answer key (`python/experiments/mock_round/round4/README.md`).

The first rehearsal had taught the same lesson from the other side. Its bot made 42,446 in that round's units on the hidden days, replayed the usual way, with the recorded book restored every step. With what it took kept gone from the book, it made 31,172: **27% less**, from one assumption about the exchange and nothing in the strategy (`data/depletion.csv`, from `python/experiments/mock_round/private/evaluate.py ... --days hidden [--deplete]`).

Neither gap was a strategy problem, and either would have misled every backtest that night. The portal's figures were the only place they showed.

The idea

The twelve figures

An upload comes back with a dozen numbers. On the Arena's portal they have these names (`ON_THE_DAY.md`, "Known so far"); the Varde rehearsal's portal reported the same ones, and its upload gave:

Figure	Varde upload	What it is
`score`	367.61	the PnL that ranks you: on the Varde portal, cash plus the positions at the mark, less any penalty (inferred for the Arena)
`fills`, `tradedVolume`	5,056; 8,466	how many fills, and how many lots
`inventory`	−6	the last session's end position, summed over products
`cash`	2,310.30	cash at the end, fees included
`markPrice`	233.44	the last session's marks, averaged over products
`breachPenalty`	0	the charge for going past a limit
`pnlSe`	22.98	the score's standard error
`absInventoryQ50` to `Q997`	8, 13, 27, 36	quantiles of the absolute position, pooled over the sessions

(`data/portal_metrics.csv`, from the rehearsal's `portal.py`) Four of them repay a closer look.

The score is an identity, if you know what is summed

It is cash plus position times mark, less any penalty, as in chapter 2, and you can check it. With one product and one session the check is exact. On the Varde upload it fails twice over, and nothing is wrong: `cash` is summed over all three sessions while `inventory` and `markPrice` are the last session's, and `markPrice` is one number averaged over three products' prices, one of which the bot never traded. So 2,310.30 less 6 times 233.44 is not 367.61; the identity holds only session by session and product by product (chapter 2 draws the parts). The session rebuilt the score from the results file, valuing each session's end position at its last mid, and got €369.67 against the portal's 367.61, within 0.6%: the score was the *sum* over the sessions, not their mean (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`).

That rebuild is the first of four checks the day's question file asks of the first upload: the identity; the whole path (fills, volume, end position and the position quantiles, not just PnL); the aggregation (is `score` a sum or a mean, and which standard error is `pnlSe`); and the clock (one continuous run, or a reset per day) (`ON_THE_DAY.md`, "How is PnL scored?"). Know which figures are sums before you compare any of them.

The standard error says how much the score is worth

On the Varde portal it is the standard error of the total over the sessions scored (`python/experiments/mock_round/round4/private/portal.py`):

$$\text{pnlSe} = \text{sd}(\text{session scores}) \times \sqrt{n}$$

The Varde sessions scored 110.21, 136.57 and 120.83 euros: their spread is 13.3, times the square root of 3 is 23 (`data/sessions.csv`). Two uploads whose scores differ by less than a couple of

standard errors are not shown to differ, however tempting the better number. With a handful of uploads a round, this is what stops you chasing noise. RUNBOOK has a second, separate bar: the engine's error, the gap between an upload's portal score and our backtest of it, measured by T+2:00, and never to choose between two variants by a portal score that differs by less than that (`RUNBOOK.md`, "Each round" and "Rules that hold all day"). One is the portal's own noise, the other our simulator's distance from it.

The Arena's portal shows a `pnlSe` too, but what it measures isn't known: it could be over sessions, over blocks of one run, or over repeated runs of a noisy simulation. So `arena.backtest` with `--metrics` prints our engine's version of every figure, with guesses at the standard error (ticks taken as independent, 20 blocks' sums, and, over several days, the daily PnLs' spread times the square root of the days), and calibrate tries per-tick, per-block and per-day versions and keeps the one that fits (`python/arena/report.py`, `format_metrics_total`; `python/experiments/day/calibrate.py`; `ON_THE_DAY.md`, "Probes", 16). One more thing can set the noise floor: Prosperity 4's portal scored each upload on a random 80% of the quotes, drawn again every time, so two uploads of one file differed (`ON_THE_DAY.md`, "Known so far"). If the Arena does that, the gap between two uploads of the same file is the smallest difference worth believing.

The position quantiles say how hard the bot leans on its limits

The quantiles pool the absolute position over the sessions, summed over products, and the two products the Varde bot traded allowed 100 lots between them. A median of 8 lots and a 99.7th percentile of 36 mean the bot was usually near flat, and all but 0.3% of the time used at most about a third of its room. A bot whose median sits near its limit is carrying a bet, whether it meant to or not, and chapter 6 says what that costs. The quantiles are also the cheapest check of the whole path: a backtest that matches the score but not the quantiles reached the right number by a different route.

Calibrate: fitting the simulator to the real thing

Our backtest has a dozen settings that recorded data can't decide (chapter 10): which prints fill us, whether orders rest and where they queue, delay, depletion, the mark, the share of each print we can get, fees and caps. One upload of a plain bot measures them. Calibrate replays the uploaded file under every combination, 144 settings and more, delay 0 and 1 among them, computes the same twelve figures for each, and ranks the settings by their *worst* figure's error against the portal. If the best is within 5% on every figure, it prints the settings to paste into the day's format plug-in, and lists the settings this upload can't tell apart (every figure within 0.5% of each other), which the probes must decide. If not, it refuses, and names the usual causes: the money off by one factor (a unit or scale mismatch), a cost the engine doesn't charge, or our wrapper doing something their exchange doesn't (`ON_THE_DAY.md`, "How is PnL scored?").

Figure 29 shows why it refused. The best-ranked setting, `none` with a one-tick delay, barely trades: it scores €7, which makes it "least wrong" on cash and inventory and 98% wrong on the score. The setting closest on fills and volume, resting with half of each print available, gets fills within 2.6%, but it scores €236.58 against the portal's €367.61 and ends the last session at −12 lots against −6, which throws cash off by over 300% (`data/calibrate.csv`). On top of that, the order fee is a cost the engine can't charge at all. No setting can be right everywhere when the exchange has rules the engine lacks.

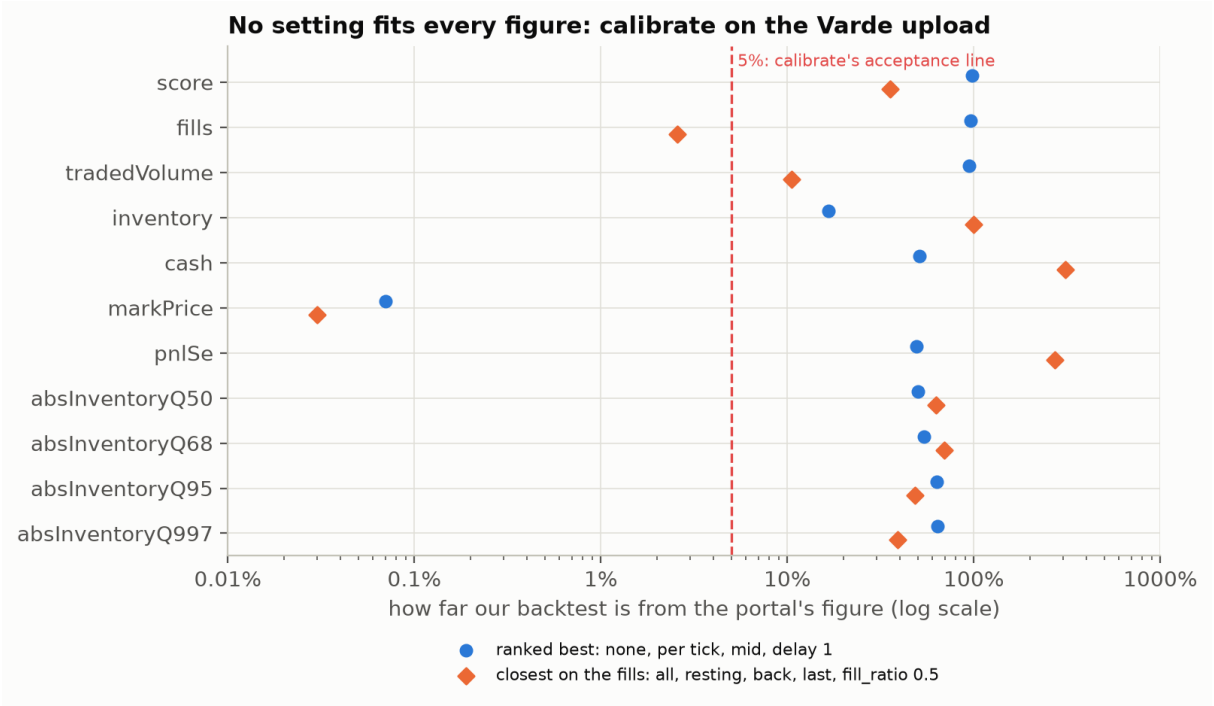


Figure 29: Calibrate on the Varde upload: each figure's error, for the best-ranked setting and for the setting closest on fills and volume alone, against calibrate's 5% line. Data: `data/calibrate.csv`, from `experiments.day.calibrate` through the round's plug-in.

Refusing a bad fit

The refusal was the most useful thing calibrate did that night. A fit that is right on some figures and wrong on others would have been worse than none: pasted into the plug-in, it would have tuned every later grid to an exchange that doesn't exist. The session did what the refusal says: it pasted no settings, kept the rules' own, widened the grid's modes to `all`, `worse` and `none` with a one-tick delay as a scenario, and read every local PnL net of the order fee. And it wrote down the scale of the gap: the local backtest overstated the portal about 2.6 times on gross (€1,332 against €511 over the three sessions), so until the next upload a local figure was a ranking, not a forecast (`python/experiments/mock_round/round4/results/DRYRUN_LOG.md`).

That last distinction is the one to keep. A biased simulator can still rank two variants correctly if the bias hits both alike; the fill model of chapter 10 is the case where it doesn't, because the bias depends on the parameter being tuned. So a refusal changes how a backtest is read, not whether it is used: every comparison under two or three fill models, and the portal's score, when it comes, as the judge.

When a gap remains, `experiments.day.shortfall` splits it per order: the same lots at a lower edge mean later or worse fills (try `--delay 1`); fewer lots mean the fill model differs, as on Varde, and depletion is the other candidate the first rehearsal named (`ON_THE_DAY.md`, "How is PnL scored?"; `IDEAS.md`, "Implementation shortfall and missed-trade cost per order").

> **i** What would differ on another market
>
> On Prosperity, the open-source backtester follows the platform's published matching, and the gaps teams reported were the fill question of chapter 10 (Kayzerjali's +2,415 against

+436 on one window, *their claim*). On ABIDES the simulator is the market, so calibrate has nothing to do. On the Varde rehearsal the gap was a hidden fee and a hidden fill rule, and calibrate's job was to notice. The Arena's portal is the first real target, and what its `pnlSe` and `markPrice` mean is unknown until its first upload.

If you know ML

Calibrate is system identification: fit a simulator's parameters so that its outputs match logs from the real system. Two things limit it, and they are where the analogy to an ordinary fit breaks. It sees twelve aggregate figures, not the hidden state, so very different settings can match the same totals; that is why it ranks by the worst figure, not the average, and why it lists the settings it can't separate. And the simulator's family may not contain the truth: refusing a bad fit is the same discipline as not deploying a model whose validation loss says it doesn't fit, however good its best metric looks. A misspecified model fitted anyway gives confident, wrong parameters.

In the repo

- `uv run python -m experiments.day.calibrate --bot BOT --days DAYS --format F --data D --portal metrics.json [--results results.zip] [--vary fill_ratio=0.25,0.5,0.75]`.
- Our backtest's figures in the portal's names: `arena.backtest ... --metrics`; a gap split per order: `uv run python -m experiments.day.shortfall --data D --days DAYS --results RESULTS`.
- The rehearsals: `python/experiments/mock_round/round4/` (README, and `results/DRYRUN_LOG.md`, "What calibrate printed"); the depletion check, `python/experiments/mock_round/private/evaluate.py ... --deplete`.
- The rows: "The kickoff prompts on a disguised round", "Implementation shortfall and missed-trade cost per order".

On the Arena

Written after the event, from the round handovers: the first upload's twelve figures; whether the identity held and what was summed; what the Arena's `pnlSe` turned out to measure; calibrate's run on that upload, its per-figure errors (G19, the Varde figure's twin) and whether it accepted or refused; and the settings that went into the plug-in. Until then: `RUNBOOK.md`, "Lunch (12:00–13:00)" and T+2:00, and `ON_THE_DAY.md`, "How is PnL scored?" and "Which backtest settings match the playground?".

Go deeper

- The wiki's pages on submission and scoring mechanics and on probing the environment: what other competitions' portals reported, and how teams measured an exchange they couldn't see (`references/wiki/concepts/submission-and-scoring-mechanics.md`, `probing-the-environment.md`).

How should I size risk?

You'll need PnL and the position limit (chapter 2), what holding a position costs (chapter 6), a result you can trust (chapter 11) and the portal's standard error (chapter 12). Written 4 November 2026, before the Arena's rules are known: everything said here about the Arena is what the organisers had said by then.

The story

Qminers' own 2024 hackathon was a sports-betting contest that ended in an in-person final: three rounds, every finalist's score reset to 1,000 points, a shorter horizon than the qualifier and so more luck in the result. Before it, the eventual winner wrote a small simulation of the final, of twenty finalists by the README's account (the committed script simulates sixteen teams), each with a model of its own quality and its own appetite for risk, the models correlated with each other (his "Quant factor", after a paper on quant funds that crowded into the same trades). He used it, by his account (*their claim*: the committed script returns nothing), to choose the fraction of his bankroll to bet, the fraction that won the final most often, not the one that made most on average (`references/qminers-hackathon-2024/README.md:41-49`).

He took the lead in the first round and extended it in the second. Before the third he worked out that its data came from just after covid, a period other finalists' models might handle better than his, and with a large lead he halved his stake. It was, in his words, "a minor mistake": his model "performed the best during this time", and at full stake he estimates he would have ended near 143,000 points. He kept "a dominant first place lead" anyway (`references/qminers-hackathon-2024/README.md:55-63`), finishing on 107,074 against the runner-up's 69,460; two finalists ended under 40 points (`references/qminers-hackathon-2024/final_scores.png`).

The point is not that halving was right or wrong. It is that he asked a different question from the one a backtest answers. A backtest asks what makes the most. A final asks what keeps the place.

The idea

What the score rewards

How much risk to carry depends first on how the competition turns PnL into a ranking, and competitions differ (`references/wiki/concepts/risk-and-protecting-a-lead.md`):

- **Prosperity** ranked teams by cumulative PnL: a large win counts in full, and so does a large loss.
- **UChicago's 2026 competition** scored each round by rank, the square of one less than a team's place among 41, the lowest total winning, and weighted the later rounds more (`references/uchicago-trading-2026/README.md:83`, `:221`). A big win counts no more than a narrow one, and a bad round costs more the worse it is, so consistency wins.

- **AlgoTrade** scored each round's profit on a log scale against the round's best, and any loss as zero (`references/algotrade2026-pitfa19/.firecrawl/algotrade-participant-guide.md:415-426`). A loss is capped and a win only compresses, which makes a long shot cheap.

The Arena, as far as anyone knew on 4 November: its leaderboard ranks each round by PnL, with no total across rounds; the site's code holds a weighted score over four phases at 10, 20, 30 and 40% that no page uses, so the weights are the site's and unconfirmed; and whether profit carries from one round to the next is unknown. The Technical Rules, published at the opening, should settle all three (`ON_THE_DAY.md`, "Known so far").

Rank is not expected profit

Under any of these rules, the objective is a place, and a place is a step function of your score against everyone else's. That changes the sign of risk. A team behind needs a large positive draw to pass the teams ahead, so variance helps it: the chance of a big enough result rises with the spread, even at the same mean. A team ahead only needs the others not to pass it, so variance hurts it: every extra unit of spread is a new way to fall. The 2024 winner's simulation is the general tool, and the repo has one for the last round: `experiments.day.standings` reads the leaderboards so far, simulates the last round many times, and gives our chance of first, of the top three and of losing a place for each risk multiplier and each hedge (`python/experiments/day/standings.py`).

Figure 30 is that calculator on a synthetic field, since there are no Arena leaderboards yet. The field was generated with 30 teams over three rounds whose medians and spreads were 5, 10 and 20 thousand in the field's units, a team's skill persisting from round to round at a correlation of 0.7, one common bet worth half a team's variance, and us 3 standard deviations of skill above the field, holding none of the bet. From those leaderboards the calculator measured what it would measure on real ones: the last round's field with a median of 2,883 and a spread of 23,934, persistence 0.52, the common bet 35% of a team's variance, our loading on it −0.10, and our lead 12,292 points with the rounds summed at the site's 10, 20, 30 and 40% (`IDEAS.md`, "A standings calculator for round 4"). The last round is simulated 5,000 times, seeded, for three versions of our last round at full risk: steady (a mean of 35 thousand, a spread of 11), weak (10 and 19) and volatile (35 and 49). The risk multiplier k scales our round's spread; on the left it scales the mean too, as a smaller position earns proportionally less, and on the right it leaves the mean alone, as if the risk cut carried no edge (`data/standings.csv`; `IDEAS.md`, "A standings calculator for round 4").

The two panels give opposite answers, and that is the lesson. If cutting risk cuts the edge with it, a leader with a good bot should keep its risk: halving the steady bot takes its chance of first from 94% to 78%. If the risk cut is pure noise, cutting it protects the lead, by a lot for the volatile bot (73% to 86% at half) and hardly at all for the steady one. Even then, "keep everything" was never far worse for a steady leader. So the question to ask before halving is not "am I ahead?" but "does the risk I'd remove carry edge?". The 2024 winner removed risk that, as it turned out, carried his best edge of the final, and gave up about 36,000 points, by his estimate, of a margin he didn't need.

Kelly: size by edge over odds

The classical answer to "how much of my bankroll" is Kelly's: bet the fraction that maximises the long-run growth of capital you reinvest. For a bet that wins b units per unit staked with probability p, and loses the stake otherwise, with q the chance of losing (Thorp, PDF p. 6):

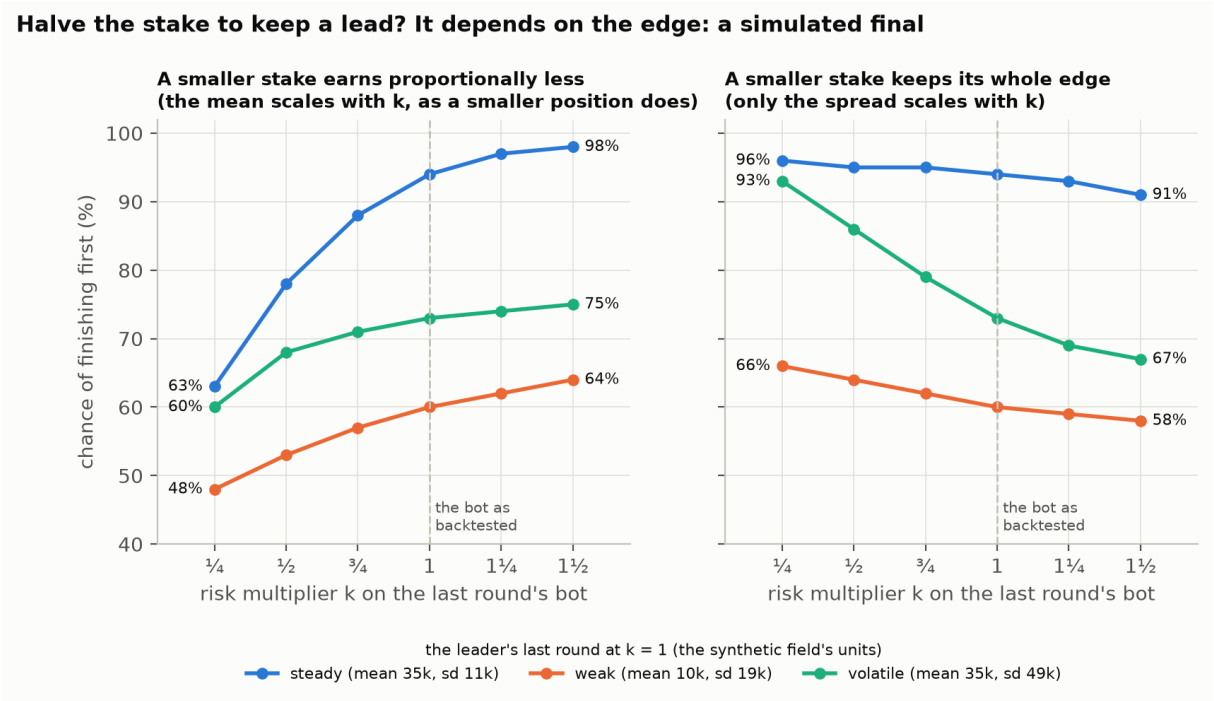


Figure 30: The leader's chance of finishing first against the risk it carries into the last round, in a simulated final: three kinds of last round, with the edge scaling with the stake (left) or not (right). An illustration of the method on a synthetic field, not a forecast of the Arena. Data: `data/standings.csv`, from `experiments.day.standings --synthetic --seed 4 --lead 3`.

$$f^* = \frac{bp - q}{b}$$

On an even-money coin that wins 53% of the time, the fraction is 0.53 less 0.47, or 0.06: bet 6% of what you have on each flip. Bet more than about 12% and capital shrinks to nothing despite the edge (Example 2.1, p. 6). Three facts from Thorp carry over to a trading round, where the formula itself doesn't. **Size follows edge over variance**: in its continuous form the fraction is the excess drift over the variance (p. 24). **Over-betting is the expensive mistake**: betting half the optimum keeps three quarters of the growth, and one and a half times Kelly keeps the same three quarters with three times half Kelly's volatility (p. 26). **Estimated edges are too high**, through data mining, markets that change and rivals drawn to the same edge, so bet between half and all of the *estimated* optimum, which protects against negative growth at a likely cost of at most a quarter of it (pp. 29–30).

What doesn't carry is the objective. A round's PnL isn't reinvested, the position limit caps the size where Kelly's bankroll would, nobody goes broke, and maximising log growth over many bets is not maximising a place over four rounds (`references/wiki/sources/kelly-criterion.md`, "Assumptions against the Arena"). In a bot, the Kelly-shaped knobs are the share of the limit a product's quotes use, `make_size`, and its `skew` (chapter 6): a product with a thin edge and a jumpy price earns a smaller share than one with a fat edge and a quiet price.

Ryan Challman's `ROBOT_DISHES` (chapter 11) is the third fact in practice. Its mean reversion made 96% of its backtest profit on one of three days (*their claim*). They read the other two days' small profits as noisy estimates of a real edge, not as noise around zero, and its size on the scored day as unknowable, so they cut its weight from 0.12 to 0.06 rather than size to the best

day: shading an estimate they had reason to think too high (`references/prosperity4-ryan-challman/README.md:115-141`).

Value at risk: size the bet to the lead

Kelly sizes against a bankroll; a leader can size against the lead instead. Prosperity 3's runners-up, Frankfurt Hedgehogs, led every round but the last and came into it 200,000 SeaShells ahead (the game's currency, which the rest of this book calls credits). Their option mean-reversion strategy had lost about 50,000 the round before, and they no longer believed in it on its own merits. But several teams close behind traded only that: if it paid, those teams would close the gap. So they kept some of it, as a hedge against their rivals rather than against the market. Having decided that, they checked its 95% value at risk, the loss it would exceed only one time in twenty: about 50,000, "only about 25% of our lead" (*their claim*) (`references/prosperity3-frankfurt-hedgehogs/README.md:616`). Turned around, their check is a sizing rule, our generalisation rather than theirs:

$$\mathrm{VaR}_{95\%}(\text{the bet}) \le s \times \text{lead}, \qquad s = \tfrac{1}{4} \text{ in Frankfurt Hedgehogs' check}$$

In that round, about 190,000 ahead of the second team, they also half-hedged their baskets and limited mean reversion, to play it safe (`references/prosperity3-frankfurt-hedgehogs/README.md:759`). They kept a healthy distance to every team that had been close behind, the ones they had hedged against, and still finished second: a team from below 100th place made about 850,000 in the last round, against the 200,000 to 400,000 typical of the top teams, and took first (*their claim*) (`references/prosperity3-frankfurt-hedgehogs/README.md:3`, `:616`, `:765-769`). A relative hedge protects against the rivals you can see, not against a long shot that comes off. The standings calculator's hedge table prices the same idea: the share of the field's common bet to hold so that a swing can't reorder the ranks. On the synthetic field, holding half to all of it cut the steady leader's chance of losing a place from 6% to 4% (`IDEAS.md`, "A standings calculator for round 4"). Small insurance, with the bet free to hold; on a field that isn't all in on one bet, there is nothing to hedge.

The tail you can't backtest

The cleanest case for sizing a risk to zero is one the backtest never shows. The winners of UChicago's 2026 Case 1 click-traded prediction markets by hand, from a physical DJ deck, on top of their algorithm. The exchange froze sporadically, and in round 6 it froze while they held large positions and news hit: about $200,000 gone in seconds, a top-5 round turned into 35th. They stopped trading by hand for the remaining six rounds, and wrote afterwards that, given a leaderboard that rewarded consistent placing, they should never have started. In the last round they computed that the top 11 of 41 was enough for first overall and stopped their bot after about 8 minutes, while other teams were manipulating the books in the final minutes (*their claim*) (`references/uchicago-trading-2026/README.md:197-203`, `:227-233`).

Two habits come out of it. Size a risk by what it can lose in its worst minute, not by its average contribution: a side bet whose expected gain is small and whose tail is large is negative under any rank-based score. And know, before the last round, what result is enough. In the repo, the per-product loss stop is the bot's own version of the first: `stop_loss` flattens a product and stops it for the day once its PnL falls that far below its high, sized from `arena.stress` at about three times its plain drawdown (`RUNBOOK.md`, "Each round", T+1:30; `IDEAS.md`, "A loss stop per product").

Round weights and carry-over

How the four rounds combine decides how much a round's lead is worth, and the day's question file works through each case (`ON_THE_DAY.md`, "How is PnL scored?"). If the total is a weighted sum of raw PnL, a lead counts in its own round's units, and round 1, a simple market while most teams are still learning the API, may be where a prepared team's lead is largest, so even at 10% it can matter as much as later rounds. If rounds are normalised or ranked, a big win counts little more than a narrow one. If profit carries over into the next round's capital, an early gain compounds and an early loss shrinks every later round, as in Qminers' 2024 contest (in 2025 a busted team was lent its 1,000 back) (`IDEAS.md`, "Round 1 as the prepared team's largest PnL lead").

So RUNBOOK leaves the decision to the one point where the information exists: before round 4, decide in writing how much risk to keep, and any relative hedge, from the standings calculator run on the three real leaderboards, our rank under each way the rounds may combine; and if rounds 2 and 3's private runs strayed from our backtests, a bot that sends nothing goes on the list (`RUNBOOK.md`, "Rules that hold all day"). Zero is a candidate, as a model that abstains is a candidate in a selective-prediction task.

What would differ on another market

On Prosperity, where the ranking was cumulative PnL, Frankfurt Hedgehogs sized their hedge in the game's currency against a lead in the same. Under UChicago's squared ranks, the same lead is worth nothing once first place is safe, so stopping was right. On AlgoTrade, with losses scored as zero, a trailing team's long shot cost little. The Qminers 2024 final compounded a bankroll, where Kelly's own objective almost applies. The Arena's rule picks among these, and none of the numbers above is a fact about it.

If you know ML

A competition ranked on one final evaluation rewards the probability of finishing first, not the expected score, the way a Kaggle private leaderboard rewards robustness over the public score; the standings calculator is a Monte Carlo over the field, exactly as you would simulate a leaderboard shake-up. Where the analogy breaks: on Kaggle you choose the model once, before the private score exists. Here the stake can change from round to round after you have seen the standings, so the choice of risk is itself a sequential decision on new information, and the rivals are making it too.

In the repo

- `uv run python -m experiments.day.standings --rounds data/arena/round{1,2,3}/leaderboard.json --team <ours> --ours <round 4's grid CSV>` (`--synthetic` for the field above; `--mean-power`, `--hedges`, `--weights`): rank, first place and losing a place per risk multiplier and per hedge, under four ways the rounds may combine.
- `uv run python -m experiments.day.leaderboard_curves --read data/arena/round<N>/leaderboard.json --team <ours>`: the field's curves, its common bet and each team's loading.
- The bot's knobs: `make_size`, `skew`, `stop_loss`, `flatten_from` (`python/bots/template.py`); `arena.stress` sizes the stop.
- The rows: "A standings calculator for round 4", "Round 1 as the prepared team's

largest PnL lead", "A loss stop per product".

On the Arena

Written after the event, from the round handovers: how the Technical Rules combined the rounds and whether anything carried over; G21, the rounds' real weights against what each round's PnL was worth to us; the standings calculator's run before round 4 on the three real leaderboards, the risk and hedge we chose in writing, and what that choice turned out to cost or save. Until then: `RUNBOOK.md`, "Rules that hold all day", and `ON_THE_DAY.md`, "How is PnL scored?".

Go deeper

- Thorp, *The Kelly criterion in blackjack, sports betting, and the stock market* (40 pp.): §2 for the coin (pp. 4–6), §7 for the continuous case and fractional Kelly (pp. 24–26), and the lessons on estimated edges (pp. 29–30). The wiki's source page on it lists what carries over (`references/wiki/sources/kelly-criterion.md`).
- The wiki's pages on risk and protecting a lead, and on harder rounds: how other teams sized risk with a lead, and how competitions make later rounds harder (`references/wiki/concepts/risk-and-protecting-a-lead.md`, `harder-rounds.md`).

The repo

How is the code laid out?

You'll need the round's loop (chapter 1), the order book, takes and resting orders (chapter 2), the fair value (chapter 3), take, clear and make (chapter 4), fill models and orders that arrive late (chapter 10) and the portal's figures (chapter 12).

The story

The third rehearsal, the Kestrel Bourse, was built to differ from our engine on purpose. Its rules said that orders rest, that a product may have at most three orders working, and that every lot beyond a position limit costs 0.25 credits a step. They left out what the first upload had to reveal, among it that an order reaches the exchange one step after the call that sent it (`python/experiments/mock_round/round3/README.md`; `private/portal.py`, `TRUTH`).

Two agent sessions played it, one on the repo before its calibration tools and one after. Both worked out the hidden rules within minutes of their first upload. Both lost money anyway: on the public sessions −4,850 and −283 credits, on the hidden ones −3,553 and −489, where the round's own plain market maker made +325 (`data/kestrel.csv`; `IDEAS.md`, "The kickoff prompts on a disguised round").

Most of the loss was not in the strategy but in the few lines between the strategy and the exchange. With orders a step late, an order sent at one step is, at the next, neither on the book nor gone: it is on its way. Neither session's bot counted those. So at each step it saw no order where it wanted one and sent another; the cap refused most of them; and the room under the limit was spent twice. When the late orders arrived and filled, the bot held more than its limit and paid for every lot over it: 2,192 credits of penalty on the first session's hidden days.

The repair changed nothing but the counting. The second session's bot with the template's `on_the_way` and `went` switched on made +142 credits on the public sessions instead of −283 and −389 on the hidden ones instead of −489; the breach penalty fell from 198 credits to 0 on the public days and from 25 to 0 on the hidden ones. What it still lost was one product, BRAM, a book one tick wide where, under the round's strict matching, a quote at the touch fills only when the price moves through it: −781 credits on the hidden days. That was a choice of parameters, not a bug (`data/kestrel.csv`; `IDEAS.md`, "The kickoff prompts on a disguised round").

That is why the bot file is split as it is, and why the split is worth a chapter.

The idea

The repo on one page

Figure 31 is the whole repo as one diagram.

Two rules hold it together. The engine knows no exchange: everything that knows one lives in a format plug-in and in the bot's ADAPTER. And every tool works on what the plug-in loads,

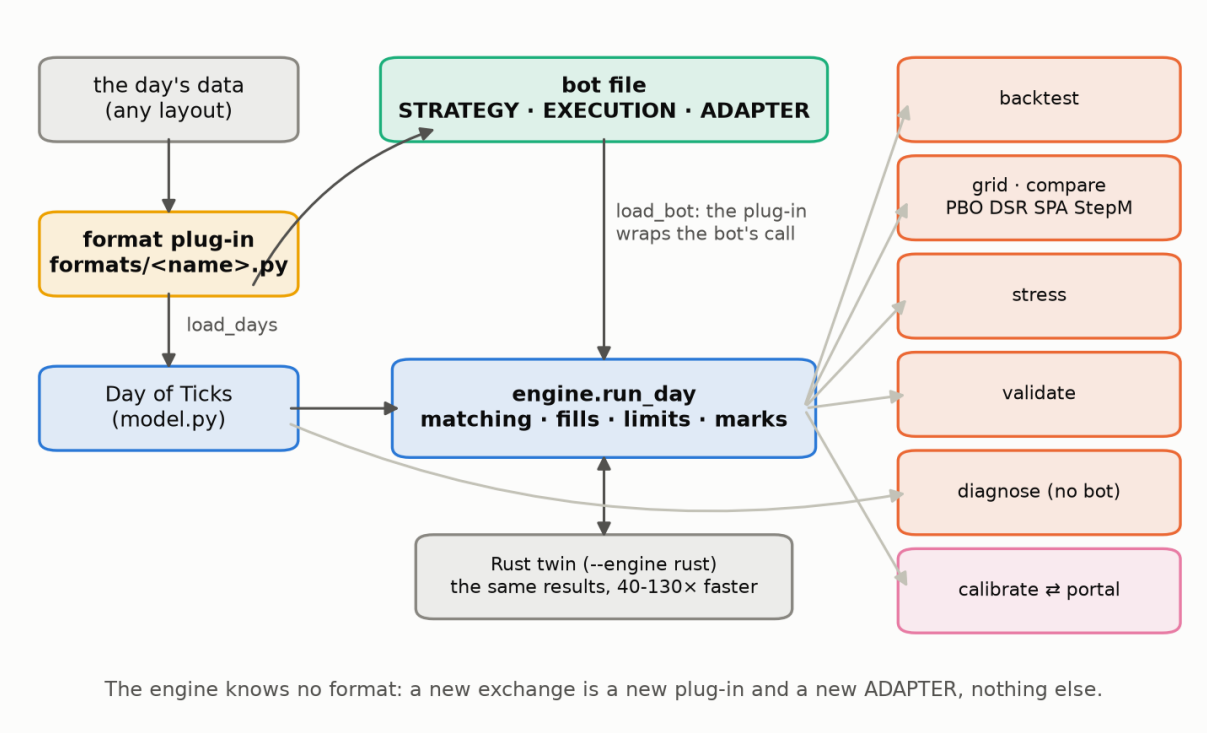


Figure 31: The repo's architecture. A format plug-in turns any exchange's data into Days of Ticks and wraps the bot's call; the engine and every tool work only on those. Drawn from `docs/guidebook/research/repo_map.md`, A.

so a new exchange costs those two pieces of code, not a new set of tools (`ARCHITECTURE.md`, "Architecture").

The engine: one loop over ticks

A `Day` is a list of `Ticks`, each a timestamp's books, the trades printed at it and its marks (`python/arena/model.py`). `run_day(day, bot, config)` replays a day against any object with a `step` method and returns a `DayResult`: every tick's positions, marks and PnL, every order, cancel and fill, every error and every limit breach. It is two lines long, because it is `run_market` with one participant (`python/arena/engine.py:223`). `run_market` is the loop, and each tick goes through it in the same order:

1. If orders rest, those the new snapshot crosses fill at their own price, and each one's place in its queue is updated from how its level changed.
2. Each bot is called with the book it sees (the others' resting orders included, never its own), its position, the fills it hasn't been shown, the market trades since its last call and its open orders.
3. With a delay, the decisions join a queue and those made that many ticks earlier come out instead.
4. The bots' actions reach the exchange in a seeded random order. The cap on actions drops what is over it; cancels apply; the caps on new and on working orders drop theirs; then, unless the format charges a penalty instead, the limit check rejects every new order for a product if all its buys, or all its sells, could carry the position past the limit.
5. Each new order trades with the recorded book and other bots' orders, best price first. What is left meets the tick's market trades under the fill rule (chapter 10). In the default mode it then dies; in resting mode it stays.

6. Positions are valued at the tick's mark, any penalty is charged, and the tick is recorded.

Every assumption chapter 10 described is a field of `EngineConfig`, so a mode is a setting, not a separate engine: `trade_matching`, `resting`, `queue` and `queue_power`, `fill_ratio`, `mark`, `limit_penalty`, `delay`, the fees, `deplete` and three order caps (`engine.py:118-140`). The default settings are Prosperity's rules, and they matched the open-source Prosperity 3 backtester fill for fill; a test keeps them that way (`ARCHITECTURE.md`, "Parity with prosperity3bt is pinned").

Formats are plug-ins

A format plug-in, one file in `python/arena/formats/`, is the only code that knows an exchange's files and its bot interface. It must provide five things:

- `PROVIDED_MODULES`: the modules the exchange lets a bot import, such as Prosperity's `datamodel`, put in place before the bot file runs.
- `resolve_data_dir`: turns `--data` into a folder (`p3` is the Prosperity 3 data in `references/`).
- `load_days`: reads the folder into `Days`, a limit for every product or an error.
- `load_bot`: wraps the bot file: it builds the exchange's state from a tick, calls the bot's entry point and converts the answer into the engine's orders and cancels.
- `check_source`: what `arena.validate` checks about the entry point, such as a class `Trader` with a `run` method.

Three more are optional. `RESTING`, set to true, says the exchange's orders rest until cancelled, so every tool runs as with `--resting`. `ENGINE` holds the other engine settings the exchange's rules fix, the queue, the mark, a fee, and every tool starts from them (`arena.cli.format_config`); a flag on the command line still overrides one. `CREDITS` is what one unit of the quoted price is worth per lot where it isn't 1, such as the Kestrel's points at 2 credits (`python/arena/formats/__init__.py`; `ARCHITECTURE.md`, "Formats are plug-ins").

There are five plug-ins: `prosperity.py`, the stand-in the prep ran on; `drill.py`, a practice format made unlike it on purpose; `snapshots.py` and `orderlog.py`, skeletons for shapes the Arena's data might take; and `qminers.py`, the Arena's own, registered already as a copy of `snapshots.py` marked `TODO(day)`, to be filled in on the day by the format swap's checklist (`ON_THE_DAY.md`, "The format swap, step by step").

The bot: PARAMS, then three sections

The template (`python/bots/template.py`) is one self-contained file of about 1,100 lines, standard library only, because the competitions run one file. Its state between ticks lives in the string the exchange hands back each call (`traderData` on Prosperity). It opens with four globals the ADAPTER owns: `STRICT` (raise on errors; the backtester sets it), `DELAY`, `ORDERS_REST` and `MAX_RESTING`. Then the settings. `DEFAULTS` holds 71 of them, each with a comment saying what it does (`template.py:45`); `PARAMS` has one entry per product with only what differs (`template.py:165`). A product's settings are the two merged, the entry winning. A product not in `PARAMS` isn't traded; a key not in `DEFAULTS` raises, which catches typos; and an entry without `limit` raises too, because the default limit of 50 is a placeholder (`params_for`). Chapter 15 goes through the settings one group at a time.

STRATEGY says what the bot wants, as a `Plan` for each product: the orders that trade with the book now (`takes`) and the quotes that wait (`quotes`), kept inside the limit as they

are added. The room to buy ignores planned sells, because both sides can fill in the same tick. `trade_product` (`template.py:316`) runs the same steps in the same order for every product:

1. **The fair value**, by the product's `fair` method (chapter 3), or from an options smile.
2. **The target**, the position to work towards, 0 unless something sets it. The first that applies decides: the close (`flatten_from`), a fired loss stop, the volatility guard, then the target modes in a fixed order, `basket`, `level`, `copy`, `jump_min`, `follow`, `hedge` (chapter 8). `arena.validate` names any setting that can't matter because an earlier mode wins (`param_conflicts`).
3. **Leans on the fair**: the book's imbalance and the linear terms, each off by default.
4. **Take**: trade with any order priced at least `take_edge` better than the fair.
5. **Clear**: trade back towards the target with orders up to `clear_edge` worse than the fair.
6. **Make**: quote each side `make_edge` from the fair, or `make_inside` ticks inside the best remaining quote if that is further out, moved by `skew` against the position, never crossing the book.

Take and clear use up the levels of a copy of the book, so make sees only what is left.

EXECUTION turns the plan into orders, given what is already working. `execute` (`template.py:939`) keeps a resting order, and its place in the queue, while a quote on its side at its price still wants at least its size, and sends only the difference; it cancels every other resting order; it sends takes as immediate-or-cancel, so nothing rests at a crossing price; and it keeps within `MAX_RESTING`. Where orders arrive late, `went` (`template.py:999`) writes each call's cancels and new orders into the product's memory for `DELAY` calls, and `on_the_way` (`template.py:985`) reads them back: those orders start the next plan as already planned, and the resting orders whose cancels haven't landed keep their room, because they may fill first. With `DELAY` 0 nothing is kept. This is the Kestrel repair, and it lives here because it is the same on every exchange.

ADAPTER (`template.py:1010` to the end) is, with the `datamodel` import at the top of the file (`template.py:28`), the only Prosperity code in it. It reads the exchange's state into the template's own book, trades, fills and resting orders, and writes the plan back as the exchange's `Order` and `Cancel`. `Trader.run`, the entry point, wraps each product in its own `try`: a product that raises is skipped for that tick, its resting orders cancelled and each distinct error printed once, so one bug can't stop the others trading. On the day, only the imports, `PARAMS` and this section are rewritten (`ON_THE_DAY.md`, "The format swap, step by step", step 5).

One tick, followed through

Figure 32 follows one real tick through those stages: Kelp on Prosperity 3's round 1, day 0, at timestamp 405,400, with the template as committed (`data/one_tick.csv`).

The bot comes in short 13 lots. The book shows one bid, 20 lots at 2,033, and three asks: 5 at 2,034, 8 at 2,035 and 20 at 2,037. Kelp's fair is the sized mid, the first level each side with at least 15 lots, so the small asks are skipped: the fair is halfway between 2,033 and 2,037, 2,035 credits.

Take buys every ask at least one credit below the fair: the 5 lots at 2,034. Clear then works the position to its target of 0. The bot is still short 8 if the take fills, and the 8 lots at 2,035 cost no more than the fair, so it buys them. Make quotes what is left: half a tick each side of the fair, rounded outward, gives a bid at 2,034 and an ask at 2,036, and `make_inside` leaves them there, since each is already a tick inside the remaining book. The bid is for 50 lots, because the room to buy counts the 13 already planned against a position of −13; the ask is for 37.

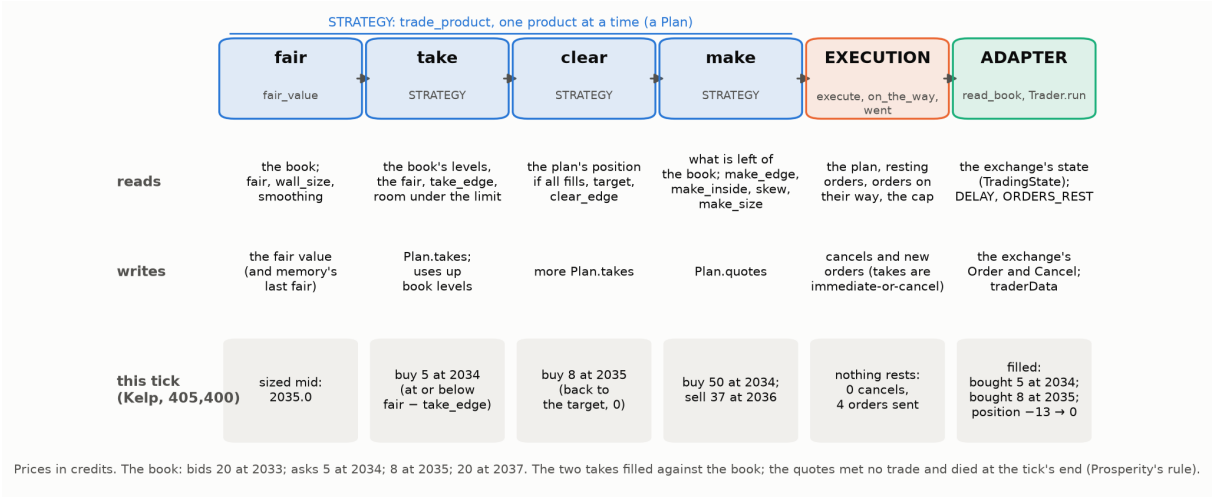


Figure 32: One tick of Kelp through the template: what each stage reads, what it writes, and what it did at timestamp 405,400 of Prosperity 3's round 1 day 0. Data: `data/one_tick.csv`, from the engine's run of that day and the template's own functions.

EXECUTION has nothing resting to keep or cancel, since on Prosperity every order dies at the end of its tick, so it sends the four orders: the two takes as immediate-or-cancel, then the two quotes. The engine's limit check passes (all the buys together would bring the position to exactly +50), the takes fill against the book, the quotes meet no trade and die, and the position ends the tick flat. Kelp's PnL for the day moves from 2,352 to 2,357 credits.

Nothing in the four strategy stages knew it was on Prosperity. Under the Kestrel's rules the same lines would have run, and the two quotes would have been on their way when the next call came.

The tools, the twins and the notes

The tools sit on top (`python/arena/`): `arena.backtest` for one run, `arena.grid` and `arena.compare` for many with chapter 11's statistics, `arena.stress` for perturbed days, `arena.diagnose` for the data with no bot, `arena.validate` before every upload, and `arena.sim`, a separate market that reacts. Thirty experiments to run on the day's data, calibrate among them, sit beside them (`python/experiments/day/`).

The Rust folder holds a second copy of the engine, the agent market, the heavy statistics and our bots, ported function for function. It gives the same results to the bit, about 130 times faster for one backtest and 40 to 80 times for a grid, behind `--engine rust` (`rust/README.md`). Two copies drift unless something stops them: a table of twins holds a hash of the Python each port was last checked against, and a stale twin is refused until the Rust benchmark's parity run has compared every recorded field again (`rust/twins.json`; `experiments.rust_bench --parity-only --record`). The uploaded bot is always the Python file.

What the prep learnt is in three places: the ideas file, 113 tested ideas and 11 waiting, each with its command, numbers and verdict (chapter 15); the wiki, whose 23 concept pages join the literature, other teams' write-ups and our measurements; and the day's documents, the runbook, the on-the-day notes and the kickoff prompt (`IDEAS.md`; `references/wiki/`; `RUNBOOK.md`, `ON_THE_DAY.md`, `KICKOFF_PROMPT.md`).

If you know ML

The split is the one between a model, its serving code and a data loader. The engine is the evaluation harness, the plug-ins are dataset loaders behind one interface, STRATEGY is the model, EXECUTION the serving logic and the ADAPTER the input and output layer for one deployment target; `PARAMS` over `DEFAULTS` is a config file over its defaults. Where the analogy breaks: serving code rarely changes a model's measured accuracy, while here EXECUTION decides which orders exist at all, so it is part of the strategy's measured result. And as with serving bugs, a replay without the delay can't show it: the Kestrel bug only appeared under the rule it depended on.

In the repo

- `ARCHITECTURE.md`, its paragraph "Formats are plug-ins" before writing a plug-in; `python/arena/formats/__init__.py` for the interface.
- The engine: `python/arena/engine.py`, its module docstring for every rule, `run_market` for the loop.
- The template: `STRATEGY`, `EXECUTION` and `ADAPTER` are banner comments in `python/bots/template.py`; `trade_product`, `execute`, `on_the_way`, `went`, `Trader.run`.
- `uv run python -m arena.validate BOT DAYS --late 1` replays the first day a step late (the default while the run's delay is 0) and fails a bot that re-posts into the caps or its limits.
- The rows: "The kickoff prompts on a disguised round", "Orders that arrive a tick late", "Order caps".

On the Arena

Written after the event, from the round handovers: what the Arena's format was, how its plug-in and ADAPTER were written and how long they took, which of `RESTING`, `ENGINE` and `DELAY` its rules fixed, and any bug that, like the Kestrel's, lived between the strategy and the exchange.

Go deeper

- `ARCHITECTURE.md` in full, then `python/arena/engine.py`'s module docstring: the matching rules in each mode.
- The Rust folder's README (`rust/README.md`): what is in Python, in Rust and in both, and how a twin is kept in step.
- `references/wiki/concepts/many-products-architecture.md`: how the teams that kept control of Prosperity 4's 50-product round laid out their bots: one object per product, strategies that propose targets, and safety nets above them.

What does each parameter do?

You'll need the fair value (chapter 3), take, clear and make (chapter 4), the skew (chapter 6), trends and the stress test (chapter 7), the target modes (chapter 8), held-out days and the grid's `pick` (chapter 11) and the template's `PARAMS` and `DEFAULTS` (chapter 14).

The story

On 15 October the template went through the stress harness, which replays each day with its prices moved, by a trend, a shift, a spike, doubled volatility or the walls removed, and reports what the bot makes on each copy. On Prosperity 3's round 1 it found that a trend over the day took the template from 81.3 thousand credits to 19.6 thousand going up and 14.3 thousand going down. Rainforest Resin's fixed fair of 10,000 lost 10.4 thousand each way: it kept trading against a price that had left (`IDEAS.md`, "Stress: what a regime change costs").

Two parameters came out of that run. `fixed_guard` gives up a fixed fair while the market is further than the guard from it, and quotes around the sized mid instead. `stop_loss` flattens a product and stops it for the day once its PnL has fallen that far below the day's high, set at about three times its worst drawdown on ordinary days, so that it never fires on them (`IDEAS.md`, "A fixed fair that gives way", "A loss stop per product").

Re-run on the template as it is now, the guard is worth what it was then. With it, the trends cost little: 66,554 and 62,214 credits against 72,674 on the days as recorded. Without it, Resin again loses 10,413 and 10,309, and the trend days make 10,444 and 6,228. But neither parameter is free. With the guard on, a price that shifts up and stays leaves Resin 32.0 thousand with its stop against 49.1 thousand without it: the stop fires on the move and the bot sits out a day that would have paid. Chapter 7 tells why the stop can't help without the guard and costs with it (`data/stress_variants.csv`, from `arena.stress python/bots/template.py 1` with `--set`).

So a parameter is a bet on what the market will do, with a price on some other market. That is why each one has a row in `IDEAS.md` saying what it was tested against, and why this chapter reads the rows before the parameters.

The idea

How to read an IDEAS row

The ideas file, `IDEAS.md`, is a table with four columns. **Idea** names it in bold, with where it came from. **Result on Prosperity data** holds each test, dated, with the command that reproduces it and its numbers; a later test is added under the earlier ones, never in their place. **Verdict here** is one of four words (the index also carries a fifth, `mixed`, that the file's header doesn't define). **Re-test on the Arena** says when and how to try it again on the real data.

The file defines the four verdicts in its own words:

- **kept**: it is in the template;
- **dropped here**: it is not in the template, and is re-tested on the day anyway;
- **measured**: a fact about the data, not a strategy;
- **open**: tested, with no conclusion yet.

In practice "in the template" means in its settings for the products it was tested on, or available and off by default; a dropped setting often stays in `DEFAULTS`, off, so it can be re-tested. "Dropped here" is the one to read carefully. It is a result on one market, under its fill models, with its takers; the file's own words are that a result there "is a prior about the Arena, not a verdict". A setting dropped on Prosperity may be the right one on a market whose prices trend or whose takers are informed, and the re-test column says which sign to look for. A verdict cell can name more than one word: a mode kept and one of its options dropped, or a setting dropped for one product and kept for another. The index at the top of the file, and Figure 33, take the first; the tables below name the others.

A kept row, read in full: "A fixed fair that gives way". The idea is `fixed_guard`. The result has three dated entries; the story above quoted the third. On 15 October, Resin with a guard of 5: the plain days unchanged at 53,976 credits, the trend days 47.6 and 47.1 thousand where they had been −10.4 and −10.3 thousand, the worst day +6.9 thousand instead of −3.5 thousand; but doubled volatility and a spike up cost more than before. Later the same day, a second test found a flaw: a rival's large order 10 ticks behind the best moved the wall mid past the guard, and Resin made 15.3 thousand against 39.6 thousand without it, so the guard now reads the sized mid, which a far order can't move. The verdict: kept, for Resin and Prosperity 2's Amethysts. The re-test: for any fixed-fair product on the day. One row holds the idea, its gain, its cost, the bug a second test found, and when to use it.

A dropped row, read in full: "`skew` against Kelp's long position". Kelp sits at its limit of +50 about a tenth of the time, and the textbook answer is to skew the quotes against the position (chapter 6). The result says three things. On each day mirrored, prices reflected and buyers swapped for sellers, Kelp's mean position is exactly the negative of the real day's, so the long comes from the takers' flow (34 to 36% of Kelp's trades at the touch are buys), not from anything the bot does. The flow isn't informed: the position's correlation with the next 10 to 1,000 ticks' price change is −0.10 to +0.18. And skew costs at every step: 18,698 credits at 0, then 18,210, 17,350 and 15,893 for 1, 2 and 3 ticks under the optimistic fill rule; 14,530 down to 9,514 under the strict one (`data/skew_grid.csv`). The verdict: dropped here, default 0, for Kelp; Pepper Root's skew of 20 is kept, with its long target (the next section). The re-test: sweep `skew` per product, and skew only where the same test of the position against later prices is clearly negative. It doesn't say skew is wrong; it says that on this flow holding the position cost nothing measurable.

The settings, group by group

The template's 71 settings fall into five groups, plus the ADAPTER's globals. Each table gives the setting's default (`python/bots/template.py:45`), what it does, and the verdict of the row that tested it, named in brackets, with any sub-option the row drops. Two settings are left out: `reduce_only`, which flattening sets for itself, and `day_length`, the timestamps in a day. Where a committed product overrides a default, the table gives an example, not every override.

Quoting: take, clear and make.

Setting	Default	What it does	Verdict
`take_edge`	1	take any book order at least this much better than the fair	kept (“The template’s market maker”)
`clear_edge`	0; for example 5 on Picnic Basket 1	trade back towards the target at prices up to this much worse than the fair	kept (same row)
`make_edge`	2; for example Kelp 0.5, Osmium 1	quote at least this far from the fair	kept (“Kelp `make_edge` and `take_edge`”, whose own best is 1; Kelp’s 0.5 was chosen later for strict fills, `docs/TODO_DONE.md`, “Tune Kelp and Squid Ink for `worse` fills?”)
`make_inside`	1	quote no nearer the fair than this many ticks inside the best book quote; none: at `make_edge` whatever the book	kept as an option (“Quoting nearer fair than one tick inside the book, against rivals that do”)
`take_edge_buy`, `_sell`, `make_edge_buy`, `_sell`	none	each side its own edge	dropped here (“Per-side edges”)
`make_levels`, `make_step`	1, 1	split each side’s quote over several prices	open (“Laddered quotes”)
`take_reserve`	0	keep this much of the limit free for takes	dropped here (“A take reserve”)
`join_min`, `join_ratio`	none, 2	a quote that would join the best stands a tick back unless the queue there is deep and the other side’s isn’t much deeper	dropped here (“A large-tick book”)

Fair value.

Setting	Default	What it does	Verdict
`fair`	`sized_mid`	the method: `fixed`, `mid`, `wall_mid`, `outer_mid`, `sized_mid`, `weighted_mid` or `hysteresis_mid`	the wall, outer and sized mids kept ("Wall-mid definitions"); `weighted_mid` dropped here ("The weighted mid as fair value"); `hysteresis_mid` dropped here ("A large-tick book")
`wall_size`	15	`sized_mid`'s smallest level that counts as the market maker's quote	kept ("Wall-mid definitions": 10 to 20 within 3%)
`smoothing`	0; Osmium 0.9	an exponential average of the fair, slower nearer 1	kept for Osmium ("A slow fair on Osmium")
`imbalance`	0	add this times the best levels' size imbalance	dropped here ("Book imbalance and the micro-price")
`lin_rho3`, `lin_momentum`, `lin_window`	0, 0, 100	lean the fair on three levels' imbalance and on the last move	open: the settings that paid on ABIDES, a cost on Prosperity ("A `linear` fair", "A linear fair and a gate on the last move, in an agent market")

Inventory.

Setting	Default	What it does	Verdict
`skew`	0; Pepper Root 20	move both quotes against the position, by this much at the full limit	dropped here for Kelp ("`skew` against Kelp's long position"); Pepper Root's 20 kept with its target ("A long bias on a drifting product")
`size_skew`	0	shrink the quote that would add to the position	dropped here ("Inventory control through quote size")
`target`	0; Pepper Root 80	the position to hold; clear and skew work towards it	kept ("A long bias on a drifting product")
`make_size`	all the room; for example Pepper Root 10, the baskets and vouchers 0	the largest quote	kept ("Using less than the full position limit")

Setting	Default	What it does	Verdict
`cross_max`, `cross_excess`	none; Hydrogel's `cross_excess` 25	cross at most this much a tick, and clear only the part of the gap beyond it	kept ("Passive-first execution toward a target")

The skew works on the plan's position if every order fills, its net, through one line of `make` (`template.py:851`):

$$\text{shift} = \text{round}\left(\text{skew} \times \frac{\text{net} - \text{target}}{\text{limit}}\right)$$

Target modes. Each sets the target in place of the static one; the first that is set wins (chapter 14).

Setting	Default	What it does	Verdict
`level`, `level_band`, `level_exit`, `level_weight`	none, 20, none, none	hold the full limit against a move of `level_band` from a price the product returns to	kept ("Trading toward a known level"), `level_weight` dropped here; a level from the day itself open ("The day's own level where the level drifts across days")
`cpd_hazard` and five more `cpd_` settings	none	detect a shift of that level as it happens	dropped here ("A changepoint detector on the level")
`basket`, `hedge`, `hedge_ratio`	none, off, 0	trade a basket's spread against its legs	the mode kept, taking only ("Trading a basket's spread"); hedging with the legs dropped here
`smile`, `underlying`, `expiry`, `smile_window`	none, none, none, 20	price a call off a fitted volatility smile and trade its fast deviations	the mode kept ("Options: trading implied-volatility deviations from a fitted smile, delta hedging"); the smile's shape and the delta hedge dropped here
`follow`, `strike`, `max_cost_per_delta`	none, none, 20	a call that holds the underlying's bet as delta	measured; the mode kept, its cost gate `max_cost_per_delta` dropped here ("What the P4 vouchers are")
`jump_min` and four more `jump_` settings	none	fade jumps that snap back	kept ("Fading jumps that snap back")

Setting	Default	What it does	Verdict
`copy`, `copy_hold`, `copy_edge`	none, none, 5	follow named traders	kept (“Following informed counterparties”)
`copy_sizes`, `copy_warmup`	none, 100	follow a nameless informed trader by its trade size	measured (“A nameless informed-trade detector”)
`gate_move`, `gate_window`	none, 20	pull the side a recent move runs over	open (“A linear fair and a gate on the last move, in an agent market”)

Protection and the close.

Setting	Default	What it does	Verdict
`stop_loss`, `stop_edge`	none; for example Kelp 1,200, Resin 1,000, Hydrogel 48,000	flatten and stop for the day once the day’s PnL is this far below its high	kept (“A loss stop per product”)
`fixed_fair`, `fixed_guard`	none; Resin 10,000 and 5	a fixed fair, given up while the sized mid is further than the guard from it	kept (“A fixed fair that gives way”)
`level_guard`	none	drop a level bet once the price is many bands away	dropped here (“Dropping a level bet once the price is far from the level”)
`vol_guard`	none	plain market making while volatility is high	dropped here (“A volatility guard”)
`flatten_from`, `flatten_edge`	none	from this timestamp, work the position to zero, quoting only the side that reduces it	kept, off (“Flattening before the close”)
`flatten_keep`, `flatten_widen`	none, 0	keep a small position at the close instead	dropped here (“Keeping a small position at the close”)
`limit`	50, a placeholder	the position limit, the exchange’s, required in every entry	a rule, not a choice

The ADAPTER’s globals. `DELAY` (0) and `ORDERS_REST` (off) switch on EXECUTION’s counting of orders on their way; the row “Orders that arrive a tick late” is open, since the Arena’s timing is unknown. `MAX_RESTING` (none) is the exchange’s cap on working orders; the row “Order caps” measured that the template needs about three orders a product a tick.

Figure 33 puts the groups side by side. Of the 36 entries, 15 are kept, 12 dropped here, 6 open and 3 measured (`data/parameters.csv`).

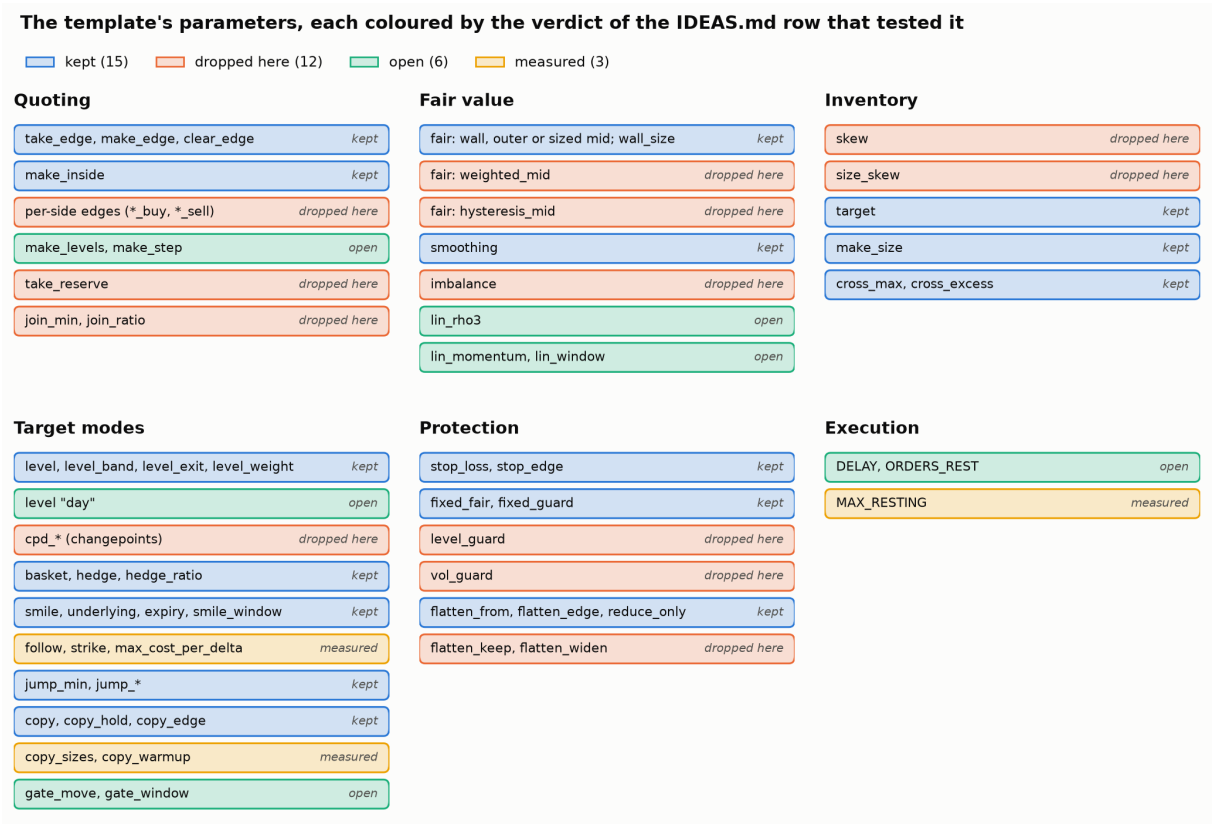


Figure 33: The template's settings by group, each coloured by the verdict of the `IDEAS.md` row that tested it, parsed from `IDEAS.md` as its index is, so a rebuild shows a changed verdict. Data: `data/parameters.csv`, from the `parameters` data step.

The pattern is the one the stress story ends on. What is kept is mostly the plain market maker and the modes that each answer one kind of product; what is dropped is mostly the refinement the literature recommends for a market unlike Prosperity's, inventory skew against uninformed flow, the micro-price, guards against regimes that never came. Those are the rows the Arena may turn over.

Changing one on the day

The runbook gives the order. New products go into `PARAMS` by the wiki's day-one page (T+0:30), after the diagnose tool's suggest option has proposed an entry for each (T+0:15); a setting changes only to the grid's `pick` (T+1:00); and each `stop_loss` is set to about three times the plain drawdown the stress tool prints (T+1:30) (`RUNBOOK.md`, "Each round"). The rows themselves are re-tested every round: down the index, every row whose "Re-test on the Arena" condition fits the round's products (`IDEAS.md`, "Keeping it up to date"; `ON_THE_DAY.md`, "Every round", "Re-test the ideas register").

> **If you know ML**
>
> `DEFAULTS` is a config's defaults, `PARAMS` the per-dataset overrides, and an `IDEAS.md` row an ablation: one switch, the baseline beside it, the command to rerun it. "Dropped here" is an ablation that lost on one validation set, which in ML you'd keep in the search space for the next dataset, not delete; that is the file's rule too. Where the analogy breaks: these settings interact through the fill model, which is itself unknown (chapter 10), so an ablation's sign can flip between two simulators of the same data, and the next dataset, the Arena, is drawn from a different distribution by design.

> **In the repo**
>
> - Every setting and its comment: `python/bots/template.py`, `DEFAULTS` (line 41) and `PARAMS` (line 161).
> - A product's settings that can't matter: `arena.validate` prints them (`param_conflicts`).
> - The rows by verdict: `IDEAS.md`'s index, its first 150 lines; rebuilt by `uv run python -m experiments.ideas_index`.
> - This chapter's figure: `uv run python docs/guidebook/book/scripts/make_data.py --only parameters` parses the verdicts again.
> - The stress test with a setting switched off: `uv run python -m arena.stress BOT DAYS --set PARAMS.<P>.fixed_guard=None`.

> **On the Arena**
>
> *Written after the event, from the round handovers:* which settings each round's products took, which "dropped here" rows the Arena's data turned into kept ones and which kept ones it dropped, and the verdicts as they stood after the last round.

Go deeper

- The wiki's concept pages behind the groups: `references/wiki/concepts/fair-value-and-wall-mid.md`, `references/wiki/concepts/market-making-and-inventory-skew.md`, `references/wiki/concepts/optimal-quoting-models.md` and `references/wiki/concepts/drift-and-regime-change.md`, each with the literature it draws on.
- The ideas file's paragraph "What a market we didn't write said about these priors", just after its index: which verdicts held on ABIDES and which didn't.

What did we do on the day?

You'll need the whole book; this chapter only explains why the day was planned as RUNBOOK says. It was written on 4 November 2026, before the event, from what the organisers had said by then, and it gives no instruction of its own: every instruction is RUNBOOK's, and where the two differ, RUNBOOK is right. What happened in each round is written after the event, in the boxes marked "On the Arena".

The story

Qminers' own 2024 hackathon was a sports-betting contest. Its online stage ran on hidden data with a limit of 20 submissions, and one entrant scored 77,382 points, as he puts it a profit equal to the rest of the field's combined. The final was in person, over a single Saturday: new data, scores reset to the 1,000 points everyone started with, and a shorter horizon, "thus introducing more variance to the competition" (`references/qminers-hackathon-2024/README.md:33-43`).

Before it, he wrote a small simulation of the final, of twenty finalists by the README (sixteen teams in the committed script), all starting level, with models of varying quality, correlated with each other, each with its own appetite for risk. He used it to choose how much of his bankroll to bet, choosing the fraction that maximised his chance of winning the tournament, not his expected score (`README.md:45-49`). (The code committed for it doesn't run as it stands; the idea carries over, not the number (`references/wiki/sources/qminers-hackathon-2024.md`, "Traps").)

The final had three rounds. He took the lead in the first and extended it in the second. Before the third he worked out that its scrambled data came from just after covid, a period other finalists' models might handle better than his, and halved his stake so as not to lose the lead. It was, he wrote, "a minor mistake": his model "performed the best during this time". He finished on 107,074 points, against 69,460 for the second, and estimates he would have ended near 143,000 at full stake (`README.md:55-63`; `final_scores.png`).

The point is not that halving was right. It is that the question changed once he was ahead: from "what makes the most" to "what keeps the place".

The idea

How the plan was made

The plan was built in steps, each from something learned, and the dates say where each part came from (`ON_THE_DAY.md`, "Known so far"; `IDEAS.md`, "The kickoff prompts on a disguised round").

- **7 October.** The organisers' e-mail gave the schedule: a playground in the morning, then four rounds of five hours through the night.

- **10 October.** The Arena's site, read from its own code, showed the shape of the game: one Python file uploaded to a web portal each round, a capped number of submissions, and public backtest data to download.
- **24 and 25 October.** We would be two, so each of us holds two of four roles: A drives and records, B runs the data and the research. The portal gained its twelve figures for each upload, one evaluation at a time, and a new upload becomes final only once it completes.
- **26 October.** The leaderboard ranks by PnL, not by a risk-adjusted score. The first disguised rehearsal ran the same day, and three more followed on 28 October, 30 October and 3 November (chapter 1). Each changed the plan: the third taught the template to count its orders still on their way and, once its engine fit turned out to be circular, made `calibrate` refuse any fit worse than 5%; the fourth tested that refusal on a portal that wasn't our engine.
- **2 November.** The last e-mail: no elimination, every team plays all four rounds; any AI tools allowed; nobody may sleep in the building, but the whole team may leave to rest.
- **4 November.** This chapter, and the field guide's version of it.

The weekend

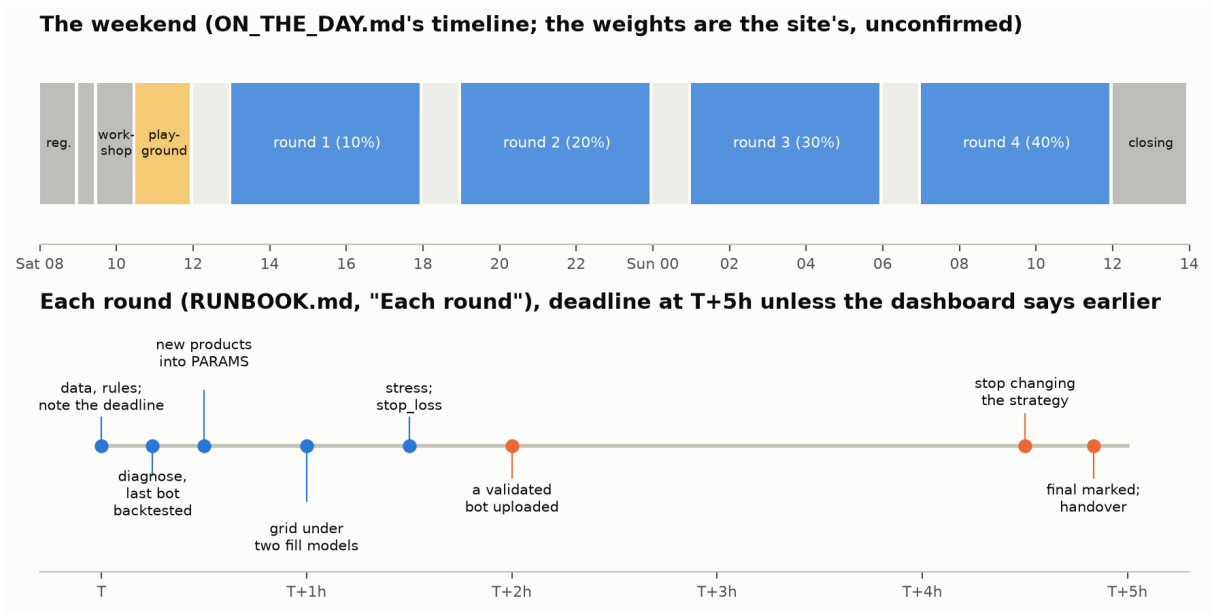


Figure 34: The weekend and one round, as planned. The weekend from `ON_THE_DAY.md`'s timeline, the round from `RUNBOOK.md`'s "Each round"; both drawn from those files by `make_data.py`.

Saturday starts with registration at 08:00, as a whole team. The Technical Rules are published at the opening, and the *playground*, 10:30 to 12:00, is planned as the one chance to try the real exchange before anything is scored. Then four rounds, at 13:00, 19:00, 01:00 and 07:00 (Figure 34). The site's code weights them 10, 20, 30 and 40%, but no page uses those weights, and the leaderboard it does show is per round, with no total: how the rounds combine is unknown until the rules say (`ON_THE_DAY.md`, "Known so far" and "The day: timeline and roles").

The morning: settle what changes the code

RUNBOOK's morning is about the answers that change code, before any strategy. One person reads the Technical Rules in full and writes each answer into the "First hour" section of

ON_THE_DAY: the entry point, the data format, whether orders rest, the limits, the Python version, the time limit, the submission cap. Any module the organisers hand out goes into the plug-in unchanged. In the playground three tracks run at once: a probe bot that measures the exchange's rules, the data loader, and the bot's adapter. Over lunch, the first upload's figures go into calibrate.

The reason is the rehearsals. In the second and third, most of the time went on mapping the rules onto the engine and writing the adapter, and the third's first upload lost its money to the adapter, not to the strategy (chapter 1). Chapter 2 is the list of what can differ: whether orders rest, how the limit is enforced, what the data looks like.

Why each round runs as it does

RUNBOOK's "Each round" is chapter 1's loop with times on it. Each step has its reason in this book:

- **The data and the rules first, then `diagnose` and the premise test** (T+0:00 to T+0:15): before tuning anything, find out what kind of market each product is (chapters 3, 5 and 7). Where a product behaves like an agent market rather than like Prosperity, a bot that does nothing is the benchmark (`KICKOFF_PROMPT.md`, round 1, step 5).
- **The grid under two fill models** (T+1:00): until the playground or the rules settle which fill model the Arena has, a choice must hold under both (chapters 10 and 11). A setting changes only to the grid's `pick`, with the one exception RUNBOOK names for round 1 if the total turns out to be raw PnL.
- **Stress, and a loss stop sized from it** (T+1:30): a loss stop about three times the bot's plain drawdown limits the damage when the market stops behaving like its data (chapters 7 and 13).
- **A validated bot, at least as good as the last one, uploaded by T+2:00:** its portal score against our backtest is the engine's error, and no two variants closer than that error can be told apart (chapter 12).
- **Stop changing the strategy half an hour before the deadline,** or earlier if twice the first evaluation's time is longer: the last change has the least evidence behind it, and an upload that fails can't be marked final.
- **The handover note ten minutes before the deadline:** what was learned, for the next round's session, which starts fresh. Chapter 1's rehearsal wrote one; the Arena's four are what this chapter's boxes will be written from.

The rules that hold all day

Above every step sit RUNBOOK's "Rules that hold all day". Never upload code `arena.validate` hasn't passed, and never a `GRAY_AREA` bot without an organiser's yes (chapter 9). A round's first upload is a bot that has already run in the playground, or last round's final bot, because the validator checks only against our copy of their interface. Never choose between two variants by a portal score that differs by less than the engine's error. One evaluation at a time, and an upload that completes becomes final, so mark the chosen one final again after any exploratory upload. Naps only in rounds 2 and 3, and only when the round looks like the one before. Each rule is a lesson from somewhere in this book, written down so that nobody has to remember it at four in the morning.

Each round, choose again

The site promises "a forgiving market" first and "a sophisticated finale". So the round-1 bot is a starting point for round 2, not a default: RUNBOOK has each round re-run the premise test, re-grid every setting on the round's own data, and try one alternative to the last round's approach by T+1:00. Every number in this book before the event was measured on someone else's market: Prosperity's, our simulator's, ABIDES's or a rehearsal's. Each is a prior, to be tested again on the Arena's data, round by round.

Risk, once there is a score to protect

Two things decide how much risk to carry into the later rounds, and neither was known on 4 November: how the four rounds combine into the final ranking, and whether profit carries over. The plan has the decision made in writing at breakfast, before round 4, from `experiments.day.standings`: our rank under each way the rounds might combine, given the standings and our round-4 grid (`RUNBOOK.md`, "Rules that hold all day"). If rounds 2 and 3's private runs strayed from our backtests, a bot that sends nothing is on the list. The 2024 story is the frame: once you are ahead, the variance you could avoid counts for more than it did, and when you are behind, for less (chapter 13).

The rounds as they went

⚠ On the Arena: round 1 (13:00–18:00)

Written after the event, from `data/arena/round1/HANDOVER.md` *and* `leaderboard.json`*:* the round's products and rules, the first upload and its portal score against our backtest, the fill model and mark the portal matched, what the private run did to the public score, our rank, and the first candidate it left for round 2.

⚠ On the Arena: round 2 (19:00–24:00)

Written after the event, from `data/arena/round2/HANDOVER.md` *and* `leaderboard.json`*:* what changed from round 1, the alternative tried against round 1's approach and whether it won, the uploads, the private run, our rank, and whether anyone slept.

⚠ On the Arena: round 3 (01:00–06:00)

Written after the event, from `data/arena/round3/HANDOVER.md` *and* `leaderboard.json`*:* the same, for the night round, and the standings that went into the breakfast decision on risk.

⚠ On the Arena: round 4 (07:00–12:00)

Written after the event, from `data/arena/round4/HANDOVER.md` *and* `leaderboard.json`*:* the risk decided at breakfast and why, the final bot, the private run, and the final ranking.

If you know ML

A competition ranked on one final evaluation rewards the probability of being first, not the expected score, the way a leaderboard with a private test set rewards robustness over the public score. Where the analogy breaks: on a leaderboard you choose the model once, while here the stake can change from round to round, after you have seen the standings, so the choice of risk is itself a decision made on new information. And the rounds may not be the same task: the organisers promise harder ones, so a model fitted to round 1 is a prior for round 2, not a solution.

In the repo

- `RUNBOOK.md` (one page), `ON_THE_DAY.md` (every open question about the Arena, and the timeline) and `KICKOFF_PROMPT.md` (the prompts that start each round's session).
- `docs/HANDOVER_TEMPLATE.md`, copied to `data/arena/round<N>/HANDOVER.md` each round; the round's leaderboard in `leaderboard.json` beside it.
- `experiments.day.standings` and `experiments.day.leaderboard_curves`: what the standings say about risk, once there are standings.

On the Arena

Written after the event, from the round handovers: the weekend as it went against Figure 34 as planned (the timeline redrawn with both), which of RUNBOOK's times held, and what the plan got wrong.

Go deeper

- The wiki's pages on risk and protecting a lead, and on harder rounds: how other teams sized risk with a lead, and how competitions make later rounds harder (`references/wiki/concepts/risk-and-protecting-a-lead.md`, `harder-rounds.md`).
- The wiki's page on Qminers' 2024 hackathon: its final in full, with what the screenshots confirm (`references/wiki/sources/qminers-hackathon-2024.md`).

Appendix A: Glossary

Each term with the chapter that defines it, and its nearest neighbour in machine learning where there is an honest one.

Adapter The part of the bot that knows one exchange: it reads the exchange's state and writes its orders. Chapter 14. *ML: the input and output layer for one deployment target.*

Adverse selection The tendency of a waiting order to be filled just before the price moves against it, because the traders keenest to take it know something. Chapter 5. *ML: selection bias, chosen by an adversary.*

Ask, bid An offer to sell, an offer to buy. The best ask and best bid together are the *touch*. Chapter 2.

Backtest A replay of recorded days with the bot trading in them. Chapter 10. *ML: offline evaluation in a simulator.*

Basket A product whose value is a fixed weighted sum of other products, its legs; its spread is its price less that sum. Chapter 8. *ML: a constraint between outputs.*

Black–Scholes The formula that prices an option from the underlying's price, the strike, the time left and the volatility. Chapter 8.

Bot as is The bot with its current settings, the baseline a grid's cells are tested against. Chapter 11.

Calibrate The tool that searches the engine's settings for one that reproduces the portal's figures, or refuses. Chapter 12. *ML: system identification.*

Clear Trading back towards the target position with orders up to `clear_edge` worse than the fair value; the target is flat unless a strategy sets one. Chapters 4 and 14.

Cross To trade at once against an order already in the book, buying at the ask or selling at the bid. Chapter 2.

CSCV Combinatorially symmetric cross-validation: every split of the data's blocks into two halves, choosing on one and ranking on the other; it estimates PBO. Chapter 11. *ML: cross-validation of the selection itself.*

Delay Orders reaching the exchange some steps after the bot sends them; the engine's delay option. Chapter 10.

Delta How much an option's price moves when the underlying moves by one; the units of underlying that hedge it. Chapter 8. *ML: a partial derivative.*

Depletion Whether volume the bot takes from the recorded book stays gone in later ticks; the engine's deplete option. Chapters 10 and 12.

DSR The deflated Sharpe ratio: the chance that the best cell's Sharpe ratio beats the best expected from that many cells with no edge. Chapter 11. *ML: a correction for the number of trials.*

Engine The code that replays recorded days against a bot and decides its fills, in the repo's `arena` package. Chapters 10 and 14.

Fair value The bot's estimate of what a product is worth now; every quote is placed relative to it. Chapter 3. *ML: an estimate of a latent variable.*

Fill A trade that executes one of your orders, wholly or partly. Chapter 2.

Fill model The engine's assumption about which recorded trades would have filled orders that weren't really there: `all`, `worse`, `none`, or resting with a queue. Chapter 10. *ML: the simulator's sim-to-real gap, one-sided.*

Format plug-in The module that reads one exchange's data into Days of Ticks and wraps its bot interface. Chapters 2 and 14. *ML: a dataset loader behind a common*

interface.

GRAY_AREA A flag in a bot file marking a strategy that may break the spirit of the rules; `arena.validate` fails such a file, so it is never uploaded without an organiser's yes (RUNBOOK, "Rules that hold all day"). Chapter 9.

Grid Every combination of a few settings, each backtested and ranked. Chapter 11. *ML: grid search over hyperparameters.*

Half-life How long a price pulled back towards a level takes to close half its gap to it. Chapter 7.

Held-out day A day left out when a setting is chosen, then used to score it; `--holdout` keeps one out of the whole choice. Chapter 11. *ML: a validation fold, and a test set used once.*

Impact How far the price moves, on average, in the direction of a trade after it; $\mathcal{R}_\infty$ in the long run. Chapter 4.

Implied volatility The volatility at which the Black–Scholes formula gives an option's market price; plotted against the strike, the smile. Chapter 8. *ML: a parameter fitted by inverting a model.*

Informed trader A trader whose trades predict the price's next move, such as Prosperity 3's Olivia; trading with one is adverse selection. Chapter 5.

Inventory risk The price risk of holding a position between a buy and the sale that closes it. Chapter 6.

Kelly fraction The share of a bankroll to bet that maximises its long-run growth: the edge over the odds. Chapter 13.

Level All the orders at one price on one side of the book. Chapter 2.

Lift over the median cell How much more the grid's best cell makes than its median cell; the permutation test compares it with the shuffled copies'. Chapter 11.

Limit order An order to buy at a price or lower, or sell at a price or higher. Chapter 2.

Lot One unit of a product, the size in which orders and positions are counted. Chapter 2.

make_edge, take_edge How far from the fair value the bot quotes, and how far the book must be mispriced before it takes. Chapters 4 and 11.

Making, taking Making is posting an order that waits; taking is crossing the book to trade at once and paying the spread. Chapter 2.

Mark The price at which a position is valued when PnL is computed; the Varde portal's `markPrice` averages it over products, and the Arena's may (ON_THE_DAY infers it; calibrate tries both readings). Chapters 2, 10 and 12. Also the name Prosperity 4 gave its bots, "Mark 14", "Mark 67", when it showed who traded. Chapters 5 and 11.

Market maker A trader that quotes on both sides and earns the spread from those who want to trade now. Chapter 4.

Markout A fill's profit some ticks later, measured from the mid or the wall mid. Chapter 5. *ML: the label, observed with a delay.*

Mean reversion A price's tendency to come back towards a level or an average after it moves away. Chapter 7.

Micro-price Stoikov's estimate of the price far ahead, the mid plus an adjustment learned from the book's imbalance and spread. Most teams use the name for the simpler weighted mid, which leans the mid towards the side with less size. Chapter 3.

Mid Halfway between the best bid and the best ask. Chapter 2.

Order book The orders waiting at a moment, bids on one side and asks on the other, grouped by price. Chapter 2. *ML: the state.*

PARAMS, DEFAULTS The template's settings: `DEFAULTS` holds every setting with its default, and `PARAMS` each traded product's entries over them. Chapters 14 and 15. *ML: a config file over its defaults.*

Passive order An order that doesn't cross the book and waits for someone to trade with it; a market maker's quotes. Chapters 2 and 10.

PBO The probability of backtest overfitting: how often the best setting on one half of the data ranks below the median on the other. Chapter 11. *ML: a measure of model-selection overfitting.*

Permutation test The whole search re-run on

days with their ticks in random order; if it finds as much, the choice didn't need the order of events. Also the shuffle test. Chapters 5, 10 and 11. *ML: a permutation null.*

Pick The grid's chosen setting: the best cell that StepM confirms beats the bot as is under the grid's first fill model, and that also beats it under every other model the grid ran (the default gate; once the fill mode is pinned, RUNBOOK gates on the first mode only). Chapter 11.

PnL Profit and loss: cash plus the position valued at the mark. Chapter 2. *ML: the reward, after a choice of reference price.*

pnlSe The portal's standard error of the score; on the Varde portal, the spread of the session scores times the square root of their number. Chapter 12. *ML: the error bar on a test metric.*

Position, limit What you hold, positive if long, negative if short; the most the exchange lets you hold either way. Chapter 2.

Premise test The repo's check of whether a product is Prosperity-like (uninformed takers), agent-like (informed traders, moves that continue) or mixed. Chapter 5.

Print, aggressor A trade between two other participants, and the side of it that crossed the book. Chapter 2.

Queue The order in which waiting orders at one price are filled: earliest first. The engine's queue models, `back`, `prob` and `front`, guess our place in it. Chapters 2 and 10.

Rehearsal A disguised mock round run before the event, with its own exchange, data and portal; Varde was the fourth. Chapter 1.

Relative hedge Holding some of the bet the teams behind are making, so that if it pays they can't pass you. Chapter 13.

Reservation price The price a market maker quotes around, in Avellaneda and Stoikov's model: the mid less its position times its risk aversion, the variance and the time left. Chapter 6.

Resting An order that stays in the book between ticks until filled or cancelled. Chapter 2.

Round trip A buy and the sale that closes it, or the reverse; what a market maker earns on is the gap between them. Chapter 4.

Settlement price The price at which a portal values what is still held at the end, the Varde Exchange's name for its mark. Chapter 2.

Sharpe ratio A gain's mean over its standard deviation. Chapter 11. *ML: a signal-to-noise ratio.*

Signature plot The variance of price changes over h ticks, per tick, plotted against h: flat for a random walk, falling for a price that reverts, rising for one that trends. Chapter 3.

Skew Moving both quotes against the position held, so that the side that reduces it fills more often. Chapter 6.

SPA Hansen's test of superior predictive ability: does any cell beat the bot as is, allowing for every cell tried? Chapter 11. *ML: a multiple-comparison test against a baseline.*

Spoofing Placing orders one means to cancel, to move others' view of the price; banned on real exchanges. Chapter 9.

Spread The gap between the best ask and the best bid. Chapter 2.

StepM Romano and Wolf's step-down test: which cells beat the bot as is, with the chance of naming even one false cell held at a set level; read at 4%. Chapter 11. *ML: family-wise error control.*

Stop loss The template's `stop_loss`: it flattens a product and stops it for the day once its PnL falls a set distance below its high. Chapters 6 and 13.

Stress test `arena.stress`: the bot re-run on copies of each day with the prices changed (more volatile, trending, shifted, spiked, the walls removed), to see what a change of regime costs. Chapter 7. *ML: evaluation under distribution shift.*

STRATEGY, EXECUTION The template's two generic parts: what the bot wants (fair value, take, clear, make), and the orders and cancels that get it there. Chapter 14.

Target, target modes The position the bot works towards, flat for a plain market maker; a target mode (`basket`, `level`, `copy`, `jump`, `follow`, `hedge`) sets it from

a rule outside the book. Chapter 8. *ML: a forecaster whose output is a position.*

Template The repo's bot, `python/bots/template.py`, one file, with settings per product. Chapter 14.

Tick One step of the recorded market; also the smallest price step, the *tick size*. Chapter 2. *ML: a time step.*

Value at risk The loss a position would exceed only a given share of the time, one in twenty for the 95% value at risk. Chapter 13. *ML: a quantile of the loss distribution.*

Wall, wall mid The largest level on each side of the book; the wall mid is their midpoint. On Prosperity the walls are a market-making bot's large, stable quotes. Chapters 2 and 3.

Zoo The repo's collection of bots written by language models from fixed prompts, used as rivals and as a baseline. Chapter 9.

Appendix B: The datasets

Every market the book measures on, where its days come from, and the units its numbers are in. Each one reaches the tools through a format plug-in (chapter 14) as days of ticks, so every tool can be pointed at any of them.

Prosperity 2, 3 and 4: the stand-in

IMC Prosperity is a yearly online trading competition, each round with a few days of data, and that data is what the whole prep ran on in place of the Arena's. A day is 10,000 ticks, at timestamps 0 to 999,900 in steps of 100, for 54 of the 69 days here; the other 15, tutorial and submission data, are 2,000 or 1,000 ticks long. Each tick has three book levels a side and the trades printed since the last one; prices and PnL are in **credits**. A product's book is generated by IMC's simulator: a large quote at about the value, the "wall", that chapter 3 builds the fair value on, and takers who mostly don't know more than it.

The days come with three open-source backtesters, cloned into the references folder at pinned commits: jmerle's for Prosperity 2 and 3 and another author's for Prosperity 4. The data names p2, p3 and p4 point at their data folders (`references/clone.sh`; `python/arena/formats/prosperity.py`, `DATA_DIRS`). Between them they hold 69 days (`data/datasets.csv`, from `prosperity.list_days`; the tick counts from `prosperity.load_days`):

Data	Rounds	Days	Products, for example
`p2`	0, 1, 3, 4, 6, 7	20	Amethysts and Starfruit; a gift basket and its three legs; Coconut and its coupon
`p3`	0 to 8	32	Rainforest Resin, Kelp and Squid Ink; two picnic baskets and their legs; Volcanic Rock and five vouchers; Magnificent Macarons
`p4`	0 to 5	17	Osmium and Pepper Root; Hydrogel, Velvetfruit and ten vouchers; 50 products in round 5

Two cautions. Prosperity 2 has no round 2 here: its conversions aren't supported, by its backtester or by ours. And the higher-numbered rounds are not new markets: in Prosperity 3, round 6 holds the data the submissions ran on, round 7 the end-of-round data and round 8 an

earlier version of round 2, and Prosperity reuses days across rounds, so a day can be "unseen" in name only. `arena.grid` flags days that share their books and trades (`repeated_days`) (`references/prosperity3-backtester-jmerle/README.md:79-81`; `ARCHITECTURE.md`, "Grid search"). Some rounds name the traders behind each trade, which the copy mode of chapter 8 uses: Prosperity 2's rounds 1, 3 and 4, Prosperity 3's round 5 and day 5 of its round 6, and Prosperity 4's round 4.

Our engine reproduces the Prosperity 3 backtester fill for fill in its default mode, so a result here is the result that backtester would give (chapter 14). It is still a result about IMC's generator.

`arena.sim`: a market that reacts

A replay can't react to our orders; `arena.sim` can. It is our own agent-based market (`python/arena/sim.py`): each tick a fundamental price moves, a market maker re-quotes its walls, limit traders post near the value, the book is shown to the bots, and then noise, value and momentum takers trade against it, the bots' orders included. Every seed is a new day, and a run with a bot and the recording without it share every random number, so the gap between them is the market's reaction (`README.md`, "Simulated market").

Its presets copy products the prep knew: `KELP`, `RAINFOREST_RESIN` and `SQUID_INK`, calibrated so that `arena.diagnose` reads them as Prosperity 3's round 1, are the default market; `HYDROGEL_PACK`, `KELP_FLOW`, `KELP_NOISY`, `ROBOT_DISHES` and `LARGE_TICK` add a level, one-sided takers, a maker that doesn't know the price, jumps and a one-tick book. `generate` writes days in Prosperity's layout, prices in credits; `run` puts a bot inside the market and compares it with replaying the same days. `--regime` shifts or trends every price from mid-day.

It is a market we wrote, calibrated to the one we already had, so it tests how a bot meets a reaction, not whether Prosperity's premise holds.

ABIDES: a market we didn't write

ABIDES is JP Morgan's public agent-based market simulator, cloned with the other references (`references/abides-jpmc`). Its RMSC04 configuration is one stock, `ABM`, traded from 09:30 to 16:00 by 1,000 noise agents, 102 value agents who see a noisy view of a hidden mean-reverting value, 12 momentum agents and 2 adaptive market makers. Its book has no wall at the value, and its value agents carry information: it is the "market unlike Prosperity" the prep needed (`python/experiments/abides_days.py`).

`experiments.abides_days` runs it in its own environment and converts each day to Prosperity's layout: a tick a second, 23,400 a day, three levels a side, prices in **cents**, a limit of 100, and the trades of the second after each snapshot, without names. A file beside each day holds the hidden value, the answer key no real market gives. The book's ABIDES numbers come from 10 days, seeds 1 to 10, kept out of git, since the seeds make them again (`data/abides/`; `IDEAS.md`, "An independent generator: ABIDES's RMSC04"). It is a recording too: it tests the book and the agents' information, not a market that reacts to us.

LOBSTER: a real book, still to come

LOBSTER sells reconstructed NASDAQ order books, message by message; its sample days would give the prep a real market's informed flow beside ABIDES's simulated one. The converter is written (`experiments.lobster_days`, the same layout as ABIDES's, prices in cents), but the files are not: the samples are no longer a public download, so a person has to request them (`TODO.md`, "LOBSTER's sample, one stock at level 5"; `docs/MANUAL_TODO.md`, "LOBSTER's sample files"). Nothing in this book is measured on LOBSTER.

The mock rounds: rehearsals in disguise

Four rehearsals of the day, each made by an agent out of sight of the sessions that played it, in `python/experiments/mock_round/`. Each takes `arena.sim` products, renames and reprices them, and writes them in a layout and bot interface unlike any in the repo, with public days to work on, hidden days that score, and a portal. Each is in its own units:

Rehearsal	Date	What it tested	Units
Meridian (`mock_round/`)	26 October	the kickoff prompts, with and without the repo	credits
Halden Exchange (`round2/`)	28 October	the repo before and after its restructure	credits; prices in pips of 0.25 credits
Kestrel Bourse (`round3/`)	30 October	unstated rules: strict fills, last-trade mark, orders a step late	credits; prices in points worth 2 credits
Varde Exchange (`round4/`)	3 November	a portal that is not our engine	euros; prices in cents

All four are spent: their answer keys are committed, so a new rehearsal needs a new round (`python/experiments/mock_round/README.md`, `round2/README.md`, `round3/README.md`, `round4/README.md`, each "Spent"). Their results are in the ideas file's row "The kickoff prompts on a disguised round". The drill format has practice data of its own, three simulated days that `experiments.format_drill` writes in it.

The Arena

The Arena's own data arrives with each round, into a folder of its own. It stays out of git, except each round's handover note and saved leaderboard, which are the record appendix E is written from (`data/arena/round<N>/`, `HANDOVER.md`, `leaderboard.json`). Until the event the folder is empty, and nothing in this book is measured on it.

The zoo is bots, not data

The zoo folder of bots is easy to mistake for a dataset, and it isn't one. It holds 32 bots written by language models, Claude, Gemini, ChatGPT and local models, each given only a Prosperity-style spec and one of four prompts: a baseline, a market maker, mean reversion and momentum

(`python/bots/zoo/`, `SPEC.md`, `PROMPTS.md`). They are what a team that hands the rules to a model would start from, so they are the baseline the template must beat and a population of rivals to trade against in one market. They run on any of the data above like any other bot; they are never edited once written, and each has a Rust twin (`python/bots/zoo/README.md`).

Appendix C: The commands on one page

Every command runs from the repo's root. `BOT` is a bot file, `DAYS` a day spec (`1`, `1-0`, `1--1`; `all` outside Prosperity), `D` a data folder or `p2`, `p3`, `p4`. On the day, add `--format qminers --data D` to each command that reads the day's data, as `RUNBOOK.md` does (not `arena.compact`, the data generators, the sync, the benchmark, the index or the lint). Flags here were checked with `--help`; `--engine rust` works wherever a tool takes it (not `arena.validate`).

Set up (once per laptop; the clones are about 2 GB, the tests about 3 minutes)

```
references/clone.sh && uv sync && uv run pytest
```

Read the data (chapter 3; appendix B; only `premise` needs a bot)

```
uv run python -m arena.diagnose DAYS --data D --profile --suggest      # notes
 ↪  per product and a PARAMS entry each
uv run python -m experiments.day.premise --data D --days DAYS --bot BOT  #
 ↪  Prosperity-like or agent-like, per product
uv run python -m experiments.day.structure --data D --days DAYS        #
 ↪  repeats, fixed sums, price grids
```

Backtest one file (chapter 10)

```
uv run python -m arena.backtest BOT DAYS --data D [--match-trades
 ↪  all|worse|none] [--resting --queue back|prob|front]
    [--delay N] [--fill-ratio R] [--deplete] [--mark mid|touch|last]
     ↪  [--metrics] [--engine rust] [--out results/]
uv run python -m arena.analysis results/                                # charts,
 ↪  fill tables, markouts
```

Search the settings (chapters 11 and 15)

```
uv run python -m arena.grid BOT DAYS --data D --grid
 ↪  PARAMS.KELP.make_edge=0.5,1,2 --modes all,worse \
    [--holdout last] [--screen 0.25 | --search tpe --budget 1000] [--permute
 ↪  19] [--scenario rest:resting] --engine rust
uv run python -m arena.compare BOT DAYS --bot python/bots/zoo/claude/*.py
 ↪  --modes all,worse
```

Change a setting only to the summary's `pick` (`RUNBOOK.md`, "Each round", T+1:00).

Stress, and the market that reacts (chapters 7, 9 and 13)

```
uv run python -m arena.stress BOT DAYS --data D                          #
 ↪  perturbed days; stop_loss about 3x the plain drawdown
uv run python -m arena.sim run BOT --days 5 [--regime trend_up] [--resting]
uv run python -m arena.backtest BOT DAYS --data D --vs BOT               # a
 ↪  shared market, against a copy
```

Before an upload (chapter 14)

```
uv run python -m arena.validate BOT DAYS --data D --time-limit-ms <theirs>
 ↪  --max-size <the cap> [--late 1]   # exit 1 on FAIL
uv run python -m arena.compact BOT submit/copy.py                        # over
 ↪  the size cap: validate the copy
```

After an upload (chapter 12)

```
uv run python -m experiments.day.calibrate --data D --days DAYS --bot <the
 ↪  uploaded file> --portal metrics.json \
    --results <the results ZIP> [--vary delay=0,1]
uv run python -m experiments.day.leaderboard_curves --read
 ↪  data/arena/round<N>/leaderboard.json --team <ours>
```

Before round 4 (chapter 13; the whole command is in `RUNBOOK.md`, "Rules that hold all day")

```
uv run python -m experiments.day.standings --rounds
 ↪  data/arena/round{1,2,3}/leaderboard.json --team <ours> \
    --ours <round 4's grid CSV> --format qminers
```

Other data (appendix B)

```
uv run python -m experiments.abides_days generate --days 10              #
 ↪  ABIDES's days into data/abides
uv run python -m arena.sim generate data/sim --days 5                    #
 ↪  arena.sim's days, Prosperity's layout
uv run python -m experiments.format_drill data/drill                     # the
 ↪  drill format's practice days
```

Keeping the repo in step (chapter 14)

```
uv run python -m experiments.format_drill --sync                         # after
 ↪  changing DEFAULTS, STRATEGY or EXECUTION
uv run python -m experiments.rust_bench --parity-only --record           # after
 ↪  changing a twin's Python
uv run python -m experiments.ideas_index                                 # after
 ↪  editing IDEAS.md
python3 references/wiki/lint.py                                          # after
 ↪  editing the wiki
```

Appendix D: The reading route

What to read after this book, in order, and what each step adds to it: about 200 pages, in place of the whole of Bouchaud and colleagues' book. Every page range below was checked against the PDF, with `literature_md/pdfpage.py` or, for the scanned Kelly paper, on the page images (here, or by the wiki's source page). All of it is in `literature_md/`, and the wiki's index is the way in (`references/wiki/index.md`).

Read a step when the chapter it extends has left you wanting the reason, not before.

1. The map: the wiki's page on *Trades, Quotes and Prices*

About twenty minutes (`references/wiki/sources/trades-quotes-prices.md`). Its "Main results" and "Assumptions against the Arena" are the rest of the book in one page, each result next to what we measured on Prosperity. After this book it adds the shape of the field. The wiki names the book's chapters that matter here, 1, 11, 16, 17 and 21; the proposal for this book adds chapter 7, and the chapters you can skip, 5 and 6, 9 and 10, 12 to 14, 18 to 20, and 22 (`docs/guidebook/PROPOSAL.md`, §8). Chapter 1 of the book itself (PDF pp. 24–40) is the best single chapter for us.

2. Harris, *Trading and Exchanges*, chapters 13 and 14

"Dealers" and "Bid/Ask Spreads", to the end of §14.4 (PDF pp. 291–323). Plain prose, no equations. It adds the practitioner's view of chapters 4 to 6: why dealers earn the spread, how they control inventory, and what informed traders cost them, with §14.3 on adverse selection, which Harris calls the most important lesson in his book for most readers.

3. Bouchaud and colleagues, chapter 16's first two sections and chapter 17

§16.1–16.2 (PDF pp. 317–330) and chapter 17 (pp. 338–350). The theory core: the market maker's problem, Glosten–Milgrom and the MRR model, then whether market making pays at all. It adds the derivations behind chapter 4's margin and chapter 5's spread that grows with the share of informed traders, and the source of the repo's "does making pay" test.

4. Avellaneda and Stoikov, *High-frequency trading in a limit order book*

Nine pages. The reservation price and the optimal spread of chapter 6, derived. Read the setup and the approximate solution and skip the control theory; then the wiki's page on optimal quoting models (`references/wiki/concepts/optimal-quoting-models.md`), which maps it, and its successors by Guéant and others, onto the template's `skew`. It adds why the skew is proportional to the position, the variance and the time left, and what the model assumes that Prosperity's data doesn't do.

5. Signals: Stoikov's micro-price, and the first three sections of Bouchaud's chapter 7

The micro-price paper (9 pages) and the book's §7.1–7.3 (PDF pp. 137–145). For an ML reader this is feature engineering: the book's imbalance predicts the next move, and the micro-price is the estimate that uses it. It adds the model behind chapter 3's micro-price, and the queue races that make imbalance informative. The wiki's page on book imbalance and the micro-price has what we measured (`references/wiki/concepts/book-imbalance-and-micro-price.md`).

6. Adverse selection measured: the first three sections of Bouchaud's chapter 11, then DeLise

§11.1–11.3 (PDF pp. 227–240), then *The negative drift of a limit order fill* (28 pages). Price impact and the response function, which are what `arena.analysis`'s markouts measure (chapter 5), then why the fills you get are, on average, the ones you wouldn't have wanted, and what that means for a backtester's fill rule (chapter 10).

7. Bouchaud's chapter 21, its third and fourth sections

§21.3–21.4 (PDF pp. 413–420). Limit against market orders, when to stay in the queue and when to go, and why backtests with phantom orders are biased. It adds the theory to chapter 10's fill models and queue positions.

8. Validation: *Pseudo-mathematics*, the wiki, then PBO

Pseudo-mathematics and financial charlatanism (34 pages), the wiki's page on validation and overfitting (`references/wiki/concepts/validation-and-overfitting.md`), then *The probability of backtest overfitting* (34 pages; CSCV on pp. 10–12). This is where an ML background transfers directly. It adds the full argument behind chapter 11: why the best of many backtests is usually luck, how long a backtest must be for a given number of trials, and the estimator `arena.grid` prints.

9. Then, as needed

- **Testing the best cell against the bot as is:** Bailey and López de Prado's *Deflated Sharpe ratio* (22 pages; the formula on pp. 7–9), Hansen's *Test for superior predictive ability* (16 pages) and Romano and Wolf's *Stepwise multiple testing* (36 pages; the studentized step-down on pp. 14–15). They add what chapter 11 states without proof: why the DSR's bar rises with the poor cells and why SPA's doesn't.
- **Sizing:** Thorp, *The Kelly criterion in blackjack, sports betting, and the stock market* (40 pages): the coin in §2, from PDF p. 3, fractional Kelly on pp. 24–26, estimated edges on pp. 29–30. It adds the reasons behind chapter 13's shading of an estimated edge.
- **Vocabulary:** Gould et al., *Limit order books*, §1–3, for the order book's terms in one place.
- **Reference books:** Cartea, Jaimungal and Penalva's *Algorithmic and high-frequency trading*, and the rest of Bouchaud and colleagues' book. Look things up in them; don't read them through.

These papers are about long samples of daily returns or about stylised markets. How far their numbers transfer to a few days of tick data in a competition is an open question, which is why the repo tests each one on Prosperity data before it trusts it.

Appendix E: The Arena, round by round

Written after the event. On 4 November 2026, when this skeleton was set down, none of it had happened; nothing below is a result.

This appendix will be the Arena's record, one section per round, each written from that round's handover note and its saved leaderboard (`data/arena/round<N>/HANDOVER.md`, `leaderboard.json`), with every number cited to them. The chapters draw on it in their "On the Arena" boxes; here it is in one place, in order.

What each round's section will hold

- **The rules:** the products, the limits, the data and the bot interface, and what changed from the round before, from the Technical Rules and the answers written into the "First hour" section of ON_THE_DAY.
- **The data:** what the public backtest data looked like, and what `arena.diagnose` and the premise test said of each product.
- **What we submitted:** every upload, its time, why it was made, and its local result next to its portal score.
- **The portal against our engine:** which fill model, queue, mark and penalty the portal matched, and how far its scores were from our backtests.
- **Scores and rank:** the public and the private run of the final bot, per product where the results show it, and our place on the round's leaderboard.
- **What we'd change:** what the round taught, and what we would do differently with the same five hours.

Round 1, Saturday 13:00–18:00

Written after the event.

Round 2, Saturday 19:00–24:00

Written after the event.

Round 3, Sunday 01:00–06:00

Written after the event.

Round 4, Sunday 07:00–12:00

Written after the event, with the final ranking, and how the four rounds combined into it.